CompBio Bootcamp

חוברת הכשרה מלאה לעוזרי מחקר בביולוגיה חישובית



מאפס לעוזר מחקר: הביולוגיה מוסברת מאפס דרך אנלוגיות ממדעי המחשב, האלגוריתמים ו-Python, בעברית, עם איורים, מעבדות חזותיות, דוגמאות קוד ותרגילים בסגנון LeetCode עם פתרונות מלאים.

11	11	30	108
מודולים מדורגים מקל ועד מומחה	מעבדות חזותיות כאיורים צעד-אחר-צעד	דוגמאות קוד מוכנות עם פלט והסבר	מקרי בדיקה גלויים ונסתרים

מהדורת אוקטובר 2026 · נבנה מתוך קוד המקור של הקורס האינטראקטיבי · Python 3

על החוברת ואיך להשתמש בה

CompBio Bootcamp הוא קורס ביולוגיה חישובית שרץ כולו בדפדפן: כל מודול מצמיד שיעור על הביולוגיה ועל האלגוריתם למעבדה חזותית אינטראקטיבית ולאתגר קוד בסגנון LeetCode, שמורץ על ידי CPython אמיתי שקומפל ל-WebAssembly (Pyodide). החוברת הזו היא המהדורה המודפסת והשלמה של הקורס: כל התוכן של 11 המודולים, בסדר המקורי מהקל לקשה, בלי קיצורים.

1 מבנה כל פרק

1. **פתיח המודול**: רמת הקושי, זמן משוער, הנושאים והפונקציה שתממשו.
2. **הביולוגיה בשפה של מתכנתים**: כל פרק נפתח בסעיף שמסביר את הביולוגיה מאפס, בלי שום רקע קודם, בעזרת אנלוגיות ממדעי המחשב (DNA כדיסק קשיח, mRNA כזיכרון RAM, ריבוזום כמפרש, מוטציה כ-bit flip, יישור כ-diff, תכנון דינמי כגיליון אלקטרוני). הסעיף כולל איור שמלווה את האנלוגיה, מילון מונחים ביולוגיה ↔ מדעי המחשב וסיכום בשורה אחת.
3. **השיעור**: הביולוגיה, ההסבר החישובי והאלגוריתמי, קופסאות סיבוכיות זמן/זיכרון, טיפים מקצועיים ומלכודות מהעולם האמיתי, בדיוק כפי שהם מופיעים באתר.
4. **המעבדה החזותית**: באתר זו אנימציה אינטראקטיבית. כאן היא הומרה לאיורים סטטיים צעד-אחר-צעד, עם אותם נתוני ברירת מחדל ואותם חישובים, ובסוף כל איור הסבר קצר מה אפשר לעשות בגרסה המקוונת.
5. **דוגמאות קוד**: תוכניות Python מוכנות, כל אחת עם הפלט המלא של ההרצה שלה ועם הסבר שלב-אחר-שלב.
6. **התרגיל**: הגדרת הבעיה, חתימת הפונקציה, דוגמאות, אילוצים, רמזים, כל הבדיקות הגלויות וכל הבדיקות הנסתרות (מקרי הקצה), והקוד ההתחלתי.
7. **פתרון עיון מלא**: הפתרון האופטימלי עם מספרי שורות, הסבר שורה-אחר-שורה, הנימוק לסיבוכיות, ואיך הפתרון עומד בכל מקרה קצה.

2 מוסכמות כתיבה

- **עברית מימין לשמאל** לכל ההסברים; **קוד, שמות פונקציות, סימוני סיבוכיות** (למשל O(n)) ו**מונחים מדעיים** (DNA, RNA, mRNA, FASTA, codon, ORF, k-mer, SNP) נשארים באנגלית, משמאל לימין ובגופן רוחב-קבוע, כך שסוגריים ופיסוק לעולם אינם מתהפכים.
- **איורי האנלוגיה** (המסומנים בתווית "איור") זורמים משמאל לימין, כמו הרצפים והקוד, עם תוויות בעברית; השלבים ממוספרים כדי שסדר הקריאה יהיה חד-משמעי.
- **רצפים** נכתבים תמיד משמאל לימין בכיוון 5′→3′, עם צבע קבוע לכל בסיס:

A אדנין · T תימין · G גואנין · C ציטוזין · U אורציל (RNA)

קופסאות הצד חוזרות בכל הפרקים ומסומנות באותה צורה:

🧬 הקשר ביולוגי

למה התופעה קיימת בתא, ואיפה פוגשים אותה במחקר.

⌘ התובנה החישובית

איך הביולוגיה מצטמצמת לפעולה על מחרוזות, מבני נתונים וסיבוכיות.

טיפ מקצועי

כלי או אידיום שחוסך עבודה בקוד מחקרי אמיתי.

מלכודות מהעולם האמיתי

באגים נפוצים: אותיות קטנות, שורות ב-FASTA, חפיפות, אינדקסים מ-0 או מ-1, הפניות משותפות.

בשורה אחת

סיכום של האנלוגיה הביולוגית במשפט אחד, בסוף סעיף "הביולוגיה בשפה של מתכנתים".

זמן O(…)

קופסת הסיבוכיות: איך זמן הריצה גדל עם גודל הקלט

זיכרון O(…)

וכמה זיכרון נוסף נדרש מעבר לקלט

3 הרצה והגשה

באתר יש שני כפתורים, והשיעורים מתייחסים אליהם: **הרצה** (⌘/Ctrl + Enter) בודקת את הפתרון מול הבדיקות הגלויות בלבד; **הגשה** (⌘/Ctrl + Shift + Enter) מריצה גם את הבדיקות הנסתרות, מקרי הקצה, ומסמנת את המודול כהושלם. בחוברת שתי קבוצות הבדיקות מודפסות במלואן בכל תרגיל, כדי שתוכלו לבדוק את עצמכם גם בלי האתר: הקלידו את הפונקציה בקובץ Python, קראו לה עם הקלטים שבטבלה והשוו לתוצאה הצפויה. ציון זמן של 8 שניות לכל הרצה באתר תופס לולאות אינסופיות; פתרונות שעומדים בסיבוכיות שבשיעור מסיימים הרבה לפני כן.

איך ללמוד מהחוברת

קראו את השיעור, עקבו אחרי האיורים עם עט ביד, הריצו את דוגמאות הקוד ושנו אותן, ואז פתרו את התרגיל לפני שאתם הופכים לעמוד הפתרון. בדקו את עצמכם מול הבדיקות הנסתרות; רק אחר כך קראו את ההסבר שורה-אחר-שורה והשוו לפתרון שלכם.

מפת הקורס: 11 מודולים מקל ועד מומחה

המודולים בנויים זה על זה: כלי ממודול מוקדם (למשל ההשלמה ההפוכה או החלון הנע) חוזר כאבן בניין במודולים המאוחרים. הזמן המשוער כולל את קריאת השיעור, הרצת הדוגמאות ופתרון התרגיל.

#	מודול	קושי	דק'	מעבדה חזותית	הפונקציה בתרגיל
1	הדוגמה המרכזית ומחרוזות	קל	25	אנימציית בועת שעתוק	`transcribe(dna_string)`
2	ניתוח רצפים ותכולת GC	קל	30	סטטיסטיקת הרכב והתכת DNA לפי טמפרטורה	`gc_content(dna)`
3	מוטציות ומרחק המינג	קל	30	השוואת גדילים, טרנזיציות/טרנסברזיות ו-Jukes–Cantor	`hamming_distance(seq1, seq2)`
4	חיפוש מוטיבים	בינוני	40	חלון נע צעד-אחר-צעד עם מונה השוואות	`find_motif(dna, motif)`
5	תרגום RNA לחלבון	בינוני	35	ריבוזום שעובר על קודונים, שרשרת חומצות אמינו וטבלת הקוד הגנטי	`translate_rna(rna_sequence)`
6	מסגרות קריאה פתוחות (ORFs)	בינוני-קשה	45	שלוש מסגרות קריאה עם קודוני התחלה/עצירה וסורק ORF	`find_first_orf(rna)`
7	ניקוד יישור רצפים	קשה	50	ניקוד עמודה-אחר-עמודה עם התאמה/אי-התאמה/פער וגרף מצטבר	`score_alignment(seq1, seq2)`
8	השלמה הפוכה (Reverse Complement)	בינוני-קשה	40	השלמה ואז היפוך, עם תוויות 5′/3′ וזיהוי פלינדרומים	`reverse_complement(dna)`
9	תדירויות k-mer	קשה	45	חלון k נע עם טבלת תדירויות ממוינת	`count_kmers(dna, k)`
10	מטריצות פרופיל ורצפי קונצנזוס	קשה	50	מטריצת פרופיל עמודה-אחר-עמודה עם סימון תיקו ושורת קונצנזוס	`consensus_sequence(dna_list)`
11	תכנון דינמי: מטריצת היישור	מומחה	60	רשת Needleman–Wunsch: אתחול, מילוי תא-אחר-תא ו-traceback	`init_dp_matrix(seq1, seq2, gap_penalty)`

סך הכול: 450 דקות של לימוד מודרך, 30 דוגמאות קוד ו-108 מקרי בדיקה.

תוכן עניינים

על החוברת ואיך להשתמש בה 2

מפת הקורס: 11 מודולים 4

מודול 1: הדוגמה המרכזית ומחרוזות 10

1. הביולוגיה בשפה של מתכנתים 10
   - מילון מונחים: ביולוגיה ↔ מדעי המחשב 11
2. הדוגמה המרכזית של הביולוגיה המולקולרית 12
3. נוקלאוטידים: האלפבית של החיים 12
   - גדילים, כיווניות והשלמה 12
4. מעבדה חזותית: צפו בשעתוק בזמן אמת 13
5. מחרוזות ב-Python 14
   - אי-שינוי: למה `dna[0] = "U"` נכשל 14
   - ארבע דרכים לשעתק 14
6. דוגמאות קוד 15
   - דוגמה 1: השלמה הפוכה (Reverse Complement) 15
   - דוגמה 2: מ-DNA לחלבון: שעתוק ותרגום של גן זעיר 16
7. המשימה שלכם 18

התרגיל: `transcribe(dna_string)` 19

פתרון עיון מלא: `transcribe` 21

מודול 2: ניתוח רצפים ותכולת GC 22

1. הביולוגיה בשפה של מתכנתים 22
   - מילון מונחים: ביולוגיה ↔ מדעי המחשב 23
2. מה מחזיק את הסליל הכפול? 23
3. טמפרטורת התכה (T_m) 24
   - למה זה חשוב לביולוגים 24
4. מעבדה חזותית: תכולת GC והתכת DNA 25
5. לולאות, ספירה ותבנית הצובר 26
   - חלופות בסגנון Python 26
   - תכנות הגנתי: חשבו על הקצוות 27
   - פענוח FASTA, שפת האם של קובצי רצפים 27
6. דוגמאות קוד 27
   - דוגמה 1: חישוב טמפרטורת התכה לפריימרים 27
   - דוגמה 2: GC skew: איתור נקודת ההתחלה של השכפול 28
   - דוגמה 3: קריאת קובץ multi-FASTA וספירת הרכב הבסיסים 29
7. המשימה שלכם 31

התרגיל: `gc_content(dna)` 32

פתרון עיון מלא: `gc_content` 34

מודול 3: מוטציות ומרחק המינג 35

1. הביולוגיה בשפה של מתכנתים 35
   - מילון מונחים: ביולוגיה ↔ מדעי המחשב 36
2. מוטציות נקודתיות 37
3. SNP: השונות שהופכת אתכם לייחודיים 37
4. מאי-התאמות למרחק אבולוציוני 37
5. מעבדה חזותית: השוואת שני גדילים 38
6. מעבר על שתי מחרוזות במקביל 39
   - Big-O: איך העבודה גדלה? 39
7. דוגמאות קוד 40
   - ▸ דוגמה 1: דוח מוטציות: מיקום וסוג של כל שינוי 40
   - ▸ דוגמה 2: מטריצת מרחקים אבולוציוניים (Jukes–Cantor) 41
8. המשימה שלכם 42

התרגיל: `hamming_distance(seq1, seq2)` 43

פתרון עיון מלא: `hamming_distance` 45

מודול 4: חיפוש מוטיבים 46

1. הביולוגיה בשפה של מתכנתים 46
   - מילון מונחים: ביולוגיה ↔ מדעי המחשב 47
2. גורמי שעתוק ואתרי קישור 47
   - למה מופעים חופפים חשובים 48
3. אלגוריתם החלון הנע 48
4. מעבדה חזותית: החלון הנע 49
5. מימוש ב-Python 50
   - סיבוכיות ואלגוריתמים טובים יותר 50
6. דוגמאות קוד 51
   - ▸ דוגמה 1: חיפוש אתרי הגבלה בפלסמיד 51
   - ▸ דוגמה 2: מוטיבים מנוונים עם קודי IUPAC וביטויים רגולריים 52
   - ▸ דוגמה 3: המילים השכיחות ביותר (k-mers) 53
7. המשימה שלכם 53

התרגיל: `find_motif(dna, motif)` 54

פתרון עיון מלא: `find_motif` 56

מודול 5: תרגום RNA לחלבון 57

1. הביולוגיה בשפה של מתכנתים 57
   - מילון מונחים: ביולוגיה ↔ מדעי המחשב 58
2. הקוד הגנטי: שלישיות של אותיות 59
   - איך הריבוזום עובד 60
3. מעבדה חזותית: הריבוזום בפעולה 60
4. מילונים: טבלאות חיפוש ב-Python 61
   - איך טבלת גיבוב עובדת 61

5. דוגמאות קוד 62
- דוגמה 1: בניית הקוד הגנטי והיפוך המילון 62
- דוגמה 2: תרגום בשלוש מסגרות הקריאה 63
- דוגמה 3: תרגום בטוח: DNA, אותיות קטנות וקודונים לא מוכרים 64

6. המשימה שלכם 64

התרגיל: `translate_rna(rna_sequence)` 65

פתרון עיון מלא: `translate_rna` 68

מודול 6: מסגרות קריאה פתוחות (ORFs) 70

1. הביולוגיה בשפה של מתכנתים 70
- מילון מונחים: ביולוגיה ↔ מדעי המחשב 71

2. התחלה ועצירה 72
- שש מסגרות קריאה 72
- למה ORF ארוך הוא סימן לגן? 72

3. מעבדה חזותית: מחפשים ORF 73

4. חיתוך מתקדם ולולאות עם צעד 74

5. דוגמאות קוד 75
- דוגמה 1: כל ה-ORFs בשש מסגרות הקריאה של DNA 75
- דוגמה 2: כמה ארוך ORF אקראי? סימולציה 77

6. המשימה שלכם 78

התרגיל: `find_first_orf(rna)` 79

פתרון עיון מלא: `find_first_orf` 82

מודול 7: ניקוד יישור רצפים 84

1. הביולוגיה בשפה של מתכנתים 84
- מילון מונחים: ביולוגיה ↔ מדעי המחשב 85

2. למה מיישרים רצפים? 85

3. איך מנקדים יישור 86
- יישור גלובלי ויישור מקומי 86

4. מעבדה חזותית: ניקוד יישור 87

5. מעבר על כמה רצפים במקביל 88
- אלגוריתם ניקוד איטרטיבי 88

6. דוגמאות קוד 89
- דוגמה 1: דוח יישור מלא: ציון, פערים ואחוז זהות 89
- דוגמה 2: יישור מרובה: רצף קונצנזוס ועמודות שמורות 90
- דוגמה 3: מציאת המיקום הטוב ביותר לקריאה קצרה 92

7. המשימה שלכם 92

התרגיל: `score_alignment(seq1, seq2)` 93

פתרון עיון מלא: `score_alignment` 96

מודול 8: השלמה הפוכה (Reverse Complement) 97

1. הביולוגיה בשפה של מתכנתים 97

- מילון מונחים: ביולוגיה ↔ מדעי המחשב 98
2. סליל כפול, שני כיוונים 99
- איפה זה פוגש אתכם במחקר 99
3. מעבדה חזותית: משלימים והופכים 100
4. היפוך מחרוזות ומיפוי תווים 100
- טבלאות תרגום 101
5. דוגמאות קוד 102
- דוגמה 1: איתור פלינדרומים ביולוגיים 102
- דוגמה 2: חיפוש מוטיב בשני הגדילים ותרגום קואורדינטות 103
- דוגמה 3: k-mers קנוניים: ספירה בלי תלות בגדיל 103
6. המשימה שלכם 104
התרגיל: `reverse_complement(dna)` 105
פתרון עיון מלא: `reverse_complement` 107

מודול 9: תדירויות k-mer 108
1. הביולוגיה בשפה של מתכנתים 108
- מילון מונחים: ביולוגיה ↔ מדעי המחשב 109
2. מהו k-mer? 110
3. למה k-mers חשובים? 110
4. מעבדה חזותית: ספירת k-mers 111
5. ספירה במילון 112
6. דוגמאות קוד 113
- דוגמה 1: גרף de Bruijn: הצעד הראשון בהרכבת גנום 113
- דוגמה 2: ספקטרום k-mer: כמה פעמים מופיע כל k-mer? 114
- דוגמה 3: השוואת רצפים לפי הרכב k-mer (מדד Jaccard) 116
7. המשימה שלכם 116
התרגיל: `count_kmers(dna, k)` 117
פתרון עיון מלא: `count_kmers` 120

מודול 10: מטריצות פרופיל ורצפי קונצנזוס 121
1. הביולוגיה בשפה של מתכנתים 121
- מילון מונחים: ביולוגיה ↔ מדעי המחשב 122
2. יישור מרובה ואזורים שמורים 122
3. מטריצת פרופיל ורצף קונצנזוס 123
- מקונצנזוס ל-PWM וללוגו רצף 123
4. מעבדה חזותית: בונים פרופיל 124
5. מערכים דו-ממדיים ב-Python 125
- בחירת מקסימום עם כלל לשבירת תיקו 126
6. דוגמאות קוד 126
- דוגמה 1: מטריצת פרופיל בפורמט של Rosalind 126
- דוגמה 2: תוכן מידע לכל עמודה: הבסיס ללוגו רצף 127
- דוגמה 3: סריקת רצף בעזרת מטריצת משקל-עמדה (PWM) 128

7. המשימה שלכם 129
התרגיל: `consensus_sequence(dna_list)` 130
פתרון עיון מלא: `consensus_sequence` 133

מודול 11: תכנון דינמי: מטריצת היישור 134

1. הביולוגיה בשפה של מתכנתים 134
- מילון מונחים: ביולוגיה ↔ מדעי המחשב 135
2. כמה יישורים יש? 136
- התובנה של Needleman ו-Wunsch (1970) 136
3. המטריצה ואתחולה 136
- מילוי ו-traceback 137
4. מעבדה חזותית: ממלאים את המטריצה 137
5. מטריצות ב-Python: יצירה נכונה 139
- אתחול השורה והעמודה הראשונות 139
6. דוגמאות קוד 140
- דוגמה 1: מילוי מלא של מטריצת Needleman–Wunsch 140
- דוגמה 2: Traceback: שחזור היישור עצמו 141
- דוגמה 3: Smith–Waterman: יישור מקומי בשינוי של שלוש שורות 142
7. המשימה שלכם 143
התרגיל: `init_dp_matrix(seq1, seq2, gap_penalty)` 145
פתרון עיון מלא: `init_dp_matrix` 148

נספח: האתר האינטראקטיבי, הפעלה מקומית וקיצורי מקלדת 149

סיום: מה הלאה? 151

מודול 1 · ביולוגיה × מדעי המחשב

הדוגמה המרכזית ומחרוזות

קל | כ-25 דקות | שעתוק | מחרוזות ב-Python | str.replace | `transcribe()`

מעבדה חזותית: אנימציית בועת שעתוק

כל תא בגופכם מחזיק את אותו דיסק קשיח: ספר הוראות באורך של כ-3.1 מיליארד אותיות, כתוב באלפבית של ארבע אותיות בלבד. במודול הזה תלמדו איך התא **מעתיק** קובץ אחד מהדיסק הזה לזיכרון העבודה שלו, ולמה מבחינת המחשב כל התהליך המרשים הזה הוא פעולה אחת על מחרוזת.

מה תלמדו

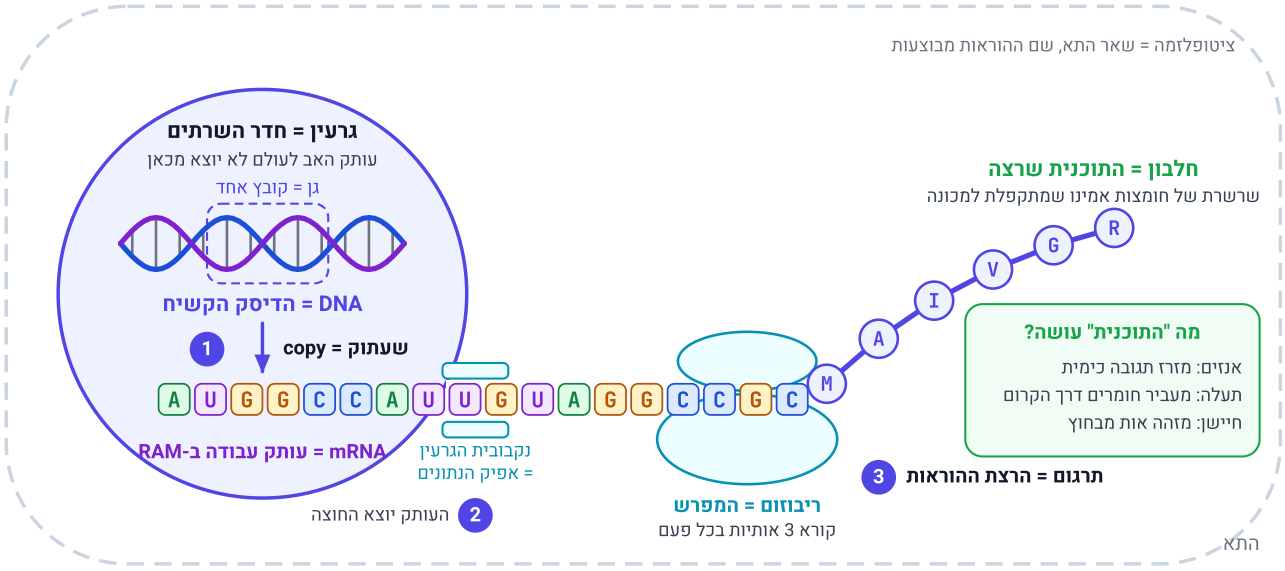
- הביולוגיה בשפה של מתכנתים: התא כמחשב, DNA כדיסק, mRNA כזיכרון, ריבוזום כמפרש
- ממה בנויים DNA ו-RNA ואיך בסיסים יוצרים זוגות
- איך תהליך השעתוק מעתיק גן מ-DNA ל-RNA שליח (mRNA)
- איך Python מייצגת רצפים כמחרוזות בלתי ניתנות לשינוי
- ארבע דרכים להמיר מחרוזת, ומה המחיר של כל אחת

1 הביולוגיה בשפה של מתכנתים

אם מעולם לא למדתם ביולוגיה, הנה כל מה שצריך כדי להתחיל, במונחים שאתם כבר מכירים. **DNA** הוא הדיסק הקשיח של התא: עותק אב של כל ההוראות, שמור בתוך **הגרעין** (חדר השרתים) ולעולם לא יוצא ממנו. **גן** הוא קובץ אחד על הדיסק הזה: ההוראות לבניית מכונה אחת. כשהתא צריך את המכונה, הוא לא שולח את הדיסק עצמו, אלא מעתיק את הקובץ ל-**mRNA**: עותק עבודה זמני, כמו קובץ שנטען לזיכרון ה-RAM. העותק יוצא דרך **נקבובית הגרעין**, האפיק שמחבר בין הגרעין לשאר התא, ושם מחכה **הריבוזום**: מפרש (interpreter) שקורא את העותק הוראה אחר הוראה ובונה ממנו **חלבון**, התוכנית שבאמת רצה ועושה את העבודה.

ההעתקה מהדיסק לזיכרון נקראת **שעתוק**, והיא כמעט העתקה מדויקת: ההבדל היחיד הוא שבעותק ה-RNA כל אות `T` נכתבת כ-`U`. בשפה של Python, השעתוק כולו הוא `dna.replace("T", "U")`, וזה בדיוק מה שתכתבו בסוף המודול. כל השאר בשיעור הזה, האלפבית של ארבע האותיות, הזיווג בין הגדילים והכיווניות, הוא הפירוט של אותו סיפור.

איור מפת התא כמחשב: מהדיסק הקשיח ועד התוכנית שרצה



זרימת המידע בתא: (1) בגרעין, ההוראות לבניית חלבון מועתקות מה-DNA לעותק עבודה זמני (mRNA); (2) העותק יוצא דרך נקבובית בקרום הגרעין; (3) הריבוזום קורא אותו שלוש אותיות בכל פעם ומרכיב חלבון. הגנום עצמו, כמו דיסק אב, נשאר בגרעין.

מילון מונחים: ביולוגיה ↔ מדעי המחשב

בביולוגיה	במדעי המחשב	למה הדימוי עובד
DNA (הגנום)	**דיסק קשיח / ROM**	אחסון ארוך-טווח של כל ההוראות; לא מריצים ממנו ישירות, רק מעתיקים.
גן	**קובץ אחד (או פונקציה אחת)**	יחידה אחת של הוראות, עם התחלה וסוף, שבונה מכונה אחת.
גרעין התא	**חדר השרתים**	המקום המוגן שבו שמור עותק האב, עם בקרת גישה.
mRNA	**עותק בזיכרון RAM**	עותק עבודה זמני, קצר-מועד, שנמחק אחרי השימוש.
שעתוק	**`copy` עם החלפת תו**	העתקה כמעט מדויקת: `T` הופך ל-`U`, וזה הכול.
ריבוזום	**מפרש (interpreter)**	קורא את העותק פקודה אחר פקודה (3 אותיות) ומבצע.
חלבון	**התוכנית שרצה**	התוצר הסופי שעושה עבודה: מזרז, מעביר, מזהה, בונה.

⚡ בשורה אחת

התא מעתיק קובץ (גן) מהדיסק (DNA) לזיכרון (mRNA), והמפרש (ריבוזום) מריץ אותו ובונה תוכנית (חלבון). מבחינת המחשב, ההעתקה היא החלפת כל `T` ב-`U` במחרוזת.

2 הדוגמה המרכזית של הביולוגיה המולקולרית

עכשיו, אחרי הדימויים, אותו סיפור במונחים המדויקים שבהם משתמשים במעבדה ובמאמרים.

בשנת 1958 תיאר פרנסיס קריק כיצד מידע רצפי זורם בתוך התא: מ-**DNA** אל **RNA**, ומשם אל **חלבון**. הכיוון ההפוך, מחלבון בחזרה לחומצת גרעין, לעולם אינו מתרחש.



DNA גם משכפל את עצמו (**שכפול**). נגיפים רטרו כמו HIV יכולים לעבור מ-RNA בחזרה ל-DNA (**שעתוק לאחור**).

- **שכפול** (DNA → DNA): העתקת הגנום כולו לפני שהתא מתחלק.
- **שעתוק** (DNA → RNA): העתקת גן אחד לשליח קצר-מועד, ה-mRNA.
- **תרגום** (RNA → חלבון): הריבוזום קורא את ה-mRNA שלוש אותיות בכל פעם (קודון אחד) ובונה שרשרת של חומצות אמינו.

3 נוקלאוטידים: האלפבית של החיים

נוקלאוטיד בנוי משלושה חלקים: קבוצת פוספט, סוכר בן חמישה פחמנים (דאוקסיריבוז ב-DNA, ריבוז ב-RNA) ו**בסיס** חנקני. רק הבסיס משתנה מנוקלאוטיד אחד למשנהו, ולכן אפשר לכתוב כל חומצת גרעין כמחרוזת מעל אלפבית זעיר.

בסיס	סמל	סוג	נקשר אל	קשרי מימן	נמצא ב-
אדנין	A	פורין (שתי טבעות)	T ב-DNA / U ב-RNA	2	DNA, RNA
גואנין	G	פורין	C	3	DNA, RNA
ציטוזין	C	פירימידין (טבעת אחת)	G	3	DNA, RNA
תימין	T	פירימידין	A	2	DNA בלבד
אורציל	U	פירימידין	A	2	RNA בלבד

למה RNA משתמש ב-U במקום ב-T?

ציטוזין מאבד לאט קבוצת אמין והופך לאורציל. אילו DNA היה מכיל U באופן רגיל, אנזימי התיקון לא היו יכולים להבחין בין C פגום לבין U אמיתי. השימוש ב-T (שהוא U עם קבוצת מתיל) ב-DNA מסמן כל U כנזק. RNA הוא מולקולה קצרת-מועד וחד-פעמית, ולכן הוא יכול להסתפק באורציל, שזול יותר לייצור.

גדילים, כיווניות והשלמה

DNA הוא **סליל כפול** שמורכב משני גדילים. הגדילים הם:

- **משלימים**: A תמיד מול T, ו-G תמיד מול C. מי שיודע גדיל אחד, יודע גם את השני.

- **אנטי-מקביליים**: לכל גדיל יש כיוון, שנכתב 3′→5′ לפי מספור הפחמנים בסוכר. שני הגדילים פונים לכיוונים הפוכים.

במהלך השעתוק, האנזים RNA פולימראז קורא את **גדיל התבנית** מ-3′ ל-5′ ובונה RNA מ-5′ ל-3′, כשהוא מצמיד לכל בסיס בתבנית את הבסיס המשלים לו. לכן ה-RNA החדש משלים לגדיל התבנית, כלומר הוא זהה לגדיל השני, **הגדיל המקודד**, פרט לכך שכל T מוחלף ב-U.

```
5′ גדיל מקודד   A T G G C C A T T G 3′
3′ גדיל תבנית   T A C C G G T A A C 5′
                ↓ פולימראז RNA
mRNA 5′         A U G G C C A U U G 3′
```

> **⌘ התובנה החישובית**
>
> בהינתן הגדיל המקודד, כל התהליך המולקולרי מצטמצם לפעולת מחרוזת אחת: **להחליף כל `T` ב- `U`**.
> ביואינפורמטיקה עובדת כך לעיתים קרובות: ברגע שמבינים את הביולוגיה, תהליך מסובך הופך להמרה פשוטה ומדויקת על מחרוזות.

---

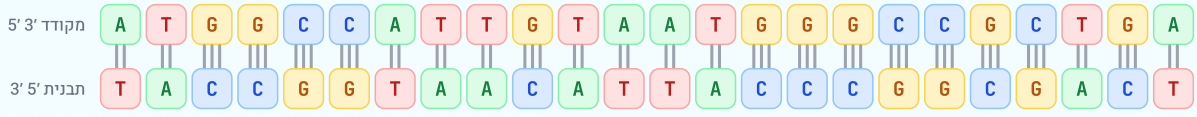
4 מעבדה חזותית: צפו בשעתוק בזמן אמת

הקלידו רצף כלשהו של גדיל מקודד ולחצו **שעתוק**. צפו בפולימראז פותח בועת שעתוק: הסליל הכפול נסגר מחדש מאחוריו, ומולקולת ה-mRNA גדלה בסיס אחר בסיס.

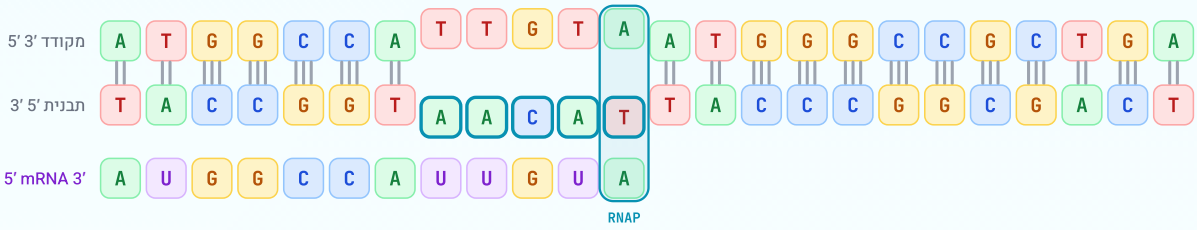
מעבדה חזותית · **שעתוק**

RNA פולימראז פותח בועת שעתוק, קורא את גדיל התבנית מ-3′ ל-5′ ומסנתז mRNA מ-5′ ל-3′.

שלב א · לפני השעתוק: סליל כפול סגור. כל בסיס בגדיל המקודד מזווג לבסיס המשלים לו בגדיל התבנית (2 או 3 קשרי מימן).

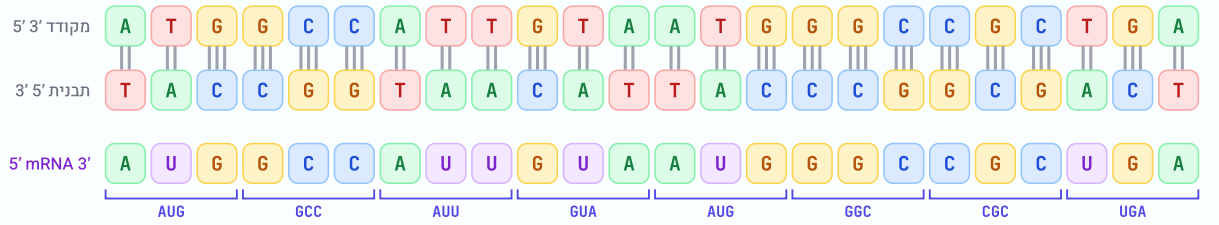


שלב ב · באמצע השעתוק (עמדה 12/24): RNA פולימראז (RNAP) פותח בועת שעתוק של 5 זוגות בסיסים, קורא את גדיל התבנית ומצמיד לכל בסיס את הבסיס המשלים ב-RNA. מאחוריו הסליל נסגר מחדש.



עמדה 12/24 · בסיס התבנית `T` מזווג עם `A` הנכנס · mRNA עד כה: `5′-AUGGCCAUUGUA-3′`

שלב ג · השעתוק הושלם: הסליל נסגר, וה-mRNA (זהה לגדיל המקודד, עם U במקום T) יוצא אל הריבוזום. למטה: חלוקה לקודונים.



השעתוק הושלם. קודונים: `AUG GCC AUU GUA AUG GGC CGC UGA`

הרצף מתחיל ב-AUG, קודון ההתחלה (Met), ולכן ריבוזום יכול להתחיל לתרגם מכאן. הקודון האחרון, UGA, הוא קודון עצירה.

`A` אדנין `T` תימין `G` גואנין `C` ציטוזין `U` אורציל (RNA)

בגרסה המקוונת: מקלידים גדיל מקודד (עד 60 בסיסים) ולוחצים **שעתוק** או **צעד**; הבועה נעה בסיס אחר בסיס, וה-mRNA גדל בזמן אמת. כאן מוצג הרצף הראשון של ברירת המחדל בשלושה שלבים.

5 מחרוזות ב-Python

מחרוזת (`str`) ב-Python היא **רצף בלתי ניתן לשינוי** (immutable) של תווים. זה מתאים מאוד ל-DNA: אפשר לגשת לפי אינדקס, לחתוך, לעבור בלולאה ולמדוד אורך בעזרת אותם כלים שמשמשים לרשימות.

```python
dna = "GATTACA"

len(dna)        # 7         -> number of nucleotides
dna[0]          # 'G'       -> 0-based indexing
dna[-1]         # 'A'       -> negative index counts from the end
dna[1:4]        # 'ATT'     -> slice [start, stop) - stop is excluded
dna[::-1]       # 'ACATTAG' -> reversed (step of -1)
"TTA" in dna    # True      -> substring membership test

for base in dna:            # iterate character by character
    print(base, end=" ")    # G A T T A C A
```

אי-שינוי: למה `dna[0] = "U"` נכשל

אי אפשר לשנות מחרוזת במקום. כל "שינוי" בונה מחרוזת חדשה:

```python
dna = "GATTACA"
dna[2] = "U"          # TypeError: 'str' object does not support item assignment
rna = dna.replace("T", "U")   # OK - returns a brand-new string
print(dna, rna)               # GATTACA GAUUACA
```

ארבע דרכים לשעתק

כל הארבע נכונות. הן נבדלות זו מזו בבהירות ובמהירות:

```python
# 1) Built-in method - clearest, implemented in C, O(n)
rna = dna.replace("T", "U")

# 2) Translation table - best when mapping MANY characters at once
table = str.maketrans("T", "U")
rna = dna.translate(table)

# 3) Comprehension + join - flexible; join builds the result in one pass
rna = "".join("U" if b == "T" else b for b in dna)

# 4) Manual accumulation - fine for learning, but repeated += on
#    immutable strings can copy the string each time (worst case O(n^2))
rna = ""
for b in dna:
    rna += "U" if b == "T" else b
```

PYTHON

זמן	זיכרון
O(n)	O(n)
כל בסיס נבדק פעם אחת	נוצרת מחרוזת חדשה באותו אורך

💡 **טיפ מקצועי: `str.translate` הוא סוס העבודה**

בהמשך תצטרכו את ההשלמה ההפוכה (reverse complement) של גדיל: החלפה של A↔T ושל C↔G, ואז היפוך. טבלת תרגום אחת מבצעת את ארבע ההחלפות במעבר יחיד, במהירות של C: `dna.translate(str.maketrans("ACGT", "TGCA"))[::-1]`.

⚠️ **מלכודות מהעולם האמיתי**

- **אותיות גדולות וקטנות**: קבצים מסוימים כותבים רצפים באותיות קטנות (soft-masking, לסימון אזורים חוזרים). `"t" != "T"`.
- **רווחים ושורות**: קובצי FASTA שוברים רצפים לשורות של 60–80 תווים. הסירו את תווי השורה החדשה לפני כל ניתוח.
- **קודי עמימות**: `N` פירושו "בסיס כלשהו". הקוד שלכם צריך להעביר אותו הלאה ללא שינוי, ולא לקרוס.

---

6 דוגמאות קוד

לפני שתכתבו פתרון משלכם, הנה דוגמאות מוכנות שמשתמשות באותם כלים ורעיונות. מתחת לכל דוגמה מופיע **הפלט המלא** של הרצת הקוד (כפי שהתקבל מ-Python 3), ואחריו ההסבר המפורט. מומלץ להקליד את הקוד בעצמכם, להריץ אותו, ואז לשנות אותו ולבדוק איך הפלט משתנה. בגרסת האתר כל דוגמה ניתנת להרצה בלחיצת כפתור.

דוגמה 1 השלמה הפוכה (Reverse Complement)

השלמה הפוכה היא אחת הפעולות הנפוצות ביותר בביואינפורמטיקה: היא מחשבת את הרצף של הגדיל השני, כפי שהוא

נקרא בכיוון 3′→5′ שלו.

```python
dna = "ATGCGTACCTTAG"

complement_table = str.maketrans("ACGT", "TGCA")
complement = dna.translate(complement_table)
reverse_complement = complement[::-1]

print("DNA:               ", dna)
print("Complement:        ", complement)
print("Reverse complement:", reverse_complement)
```

```
$ python example.py
DNA:               ATGCGTACCTTAG
Complement:        TACGCATGGAATC
Reverse complement: CTAAGGTACGCAT
```

איך זה עובד, שלב אחר שלב

1. `str.maketrans("ACGT", "TGCA")` בונה טבלת מיפוי: התו הראשון במחרוזת הראשונה ממופה לתו הראשון בשנייה, וכן הלאה. כך מתקבלים A→T, C→G, G→C ו-T→A.
2. `dna.translate(complement_table)` עובר על המחרוזת פעם אחת ומחליף כל תו לפי הטבלה. התוצאה היא הגדיל המשלים, שעדיין כתוב בכיוון 5′→3′.
3. `complement[::-1]` הופך את סדר התווים (חיתוך עם צעד `-1`). כך מקבלים את הגדיל המשלים בכיוון 3′→5′, כלומר את **ההשלמה ההפוכה**, הצורה שבה רצפים נשמרים במאגרי מידע.
4. שימו לב שכל פעולה יוצרת מחרוזת חדשה: `dna` המקורית לא משתנה, כי מחרוזות ב-Python אינן ניתנות לשינוי.
5. למה זה חשוב? גן יכול לשבת על כל אחד משני הגדילים. כלים כמו BLAST ותוכנות למיפוי קריאות בודקים תמיד גם את ההשלמה ההפוכה.

דוגמה 2 מ-DNA לחלבון: שעתוק ותרגום של גן זעיר

הדוגמה הזו מריצה את הדוגמה המרכזית מקצה לקצה: שעתוק של גדיל מקודד ל-mRNA, ואז תרגום של ה-mRNA לרצף

של חומצות אמינו.

```python
CODON_TABLE = {
    "AUG": "M", "GCC": "A", "AUU": "I", "GUA": "V",
    "GGC": "G", "CGC": "R", "UGA": "*", "UAA": "*", "UAG": "*",
}

def translate_rna(rna):
    protein = []
    for i in range(0, len(rna) - 2, 3):
        codon = rna[i:i + 3]
        amino_acid = CODON_TABLE.get(codon, "?")
        if amino_acid == "*":          # stop codon
            break
        protein.append(amino_acid)
    return "".join(protein)

coding_strand = "ATGGCCATTGTAATGGGCCGCTGAAAG"
mrna = coding_strand.replace("T", "U")
codons = [mrna[i:i + 3] for i in range(0, len(mrna), 3)]

print("mRNA:   ", mrna)
print("Codons: ", " ".join(codons))
print("Protein:", translate_rna(mrna))
```

```
$ python example.py
mRNA:    AUGGCCAUUGUAAUGGGCCGCUGAAAG
Codons:  AUG GCC AUU GUA AUG GGC CGC UGA AAG
Protein: MAIVMGR
```

איך זה עובד, שלב אחר שלב

1. `CODON_TABLE` הוא מילון (dict) שממפה כל קודון לקוד של חומצת אמינו באות אחת. הקוד הגנטי המלא כולל 64 קודונים; כאן מופיעים רק אלה שהדוגמה צריכה, והסימן `*` מסמן קודון עצירה.
2. `range(0, len(rna) - 2, 3)` מייצר את האינדקסים 0, 3, 6 וכן הלאה, כלומר את תחילתו של כל קודון. החסרת 2 מבטיחה שלא נחתוך קודון חלקי בסוף הרצף.
3. `rna[i:i + 3]` חותך שלוש אותיות. `CODON_TABLE.get(codon, "?")` מחזיר את חומצת האמינו, או `"?"` אם הקודון לא נמצא במילון, במקום לזרוק `KeyError`.
4. כשמגיעים לקודון עצירה, `break` יוצא מהלולאה, בדיוק כמו שהריבוזום משתחרר מה-mRNA. לכן הקודון `AAG` שמופיע אחרי `UGA` לא מתורגם.
5. את החלבון בונים כרשימה ומחברים בסוף עם `"".join(protein)`. זה יעיל יותר מחיבור חוזר של מחרוזות עם `+=`.
6. השורה `coding_strand.replace("T", "U")` היא השעתוק עצמו, אותו רעיון שתממשו בתרגיל כפונקציה עצמאית.

7 המשימה שלכם

עברו לסעיף **התרגיל** שבהמשך הפרק וממשו את `transcribe(dna_string)`. השתמשו ב**הרצה** כדי לבדוק את הדוגמאות הגלויות, ואז ב**הגשה** כדי לבדוק גם את מקרי הקצה הנסתרים.

בקוד מחקרי הייתם בדרך כלל קוראים ל-`Bio.Seq.Seq(dna).transcribe()` מהספרייה Biopython. מאחורי הקלעים היא עושה בדיוק את מה שאתם עומדים לכתוב.

מודול 1 · אתגר קוד בסגנון LEETCODE

✎ התרגיל: `transcribe(dna_string)`

קל | שעתוק | מחרוזות ב-Python | str.replace | כ-25 דקות

מחרוזת RNA היא מחרוזת מעל האלפבית `{A, C, G, U}`. בהינתן מחרוזת DNA `t` שהיא **הגדיל המקודד** של גן, תעתיק ה-RNA שלה, `u`, מתקבל מהחלפת כל מופע של `"T"` ב-`t` ב-`"U"`.

ממשו את `transcribe(dna_string)` כך שתחזיר את תעתיק ה-RNA כמחרוזת.

דוגמאות

דוגמה 1

קלט: `dna_string = "GATGGAACTTGACTACGTAAATT"`

פלט: `"GAUGGAACUUGACUACGUAAAUU"`

הסבר: כל T הופך ל-U; A, C ו-G נשארים ללא שינוי.

דוגמה 2

קלט: `dna_string = "TTTT"`

פלט: `"UUUU"`

אילוצים

- $0 \le \text{len(dna_string)} \le 10^5$
- `dna_string` מכילה רק את האותיות הגדולות `A`, `C`, `G`, `T`
- יש להחזיר `str`, לא רשימה

רמזים

💡 מחרוזות ב-Python אינן ניתנות לשינוי, ולכן תחזירו מחרוזת חדשה.

💡 הציצו במתודה המובנית `str.replace(old, new)`.

💡 מקרה קצה: מה צריך להיות התעתיק של מחרוזת ריקה?

בדיקות גלויות (הרצה)

אלה הבדיקות שכפתור **הרצה** באתר מריץ. הפתרון חייב לעבור את כולן.

קריאה	תוצאה צפויה
`transcribe("GATGGAACTTGACTACGTAAATT")`	`"GAUGGAACUUGACUACGUAAAUU"`
`transcribe("TTTT")`	`"UUUU"`
`transcribe("ACGACG")`	`"ACGACG"`

בדיקות נסתרות: מקרי קצה (הגשה)

כפתור **הגשה** מריץ גם את הבדיקות האלה. הן נועדו לתפוס קלטים ריקים, מקרי גבול, יעילות על קלט גדול וטיפול בשגיאות. כתבו את הפתרון כך שיעמוד בהן **לפני** שאתם מציצים בטבלה.

קריאה	תוצאה צפויה	מה נבדק
`transcribe("")`	`""`	קלט ריק
`transcribe("T")`	`"U"`	תו בודד
`transcribe("AAAACCCCGGGGTTTT")`	`"AAAACCCCGGGGUUUU"`	
`transcribe("ATGC" * 2500)`	`"AUGC" * 2500`	קלט גדול (10,000 תווים): יעילות

קוד התחלתי

כך נראה העורך כשפותחים את המודול. ממשו את גוף הפונקציה במקום `pass`.

```python
def transcribe(dna_string):
    """
    Transcribe a DNA coding strand into messenger RNA.

    >>> transcribe("GATTACA")
    'GAUUACA'
    """
    # Write your code here
    pass
```

🔑 פתרון עיון מלא: `transcribe`

הפתרון שעובר את כל הבדיקות, הגלויות והנסתרות. מספרי השורות בהסבר מתייחסים לקוד שלמעלה.

```python
def transcribe(dna_string):
    # RNA is the coding strand with every thymine replaced by uracil.
    return dna_string.replace("T", "U")
```

הסבר שורה אחר שורה

שורה 1 חתימת הפונקציה: `dna_string` היא הגדיל המקודד, מחרוזת מעל האלפבית `{A, C, G, T}`. הפונקציה מקבלת `str` ומחזירה `str`.

שורה 2 ההערה מסכמת את הביולוגיה בשורה אחת: ה-mRNA זהה לגדיל המקודד, פרט לכך שכל תימין (T) מוחלף באורציל (U).

שורה 3 `str.replace("T", "U")` עובר על המחרוזת פעם אחת, ב-C, ומחזיר מחרוזת חדשה שבה כל מופע של T הוחלף ב-U. המחרוזת המקורית לא משתנה, כי מחרוזות ב-Python אינן ניתנות לשינוי. אין צורך בלולאה, במונה או ברשימת ביניים.

⌘ למה זה הפתרון האופטימלי

זו הגרסה המהירה והקריאה ביותר מבין ארבע הדרכים שבשיעור: מעבר יחיד שממומש ב-C, זמן O(n) וזיכרון O(n) (המחרוזת החדשה). הגרסה עם `+=` עלולה להעתיק את המחרוזת שוב ושוב, והגרסה עם `maketrans` משתלמת רק כשממפים כמה תווים בבת אחת.

💡 איך הפתרון עומד במקרי הקצה הנסתרים

מחרוזת ריקה: `"".replace(...)` מחזירה `""`, כנדרש. רצף ללא T (`"ACGACG"`) חוזר ללא שינוי. רצף של 10,000 תווים מטופל במעבר אחד מהיר.

מודול 2 · ביולוגיה × מדעי המחשב

ניתוח רצפים ותכולת GC

קל | כ-30 דקות | יציבות DNA | טמפרטורת התכה | לולאות וספירה | `gc_content()`

מעבדה חזותית: סטטיסטיקת הרכב והתכת DNA לפי טמפרטורה

אם ה-DNA הוא רוכסן, לא כל השיניים שלו נאחזות באותה חוזקה. אחוז האותיות G ו-C בקטע DNA הוא מספר פשוט שמנבא כמה חום צריך כדי לפתוח את הרוכסן, עוזר לקבוע אם ניסוי PCR יצליח, ואפילו מרמז מאיזה יצור הגיע הרצף. במודול הזה תחשבו אותו בלולאה אחת.

מה תלמדו

- הביולוגיה בשפה של מתכנתים: הסליל הכפול כרוכסן, ותכולת GC כספירה על מחרוזת
- למה DNA עשיר ב-GC יציב יותר בחום
- מהי טמפרטורת התכה (T_m) ואיך מעריכים אותה
- תבנית הצובר: לולאות, מונים ו-`Counter`
- איך לטפל במקרי קצה בצורה הגנתית (קלט ריק, אותיות מעורבות)

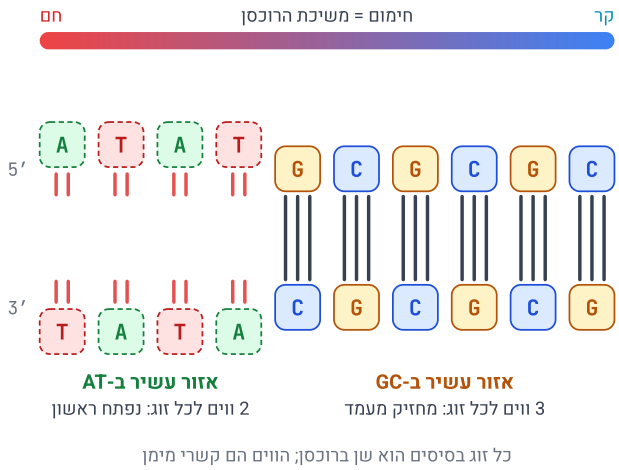
---

1 הביולוגיה בשפה של מתכנתים

דמיינו את הסליל הכפול כ**רוכסן** ארוך: כל "שן" ברוכסן היא זוג בסיסים, ושני הגדילים הם שני צידי הרוכסן. לא כל השיניים נאחזות באותה חוזקה. זוג `A` – `T` מחובר בשני ווים (שני קשרי מימן), וזוג `G` – `C` בשלושה. חימום הוא כמו למשוך את הרוכסן בכוח: קטעים עם הרבה `A` ו-`T` נפתחים ראשונים, וקטעים עשירים ב-`G` ו-`C` מחזיקים מעמד זמן רב יותר. הטמפרטורה שבה חצי מהרוכסנים כבר פתוחים נקראת **טמפרטורת ההתכה**.

מבחינת המחשב, **תכולת GC** היא סטטיסטיקה פשוטה על מחרוזת, כמו לספור כמה ביטים דולקים במחרוזת ביטים: עוברים על האותיות, סופרים כמה מהן `G` או `C`, ומחלקים באורך. המספר הזה מנבא כמה "כוח" (חום) יידרש כדי לפתוח את הרוכסן, ולכן ביולוגים בודקים אותו לפני כל ניסוי שבו צריך להצמיד או להפריד גדילים, למשל PCR. בהמשך השיעור תראו גם למה המספר הזה משתנה בין מינים ואיך משתמשים בו לבקרת איכות.

איור הסליל הכפול כרוכסן: למה GC מחזיק חזק יותר



הסטטיסטיקה שהמחשב מחשב

```
dna = "ATATGCGCGC"
```

כמה אותיות הן G או C	`G + C = 6`
אורך המחרוזת	`N = 10`
האחוז	`GC = 6 / 10 × 100 = 60%`

`A:2` `T:2` `G:3` `C:3`

GC גבוה ⇒ יותר ווים ⇒ צריך יותר חום להפריד
ולכן תכולת GC מנבאת את טמפרטורת ההתכה

משמאל: עשרה זוגות בסיסים כעשר שיני רוכסן. זוגות A–T מחוברים בשני ווים (קשרי מימן), זוגות G–C בשלושה; כשמחממים, הצד העשיר ב-AT נפתח ראשון. מימין: תכולת ה-GC היא ספירה פשוטה על המחרוזת, והיא מנבאת כמה חום יידרש כדי לפתוח את הרוכסן כולו.

מילון מונחים: ביולוגיה ↔ מדעי המחשב

בביולוגיה	במדעי המחשב	למה הדימוי עובד
זוג בסיסים	**שן אחת ברוכסן**	יחידה אחת שמחברת את שני הצדדים; הגדילים מחוברים רק דרך הזוגות.
קשר מימן	**וו קטן**	חיבור חלש שאפשר לפתוח; ככל שיש יותר ווים, קשה יותר להפריד.
A–T לעומת G–C	**2 ווים לעומת 3**	זה כל ההבדל: G–C פשוט נאחז חזק יותר.
תכולת GC	`count` חלקי `len`	אחוז התווים מסוג מסוים במחרוזת, כמו צפיפות ביטים דולקים.
התכה (דנטורציה)	**פתיחת הרוכסן**	הפרדת שני הגדילים על ידי חום; מתחילה באזורים ה"חלשים".
טמפרטורת התכה	**סף העומס**	הטמפרטורה שבה חצי מהמולקולות כבר נפתחו; עולה עם תכולת ה-GC.

בשורה אחת

DNA הוא רוכסן, זוגות G–C הם שיניים עם וו נוסף, ותכולת GC היא פשוט אחוז השיניים החזקות: ספירה אחת על המחרוזת שמנבאת כמה חום צריך כדי לפתוח את הרוכסן.

2 מה מחזיק את הסליל הכפול?

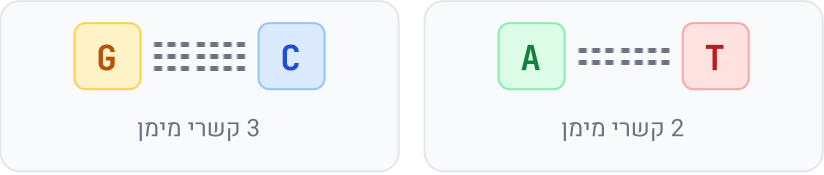
נעבור מהרוכסן לכימיה: מהם בדיוק ה"ווים" שמחזיקים את שני הגדילים יחד.

שני סוגים של כוחות לא-קוולנטיים מחזיקים את שני גדילי ה-DNA יחד:

- **קשרי מימן** בין בסיסים מזווגים: **ל-A=T יש 2** ו **ל-G≡C יש 3**.

- **ערימת בסיסים** (base stacking): הבסיסים השטוחים והארומטיים נערמים כמו מטבעות ומושכים את שכניהם מלמעלה ומלמטה. הערימה תורמת למעשה יותר ליציבות מאשר קשרי המימן, וערימות שמכילות G·C הן מהחזקות ביותר.

שני האפקטים פועלים באותו כיוון: **ככל שיש יותר G ו-C, נדרשת יותר אנרגיה כדי להפריד את הגדילים.**



3 טמפרטורת התכה (T_m)

כשמחממים DNA דו-גדילי, הגדילים נפרדים בסופו של דבר (דנטורציה, או "התכה"). **טמפרטורת ההתכה** היא הטמפרטורה שבה מחצית מהמולקולות הפכו לחד-גדיליות. שני קירובים קלאסיים:

כלל Wallace — אוליגונוקלאוטידים קצרים, פחות מ-14 נוקלאוטידים

$$T_m = 2\cdot(A + T) + 4\cdot(G + C) \quad °C$$

נוסחת תכולת GC — רצפים ארוכים יותר

$$T_m = 64.9 + 41 \cdot (G + C - 16.4) / N \quad °C$$

כלים מקצועיים (למשל Primer3) משתמשים בתרמודינמיקה של שכנים קרובים (nearest-neighbour): הם מסכמים את אנרגיית הערימה של כל זוג בסיסים סמוכים ומתקנים לפי ריכוז המלח. הנוסחאות שלמעלה הן הערכה ראשונה טובה.

למה זה חשוב לביולוגים

הקשר	למה תכולת GC חשובה
תכנון פריימרים ל-PCR	פריימרים מתוכננים עם 40–60% GC וטמפרטורת T_m של כ-52–65 °C, ולשני הפריימרים בזוג צריכות להיות טמפרטורות דומות. פריימר עשיר מדי ב-AT נופל מהתבנית; פריימר עשיר מדי ב-GC נקשר למקומות הלא נכונים.
ביולוגיה של גנומים	תכולת GC משתנה מאוד בין מינים: Plasmodium falciparum (טפיל המלריה) כ-19%, האדם כ-41%, Streptomyces coelicolor כ-72%. אפשר להשתמש בה כ"טביעת אצבע" כדי לזהות גנים שהגיעו בהעברה אופקית.
בקרת ביטוי גנים	איי CpG הם מקטעים עשירים ב-GC ליד הפרומוטורים של גנים אנושיים רבים. מצב המתילציה שלהם עוזר להפעיל או לכבות את הגנים האלה.
בקרת איכות בריצוף	קריאות Illumina עם GC קיצוני מיוצגות בחסר (GC bias). כלים כמו FastQC משרטטים את אחוז ה-GC בכל קריאה כדי לזהות זיהומים.

4 מעבדה חזותית: תכולת GC והתכת DNA

ערכו את הרצף, או צרו רצף אקראי עם הטיית GC לבחירתכם. אחר כך **העלו את הטמפרטורה**. במודל המפושט הזה, כל זוג בסיסים נמס כשהטמפרטורה עוברת סף מקומי שנקבע לפי תכולת ה-GC בסביבתו. אזורים עשירים ב-AT נפתחים ראשונים, בדומה ל"נשימה" של DNA אמיתי.

מעבדה חזותית · תכולת GC והתכת DNA

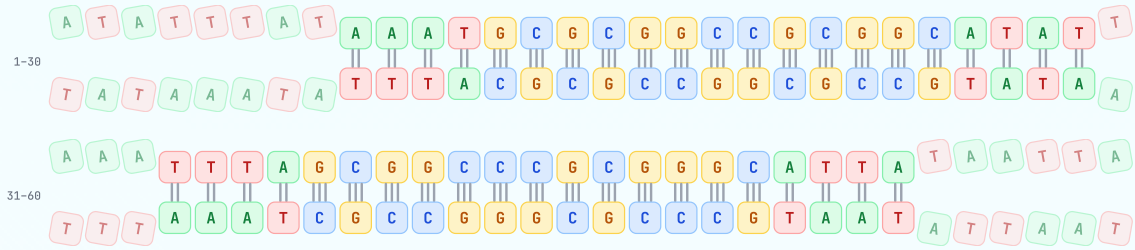
ההרכב קובע את היציבות. העלו את הטמפרטורה וצפו באזורים העשירים ב-AT נמסים ראשונים.

אורך	תכולת GC	T_M משוערת	קשרי מימן
60 bp	43.3%	71.5 °C נוסחת GC	146

A 28% | T 28% | G 23% | C 20%

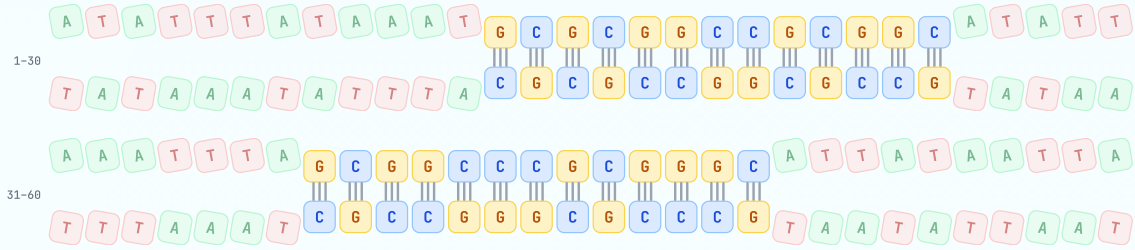
התכה הדרגתית: אותו סליל בשלוש טמפרטורות. זוג בסיסים נמס (נפתח ומתרחק) כשהטמפרטורה עובדת סף מקומי שתלוי בתכולת ה-GC בסביבתו.

50 °C



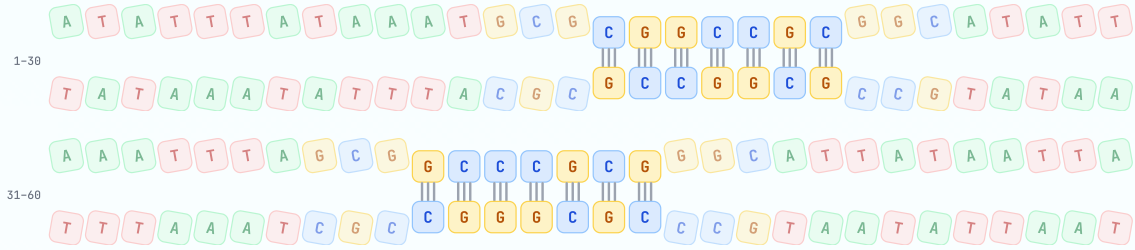
50 °18/60 · **C** זוגות בסיסים נמסו (30%). נמס חלקית: "בועות" נפתחות קודם באזורים העשירים ב-AT.

70 °C



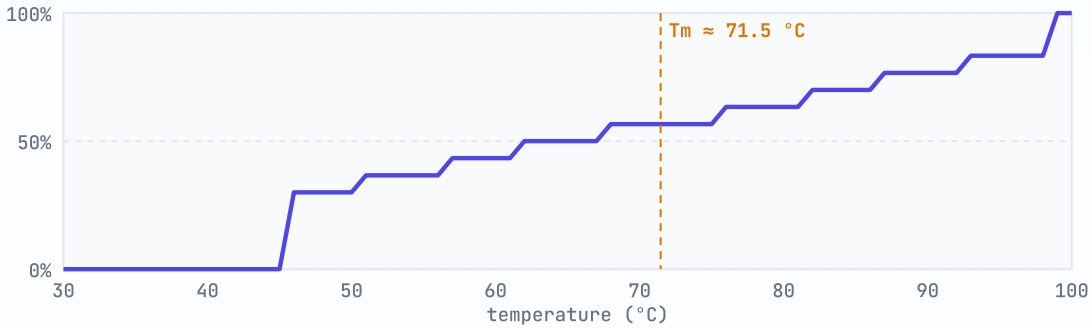
70 °34/60 · **C** זוגות בסיסים נמסו (57%). ברובו חד-גדילי: עברנו את נקודת האמצע של ההתכה.

90 °C



90 °46/60 · **C** זוגות בסיסים נמסו (77%). ברובו חד-גדילי: עברנו את נקודת האמצע של ההתכה.

עקומת התכה: שיעור זוגות הבסיסים שנמסו כפונקציה של הטמפרטורה, לפי אותו מודל מפושט. האזור המרכזי העשיר ב-GC נפתח אחרון.



מודל מפושט: כל זוג נמס מעל $45 + 50 \cdot f$ °C, כאשר f הוא שיעור ה-GC בחלון של 9 bp (ועוד 3 °C אם הזוג עצמו הוא G·C). התכה אמיתית היא תהליך שיתופי, שתלוי בריכוז המלח ובאנרגיות הערימה של שכנים קרובים.

בגרסה המקוונת: עורכים את הרצף או מגרילים רצף אקראי עם הטיית GC לבחירתכם, וגוררים את מחוון הטמפרטורה (30–100 °C). הסטטיסטיקות וההתכה מתעדכנות מיד. אפשר גם לטעון דוגמה מ-Rosalind.

5 לולאות, ספירה ותבנית הצובר

רוב הסטטיסטיקות על רצפים פועלות לפי אותה תבנית: **מאתחלים מונה, עוברים על הרצף פעם אחת, מעדכנים את המונה ובסוף משלבים את התוצאה.**

```python
dna = "AGCTATAG"

gc = 0                      # 1. initialise an accumulator
for base in dna:            # 2. visit every element once
    if base == "G" or base == "C":
        gc += 1             # 3. update the accumulator
percent = gc / len(dna) * 100   # 4. combine -> 37.5
```

חלופות בסגנון Python

```python
# Membership test on a small string instead of chained ==
gc = sum(1 for base in dna if base in "GC")

# str.count runs in C - two quick passes
gc = dna.count("G") + dna.count("C")

# Count every symbol at once (handy for full composition)
from collections import Counter
counts = Counter(dna)       # Counter({'A': 3, 'T': 2, 'G': 2, 'C': 1})
gc = counts["G"] + counts["C"]
```

⌘ חילוק ב-Python 3

`/` הוא חילוק אמיתי ותמיד מחזיר `float`: `3 / 8 == 0.375`. ואילו `//` הוא חילוק שלמים (עיגול כלפי מטה): `3 // 8 == 0`. סקריפטים ביואינפורמטיים רבים מתקופת Python 2 דיווחו בשקט על תכולת GC של 0 בגלל ההבדל הזה.

תכנות הגנתי: חשבו על הקצוות

לפני שכותבים את הלוגיקה המרכזית, כדאי לרשום את הקלטים שעלולים לשבור אותה:

- **רצף ריק**: `len(dna) == 0` גורם ל-`ZeroDivisionError`. החליטו על תשובה הגיונית (כאן: `0.0`).
- **אותיות קטנות**: `"gc"` הם עדיין G ו-C. נרמלו קודם בעזרת `dna.upper()`.
- **נקודה צפה**: `1/3*100` שווה `33.33333333333333`. הבדיקות משוות עם סובלנות, ולכן אל תעגלו בתוך הפונקציה. עגלו רק כשמציגים ערך.

פענוח FASTA, שפת האם של קובצי רצפים

```python
fasta = """>Rosalind_6404
CCTGCGGAAGATCGGCACTAGAATAGCCAGAACCGTTTCTCTGAGGCTTCCGGCCTTCCC
TCCCACTAATAATTCTGAGG"""

lines = fasta.strip().splitlines()
header = lines[0][1:]              # drop the '>'
seq = "".join(lines[1:])           # sequence lines are wrapped - re-join them
print(header, len(seq))            # Rosalind_6404 80
```

זמן `O(n)`
מעבר אחד (או שני מעברים מהירים עם str.count)

זיכרון `O(1)`
מספר קבוע של מונים, בלי קשר לגודל הקלט

6 דוגמאות קוד

לפני שתכתבו פתרון משלכם, הנה דוגמאות מוכנות שמשתמשות באותם כלים ורעיונות. מתחת לכל דוגמה מופיע **הפלט המלא** של הרצת הקוד (כפי שהתקבל מ-Python 3), ואחריו ההסבר המפורט. מומלץ להקליד את הקוד בעצמכם, להריץ אותו, ואז לשנות אותו ולבדוק איך הפלט משתנה. בגרסת האתר כל דוגמה ניתנת להרצה בלחיצת כפתור.

דוגמה 1 **חישוב טמפרטורת התכה לפריימרים**

בתכנון ניסוי PCR צריך לוודא שלפריימרים יש טמפרטורת התכה מתאימה. הפונקציה הזו בוחרת אוטומטית את הנוסחה

המתאימה לאורך הפריימר.

```python
def melting_temp(primer):
    primer = primer.upper()
    n = len(primer)
    at = primer.count("A") + primer.count("T")
    gc = primer.count("G") + primer.count("C")
    if n < 14:
        return 2 * at + 4 * gc            # Wallace rule
    return 64.9 + 41 * (gc - 16.4) / n    # GC-based formula

primers = {
    "short":   "ATGCGTAC",
    "M13_fwd": "GTAAAACGACGGCCAGT",
    "M13_rev": "AGCGGATAACAATTTCACACAGG",
}

for name, seq in primers.items():
    print(f"{name:<8} {seq:<24} len={len(seq):>2}  Tm={melting_temp(seq):5.1f} C")
```

```
$ python example.py
short    ATGCGTAC                 len= 8  Tm= 24.0 C
M13_fwd  GTAAAACGACGGCCAGT        len=17  Tm= 47.1 C
M13_rev  AGCGGATAACAATTTCACACAGG  len=23  Tm= 53.5 C
```

איך זה עובד, שלב אחר שלב

1. `primer.upper()` מנרמל את הרצף לאותיות גדולות, כך שגם פריימר שנכתב באותיות קטנות ייספר נכון.
2. `primer.count(...)` סופר כל בסיס במעבר מהיר שממומש ב-C. את A ו-T סוכמים יחד (2 קשרי מימן לכל זוג), ואת G ו-C יחד (3 קשרי מימן).
3. `if n < 14` בוחר בין הנוסחאות: לפריימרים קצרים משתמשים בכלל Wallace, ולארוכים בנוסחה שמבוססת על תכולת ה-GC ביחס לאורך.
4. המילון `primers` מחזיק שם ורצף לכל פריימר, והלולאה `for name, seq in primers.items()` עוברת על הזוגות לפי סדר ההכנסה.
5. ה-f-string מעצב את הפלט כטבלה: `{name:<8}` מיישר לשמאל ברוחב 8 תווים, ו-`{...:5.1f}` מציג מספר עשרוני עם ספרה אחת אחרי הנקודה.
6. קראו את התוצאה: לזוג פריימרים שעובד יחד צריכות להיות טמפרטורות קרובות, בדרך כלל בהפרש של עד כ-5 מעלות. בין שני פריימרי M13 מתקבל כאן הפרש של כ-6 מעלות, אבל הנוסחאות הפשוטות האלה גסות. לפני שמתכננים מחדש, בודקים עם כלי מדויק יותר כמו Primer3.

דוגמה 2 — איתור נקודת ההתחלה של השכפול: GC skew

GC skew הוא ההפרש המצטבר בין מספר ה-G למספר ה-C לאורך הגנום. בחיידקים, הנקודה שבה הוא מגיע למינימום

מסמנת לרוב את **ori**, נקודת ההתחלה של השכפול.

```python
genome = "CATGGGCATCGGCCATACGCC"

skew = [0]
for base in genome:
    if base == "G":
        skew.append(skew[-1] + 1)
    elif base == "C":
        skew.append(skew[-1] - 1)
    else:
        skew.append(skew[-1])

print("Skew:", skew)

lowest = min(skew)
positions = [i for i, value in enumerate(skew) if value == lowest]
print("Minimum skew", lowest, "at positions", positions)
```

```
$ python example.py
Skew: [0, -1, -1, -1, 0, 1, 2, 1, 1, 1, 0, 1, 2, 1, 0, 0, 0, 0, -1, 0, -1, -2]
Minimum skew -2 at positions [21]
```

איך זה עובד, שלב אחר שלב

1. הרשימה `skew` מתחילה ב-`[0]`: לפני שקראנו בסיס כלשהו, ההפרש הוא אפס. לכן בסוף יהיו בה `len(genome) + 1` ערכים.
2. בכל צעד לוקחים את הערך האחרון, `skew[-1]`, ומוסיפים 1 עבור G, מחסירים 1 עבור C, או משאירים אותו כפי שהוא עבור A ו-T. זו שוב תבנית הצובר, רק שהפעם שומרים את כל ערכי הביניים.
3. `min(skew)` מוצא את הערך הנמוך ביותר במעבר אחד על הרשימה.
4. רשימת ההבנה (list comprehension) עם `enumerate` אוספת את כל העמדות שבהן מתקבל המינימום, כי ייתכן שיש יותר מאחת.
5. הרעיון הביולוגי: במהלך השכפול, אחד הגדילים נשאר חד-גדילי זמן רב יותר, ובו ציטוזין נוטה לעבור דאמינציה. כך נוצר מחסור ב-C בחצי אחד של הגנום, והמעבר בין החצאים מסמן את ה-ori.

דוגמה 3 קריאת קובץ multi-FASTA וספירת הרכב הבסיסים

בקבצים אמיתיים יש בדרך כלל כמה רצפים, וכל אחד מהם שבור לכמה שורות. הדוגמה קוראת קובץ multi-FASTA

ומדפיסה את הרכב הבסיסים של כל רשומה.

PYTHON

```python
from collections import Counter

fasta = """>seq_alpha
ATATTTAAGC
GCATAT
>seq_beta
GGCCGCGTAA
CCGG
>seq_gamma
ATGCATGCAT"""

records = {}
name = None
for line in fasta.splitlines():
    line = line.strip()
    if not line:
        continue
    if line.startswith(">"):
        name = line[1:]
        records[name] = []
    else:
        records[name].append(line)

for name, parts in records.items():
    seq = "".join(parts)
    counts = Counter(seq)
    composition = ", ".join(f"{b}={counts[b]}" for b in "ACGT")
    print(f"{name:<10} length={len(seq):>2}  {composition}")
```

```
$ python example.py
seq_alpha  length=16  A=6, C=2, G=2, T=6
seq_beta   length=14  A=2, C=5, G=6, T=1
seq_gamma  length=10  A=3, C=2, G=2, T=3
```

איך זה עובד, שלב אחר שלב

1. `fasta.splitlines()` מפרק את הטקסט לשורות, ו-`line.strip()` מסיר רווחים ותווי שורה חדשה. שורות ריקות מדולגות בעזרת `continue`.
2. שורה שמתחילה ב-`>` היא כותרת: `line[1:]` מסיר את התו `>`, ובמילון `records` נפתחת רשימה ריקה עבור הרשומה החדשה.
3. כל שורה אחרת שייכת לרשומה האחרונה שנפתחה, ולכן מוסיפים אותה לרשימה `records[name]`.
4. אחרי הקריאה, `"".join(parts)` מחבר את השורות השבורות לרצף רציף אחד.
5. `Counter(seq)` סופר את כל התווים במעבר אחד ומחזיר מילון מיוחד. `counts[b]` מחזיר 0 עבור בסיס שלא הופיע, בלי `KeyError`.
6. מכאן הדרך לתכולת GC קצרה: מחברים את `counts["G"]` ו-`counts["C"]` ומחלקים באורך. את זה תממשו בתרגיל, כולל מקרי הקצה.

7 המשימה שלכם

ממשו את `gc_content(dna)`. **הבדיקות הנסתרות** כוללות מחרוזת ריקה וקלט עם אותיות גדולות וקטנות, אז טפלו במקרי הקצה שתוארו למעלה.

מודול 2 · אתגר קוד בסגנון LEETCODE

✎ התרגיל: `gc_content(dna)`

קל | יציבות DNA | טמפרטורת התכה | לולאות וספירה | כ-30 דקות

תכולת ה-GC של מחרוזת DNA היא אחוז הסמלים בה שהם `"C"` או `"G"`. לדוגמה, `"AGCTATAG"` מכילה 3 בסיסי G/C מתוך 8, ולכן תכולת ה-GC שלה היא `37.5`.

ממשו את `gc_content(dna)` כך שתחזיר את תכולת ה-GC **כאחוז** (`float` בין 0 ל-100).

- ספרו בלי תלות בגודל האות: גם `"g"` ו-`"c"` נספרים.
- אם הרצף ריק, החזירו `0.0`. אל תזרקו חריגה.

דוגמאות

דוגמה 1

קלט: `dna = "AGCTATAG"`

פלט: `37.5`

הסבר: C ,G ו-G הם 3 מתוך 8 בסיסים, כלומר 3 / 8 × 100 = 37.5.

דוגמה 2

קלט: `dna = "GGCC"`

פלט: `100.0`

דוגמה 3

קלט: `dna = ""`

פלט: `0.0`

הסבר: אין בסיסים, ולכן אין במה לחלק. הגנו מפני ZeroDivisionError.

אילוצים

- $0 \le \text{len(dna)} \le 10^5$
- התווים לקוחים מתוך `ACGTacgt`
- תשובות מתקבלות בשגיאה מוחלטת של עד 10^{-3}

רמזים

💡 ספרו כמה תווים הם G או C, וחלקו באורך הכולל.

💡 `dna.upper()` מנרמלת את גודל האותיות בצעד אחד.

💡 בדקו `if not dna:` לפני החילוק.

בדיקות גלויות (הרצה)

אלה הבדיקות שכפתור **הרצה** באתר מריץ. הפתרון חייב לעבור את כולן.

קריאה	תוצאה צפויה
`gc_content("AGCTATAG")`	`37.5`
`gc_content("GGCC")`	`100.0`
`gc_content("ATATAT")`	`0.0`

בדיקות נסתרות: מקרי קצה (הגשה)

כפתור **הגשה** מריץ גם את הבדיקות האלה. הן נועדו לתפוס קלטים ריקים, מקרי גבול, יעילות על קלט גדול וטיפול בשגיאות. כתבו את הפתרון כך שיעמוד בהן **לפני** שאתם מציצים בטבלה.

קריאה	תוצאה צפויה	מה נבדק
`gc_content("")`	`0.0`	קלט ריק
`gc_content("CCACCCTCGTGGTATGGCTAGGCATTCAGG…"` (מחרוזת באורך 87)`)`	`60.91954`	
`gc_content("gcGCat")`	`66.666667`	אותיות קטנות וגדולות
`gc_content("G")`	`100.0`	תו בודד
`gc_content("ACGT" * 2500)`	`50.0`	קלט גדול (10,000 תווים): יעילות

תוצאות עשרוניות מושוות בסובלנות של 10^{-3}.

קוד התחלתי

כך נראה העורך כשפותחים את המודול. ממשו את גוף הפונקציה במקום `pass`.

```python
def gc_content(dna):
    """
    Return the GC content of a DNA sequence as a percentage (0-100).

    >>> gc_content("AGCTATAG")
    37.5
    """
    # Write your code here
    pass
```

🔑 פתרון עיון מלא: `gc_content`

הפתרון שעובר את כל הבדיקות, הגלויות והנסתרות. מספרי השורות בהסבר מתייחסים לקוד שלמעלה.

```python
def gc_content(dna):
    if not dna:                 # guard: avoid ZeroDivisionError
        return 0.0
    dna = dna.upper()           # make the count case-insensitive
    gc = sum(1 for base in dna if base in "GC")
    return gc / len(dna) * 100
```

הסבר שורה אחר שורה

שורות 2–3 **שמירה (guard) מפני קלט ריק.** `if not dna:` הוא `True` עבור מחרוזת ריקה (ערך "שקרי"). במקרה כזה מחזירים `0.0` מיד, לפני כל חילוק, וכך נמנעים מ-`ZeroDivisionError`. שימו לב להחזרת `float` ולא `int`, כדי שסוג התוצאה יהיה אחיד.

שורה 4 **נרמול גודל האותיות.** `dna.upper()` מחזיר עותק באותיות גדולות, כך ש-`"g"` ו-`"c"` ייספרו כמו `"G"` ו-`"C"`. השמה חוזרת למשתנה `dna` היא רק שם חדש לאובייקט חדש; הקלט המקורי של הקורא לא משתנה.

שורה 5 **תבנית הצובר בשורה אחת.** ביטוי הגנרטור `1 for base in dna if base in "GC"` מייצר 1 עבור כל בסיס שהוא G או C, ו-`sum` מחבר. הבדיקה `base in "GC"` מחליפה את התנאי המסורבל `base == "G" or base == "C"`.

שורה 6 **אחוז.** `/` הוא חילוק אמיתי ב-Python 3 ומחזיר `float`; מכפילים ב-100. לא מעגלים בתוך הפונקציה: הבדיקות משוות עם סובלנות של 10^{-3}, ועיגול מוקדם רק מוסיף שגיאה.

⌘ למה זה הפתרון האופטימלי

מעבר יחיד על הרצף, O(n) בזמן ו-O(1) בזיכרון נוסף (מונה אחד; `upper()` יוצר עותק אחד). חלופה מהירה באותה סיבוכיות: `dna.count("G") + dna.count("C")`, שני מעברים ב-C.

💡 איך הפתרון עומד במקרי הקצה הנסתרים

`""` → `0.0` (בלי חריגה); `"gcGCat"` → 66.67 בזכות `upper()`; `"G"` → 100.0; רצף של 10,000 בסיסים עם 50% GC → 50.0 בדיוק.

מודול 3 · ביולוגיה × מדעי המחשב

מוטציות ומרחק המינג

קל | כ-30 דקות | SNPs | מרחק אבולוציוני | Big-O ו-zip | `hamming_distance()`

מעבדה חזותית: השוואת גדילים, טרנזיציות/טרנסברזיות ו-Jukes–Cantor

אין שני עותקים זהים לחלוטין של גנום: בכל העתקה נופלות כמה "שגיאות הקלדה", ביטים שהתהפכו. מדידת **מידת** השוני בין שני רצפים היא נקודת המוצא לזיהוי וריאנטים, לגנטיקה של אוכלוסיות ולבניית עץ החיים, והמדד הפשוט ביותר הוא בדיוק מה ש-diff עושה: לספור עמדות שונות.

מה תלמדו

- הביולוגיה בשפה של מתכנתים: מוטציות כ-bit flips ומרחק המינג כ-diff
- סוגי מוטציות: טרנזיציות, טרנסברזיות ואינדלים
- מהו SNP ולמה יש מיליונים כאלה אצל כל אחד מאיתנו
- איך ספירת אי-התאמות הופכת למרחק אבולוציוני
- מעבר על שני רצפים במקביל, ומה באמת פירוש O(n)

1 הביולוגיה בשפה של מתכנתים

כשתא מתחלק הוא מעתיק את כל הגנום, ולפעמים נופלת שגיאת הקלדה: אות אחת מוחלפת באחרת. זו **מוטציה נקודתית**, המקבילה הביולוגית ל-bit flip, ביט אחד שהתהפך בזיכרון. רוב ההחלפות האלה לא משנות דבר, חלקן גורמות לבאג (מחלה), ומעטות הן דווקא שיפור. כשמשווים את העותק המקורי לעותק החדש, **מרחק המינג** הוא פשוט מספר העמדות השונות: בדיוק מה ש-`diff` או `cmp` מדווחים כשמשווים שני קבצים באותו אורך.

יש שני סוגים של "שגיאות הקלדה". **טרנזיציה** מחליפה אות באות שדומה לה בצורתה הכימית (כמו להקליד `0` במקום `O`): קל לעשות אותה, ולכן היא שכיחה. **טרנסברזיה** מחליפה אות באות מהמשפחה האחרת (כמו `X` במקום `O`), ונדירה יותר. **SNP** הוא "שגיאת הקלדה" ותיקה כל כך שאנשים רבים נושאים אותה, כמו שתי גרסאות של אותו קובץ הגדרות שמסתובבות באוכלוסייה. ככל ששני גנומים צברו יותר הבדלים, כך הם התפצלו מוקדם יותר, וזו הדרך שבה סופרים "כמה זמן עבר" מאז האב הקדמון המשותף.

איור **מוטציות כ-bit flips: diff בין המקור לעותק**



שני עותקים של אותו קטע DNA, מיושרים אינדקס מול אינדקס. בכל עמדה בודקים אם האותיות שוות (=) או שונות (≠), כותבים 0 או 1, וסוכמים: שבע עמדות שונות, ולכן מרחק המינג הוא 7. צבע האות בעותק מסמן את סוג ההחלפה.

מילון מונחים: ביולוגיה ↔ מדעי המחשב

בביולוגיה	במדעי המחשב	למה הדימוי עובד
שכפול הגנום	**העתקת קובץ**	התא מעתיק את כל ההוראות לפני שהוא מתחלק; העתקה מושלמת היא המטרה, לא תמיד התוצאה.
מוטציה נקודתית	**bit flip / שגיאת הקלדה**	תו אחד השתנה; הכול סביבו נשאר במקום.
טרנזיציה / טרנסברזיה	**החלפה ב"דומה" / ב"שונה"**	החלפה בין אותיות דומות קלה יותר, ולכן נפוצה יותר; כמו טעויות הקלדה בין מקשים סמוכים.
SNP	**גרסה נפוצה של אותו קובץ**	עמדה שבה אנשים שונים נושאים אותיות שונות; הבסיס לבדיקות גנטיות.
מרחק המינג	**`sum(a != b)` / פלט של diff**	מספר העמדות השונות בין שתי מחרוזות באותו אורך.
אינדל	**הכנסה/מחיקה של תו**	מזיז את כל מה שאחריו; לכן מרחק המינג לא מתאים לו, וצריך יישור (מודולים 7 ו-11).

⚡ בשורה אחת

מוטציה היא ביט שהתהפך בעותק, ומרחק המינג הוא ה-diff: עוברים על שתי המחרוזות במקביל וסופרים כמה עמדות שונות.

2 מוטציות נקודתיות

ועכשיו לפרטים: אילו סוגי "שגיאות הקלדה" קיימים, ומה כל אחת מהן עושה לחלבון.

מוטציה נקודתית משנה נוקלאוטיד בודד. היא נגרמת משגיאות של הפולימראז, מנזק כימי, מקרינת UV ומכשלים במנגנוני התיקון. התמרות (substitutions) מתחלקות לשני סוגים:

טרנזיציה (Transition)

A ↔ G C ↔ T

פורין מוחלף בפורין, או פירימידין בפירימידין. הצורות דומות, ולכן בפועל הן **שכיחות פי 2 בערך**.

טרנסברזיה (Transversion)

A ↔ C G ↔ T ...

פורין מוחלף בפירימידין, או להפך. יש פי שניים טרנסברזיות אפשריות, ובכל זאת הן מתרחשות לעיתים רחוקות יותר.

בתוך גן שמקודד לחלבון, ההשפעה של התמרה תלויה בקוד הגנטי:

- **סינונימית (שקטה)**: הקודון משתנה, אבל חומצת האמינו לא (למשל `GAG` → `GAA`, שניהם Glu).
- **Missense**: חומצת האמינו משתנה. הדוגמה הקלאסית היא **אנמיה חרמשית**: טרנסברזיה אחת של A→T בגן HBB הופכת את `GAG` (Glu) ל-`GTG` (Val) בעמדה 6 של β-גלובין.
- **Nonsense**: קודון עצירה מוקדם קוטע את החלבון.

קיימים גם **אינדלים** (indels: הכנסות והשמטות), אבל הם מזיזים את היישור. מרחק המינג מניח ששני הרצפים כבר מיושרים ובאותו אורך, ולכן הוא סופר רק התמרות. אינדלים דורשים אלגוריתמי יישור שתפגשו בהמשך (Needleman–Wunsch, Smith–Waterman).

3 SNP: השונות שהופכת אתכם לייחודיים

פולימורפיזם של נוקלאוטיד בודד (SNP) הוא עמדה בגנום שבה אנשים שונים נושאים לעיתים קרובות בסיסים שונים (לפי המוסכמה, לאלל הנדיר יש שכיחות של 1% לפחות באוכלוסייה). כל גנום אנושי שונה מגנום הייחוס בכ-**4–5 מיליון** עמדות. SNPs עומדים בבסיס מחקרי אסוציאציה גנומיים (GWAS), הסקת מוצא, פרמקוגנומיקה וזיהוי פורנזי.

4 מאי-התאמות למרחק אבולוציוני

מחלקים את מרחק המינג d באורך היישור L ומקבלים את **p-distance**, שיעור העמדות השונות:

$$p = d_H / L$$

תיקון Jukes–Cantor (1969) — מתחשב בפגיעות מרובות

$$d_{JC} = -\tfrac{3}{4} \cdot \ln\left(1 - \tfrac{4p}{3}\right)$$

למה בכלל לתקן? לאורך זמנים ארוכים, אותה עמדה יכולה לעבור מוטציה יותר מפעם אחת (A→G→A). שינויים כאלה בלתי נראים, ולכן p הגולמי מעריך בחסר את המספר האמיתי של ההתמרות. מודל JC69 הופך שרשרת מרקוב פשוטה כדי

לשחזר אותו. כש-p מתקרב ל-0.75, הערך הצפוי בין שני רצפים אקראיים, המרחק שואף לאינסוף: האות רווי.

---

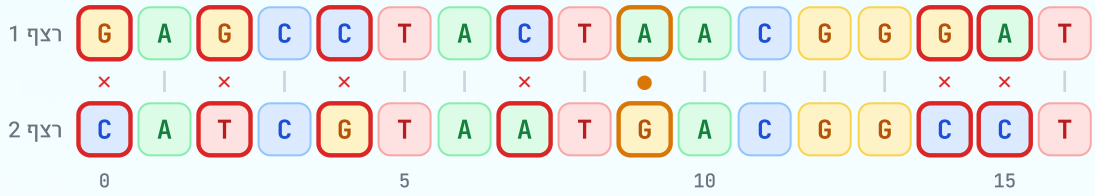
5 מעבדה חזותית: השוואת שני גדילים

ערכו כל אחד מהגדילים, לחצו **מוטציה** כדי להוסיף מוטציה נקודתית אקראית, או **סריקה** כדי לצפות באלגוריתם עובר על שתי המחרוזות וסופר הבדלים. טרנזיציות מסומנות בכתום, וטרנסברזיות באדום.

● מעבדה חזותית · מרחק המינג

השוואה עמדה אחר עמדה: כל אי-התאמה היא מוטציה נקודתית אחת.

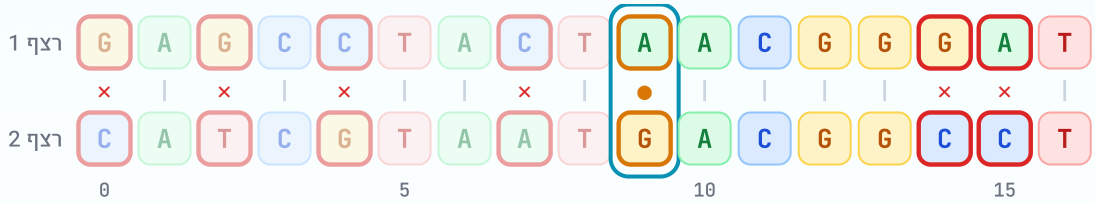
השוואה עמדה אחר עמדה של שני הגדילים. | התאמה · ● טרנזיציה (כתום) · ✕ טרנסברזיה (אדום).



D_H	P-DISTANCE	KES–CANTOR…	טרנזיציות	טרנסברזיות	TS / TV
7	0.412	0.597	1	6	0.17

7 מתוך **17** העמדות המיושרות שונות. אי-ההתאמות נמצאות באינדקסים 0, 2, 4, 7, 9, 14, 15.

סריקה: הלולאה באמצע העבודה, בעמדה i = 9. עמדות שכבר נבדקו מעומעמות, והעמדה הנוכחית מוקפת.



`i = 9` · `A` vs `G` ← **אי-התאמה (טרנזיציה)** · **מרחק עד כה = 5**

בסוף הסריקה: 17 השוואות עבור 17 עמדות, השוואה אחת לכל עמדה. לכן האלגוריתם הוא **O(n)**. מרחק המינג = **7**.

בגרסה המקוונת: עורכים את שני הרצפים, לוחצים **סריקה** כדי לצפות בלולאה עוברת עמדה אחר עמדה, או **מוטציה** כדי להוסיף מוטציה נקודתית אקראית לרצף 2 (לפעמים היא דווקא מבטלת הבדל קיים: מוטציה חוזרת). אם האורכים שונים, המעבדה מסבירה למה הפונקציה חייבת לזרוק ValueError.

6 מעבר על שתי מחרוזות במקביל

```python
s, t = "GAGCCTACTAACGGGAT", "CATCGTAATGACGGCCT"

# Index-based - works everywhere, but noisy
d = 0
for i in range(len(s)):
    if s[i] != t[i]:
        d += 1

# zip() pairs elements: ('G','C'), ('A','A'), ...
d = 0
for a, b in zip(s, t):
    if a != b:
        d += 1

# Idiomatic one-liner: True counts as 1 when summed
d = sum(a != b for a, b in zip(s, t))     # 7

# Need positions too? enumerate + zip
diffs = [i for i, (a, b) in enumerate(zip(s, t)) if a != b]
```

> ⚠️ **המלכודת השקטה של `zip`**
>
> `zip` **עוצר בקלט הקצר יותר בלי שום אזהרה**. הביטוי `sum(a != b for a, b in zip("ACGT", "AC"))` מחזיר `0`, וזה שגוי. מרחק המינג אינו מוגדר לרצפים באורכים שונים, ולכן בדקו זאת במפורש וזרקו `raise ValueError(...)`. ב-Python 3.10 ומעלה אפשר גם לכתוב `zip(s, t, strict=True)`.

Big-O: איך העבודה גדלה?

סיבוכיות זמן מתארת איך זמן הריצה גדל ביחס לגודל הקלט n, תוך התעלמות מגורמים קבועים. מרחק המינג בודק כל עמדה בדיוק פעם אחת, ולכן הוא **O(n)**, כלומר **לינארי**: אם מכפילים את אורך הרצף, העבודה בערך מוכפלת.

מחלקה	דוגמה	N = 3×10⁹ (הגנום האנושי)
`O(1)`	קריאת `dna[42]`	מיידי
`O(n)`	מרחק המינג, תכולת GC	שניות
`O(n log n)`	מיון k-mers	דקות
`O(n²)`	השוואה נאיבית של כל הזוגות	יותר מגיל היקום

O(n)
זמן
השוואה אחת לכל עמדה מיושרת

O(1)
זיכרון
zip וגנרטורים הם עצלים, ולכן לא נוצרים עותקים

7 דוגמאות קוד

לפני שתכתבו פתרון משלכם, הנה דוגמאות מוכנות שמשתמשות באותם כלים ורעיונות. מתחת לכל דוגמה מופיע **הפלט המלא** של הרצת הקוד (כפי שהתקבל מ-Python 3), ואחריו ההסבר המפורט. מומלץ להקליד את הקוד בעצמכם, להריץ אותו, ואז לשנות אותו ולבדוק איך הפלט משתנה. בגרסת האתר כל דוגמה ניתנת להרצה בלחיצת כפתור.

דוגמה 1 דוח מוטציות: מיקום וסוג של כל שינוי

בניתוח וריאנטים לא מספיק לדעת כמה הבדלים יש; רוצים לדעת איפה הם ומאיזה סוג. הדוגמה מפיקה דוח מוטציות ומחשבת את היחס בין טרנזיציות לטרנסברזיות (Ts/Tv).

PYTHON

```python
PURINES = {"A", "G"}

def classify(a, b):
    if a == b:
        return None
    same_class = (a in PURINES) == (b in PURINES)
    return "transition" if same_class else "transversion"

ref = "GAGCCTACTAACGGGAT"
alt = "CATCGTAATGACGGCCT"

ts = tv = 0
for i, (a, b) in enumerate(zip(ref, alt)):
    kind = classify(a, b)
    if kind:
        print(f"pos {i:>2}: {a} -> {b}  {kind}")
        if kind == "transition":
            ts += 1
        else:
            tv += 1

print(f"transitions={ts}, transversions={tv}, Ts/Tv={ts / tv:.2f}")
```

```
$ python example.py
pos  0: G → C  transversion
pos  2: G → T  transversion
pos  4: C → G  transversion
pos  7: C → A  transversion
pos  9: A → G  transition
pos 14: G → C  transversion
pos 15: A → C  transversion
transitions=1, transversions=6, Ts/Tv=0.17
```

איך זה עובד, שלב אחר שלב

1. `PURINES` היא קבוצה (set), ולכן בדיקת השייכות `a in PURINES` מהירה מאוד. כל בסיס שאינו פורין הוא פירימידין.
2. `classify(a, b)` מחזירה `None` כשהבסיסים זהים. אחרת היא בודקת אם שניהם מאותה מחלקה: הביטוי `(a in PURINES) == (b in PURINES)` הוא `True` כששניהם פורינים או ששניהם פירימידינים, כלומר

כשמדובר בטרנזיציה.

3. `enumerate(zip(ref, alt))` מחזיר בכל צעד את האינדקס יחד עם זוג התווים המיושרים. הפירוק `i, (a, b)` פותח את שני המבנים בבת אחת.

4. `if kind:` מדלג על התאמות, כי `None` נחשב ערך שקרי. עבור כל אי-התאמה מדפיסים שורה ומגדילים את המונה המתאים.

5. בסוף מחשבים את יחס Ts/Tv. בנתוני אדם אמיתיים הוא בדרך כלל כ-2.0–2.1 בגנום כולו, וערך נמוך בהרבה הוא סימן אזהרה לשגיאות ריצוף. כאן היחס נמוך כי מדובר ברצפים קצרים לדוגמה, ולא בנתונים ביולוגיים.

דוגמה 2 מטריצת מרחקים אבולוציוניים (Jukes–Cantor)

בפילוגנטיקה בונים מטריצה של מרחקים בין כל זוג מינים, ומשם עץ. הדוגמה מחשבת מטריצה כזו עם תיקון Jukes–Cantor.

PYTHON

```python
import math

species = {
    "human":   "ACGTTGCAAGTCCATGCATA",
    "chimp":   "ACGTTGCAAGTCCATGCGTA",
    "gorilla": "ACGTTGCATGTCCATACGTA",
    "mouse":   "ACCTTGGAAGTACCTGAGTC",
}

def p_distance(s, t):
    diffs = sum(a != b for a, b in zip(s, t))
    return diffs / len(s)

def jukes_cantor(p):
    if p == 0:
        return 0.0
    if p >= 0.75:
        return float("inf")
    return -0.75 * math.log(1 - 4 * p / 3)

names = list(species)
print(" " * 8 + "".join(f"{n:>9}" for n in names))
for a in names:
    row = [jukes_cantor(p_distance(species[a], species[b])) for b in names]
    print(f"{a:<8}" + "".join(f"{d:>9.3f}" for d in row))
```

```
$ python example.py
            human    chimp  gorilla    mouse
human       0.000    0.052    0.167    0.471
chimp       0.052    0.000    0.107    0.383
gorilla     0.167    0.107    0.000    0.572
mouse       0.471    0.383    0.572    0.000
```

איך זה עובד, שלב אחר שלב

1. `p_distance` סופרת אי-התאמות בעזרת `sum(a != b for ...)` ומחלקת באורך. זהו מרחק המינג מנורמל.

2. `jukes_cantor` מיישמת את הנוסחה `-0.75 * log(1 - 4p/3)`. כש-`p` מגיע ל-0.75 הלוגריתם אינו מוגדר, ולכן מחזירים `float("inf")`, כלומר אות רווי. כש-`p` שווה 0 מחזירים `0.0` ישירות, כדי שהאלכסון לא יוצג כ-`-0.000`.
3. `list(species)` מחזיר את שמות המינים לפי סדר ההכנסה למילון, וכך סדר השורות זהה לסדר העמודות.
4. הלולאה החיצונית עוברת על השורות, ורשימת ההבנה הפנימית מחשבת את המרחק לכל עמודה. הסיבוכיות היא $O(k^2 \cdot L)$ עבור k מינים ברצפים באורך L.
5. העיצוב `{d:>9.3f}` מיישר כל מספר לימין ברוחב 9 תווים, עם שלוש ספרות אחרי הנקודה, כך שהטבלה יוצאת מסודרת.
6. קראו את התוצאה: האדם והשימפנזה הכי קרובים, הגורילה מעט רחוקה יותר, והעכבר רחוק מכולם, בדיוק כמו בעץ האמיתי. מטריצה כזו היא הקלט לאלגוריתמים כמו UPGMA ו-Neighbor-Joining.

---

8 המשימה שלכם

ממשו את `hamming_distance(seq1, seq2)`. הבדיקות הנסתרות בודקות שאתם **זורקים** `ValueError` כשהאורכים שונים.

מודול 3 · אתגר קוד בסגנון LEETCODE

התרגיל: `hamming_distance(seq1, seq2)`

קל | SNPs | מרחק אבולוציוני | zip ו-Big-O | כ-30 דקות

בהינתן שתי מחרוזות DNA `s` ו-`t` באורך שווה, **מרחק המינג** $d_H(s, t)$ הוא מספר העמדות שבהן הסמלים שלהן שונים.

ממשו את `hamming_distance(seq1, seq2)` כך שתחזיר את הספירה הזו כ-`int`.

אם לשני הרצפים יש **אורכים שונים**, המרחק אינו מוגדר: זרקו `ValueError`.

דוגמאות

דוגמה 1

קלט: `seq1 = "GAGCCTACTAACGGGAT"`
`seq2 = "CATCGTAATGACGGCCT"`

פלט: `7`

הסבר: אי-התאמות באינדקסים 0, 2, 4, 7, 9, 14, 15.

דוגמה 2

קלט: `seq1 = "AAAA", seq2 = "AAAA"`

פלט: `0`

דוגמה 3

קלט: `seq1 = "ACGT", seq2 = "ACG"`

פלט: `ValueError`

הסבר: האורכים 4 ו-3 שונים.

אילוצים

- $0 \le$ `len(seq1), len(seq2)` $\le 10^5$
- יש להחזיר `int`
- יש לזרוק `ValueError` אם `len(seq1) != len(seq2)`

רמזים

💡 השוו קודם את האורכים, וזרקו `raise ValueError("...")` אם הם שונים.

💡 `zip(seq1, seq2)` מחזיר זוגות של תווים מיושרים.

💡 ב-Python `True + True == 2`, ולכן `sum()` על ערכים בוליאניים סופר אותם.

בדיקות גלויות (הרצה)

אלה הבדיקות שכפתור **הרצה** באתר מריץ. הפתרון חייב לעבור את כולן.

קריאה	תוצאה צפויה
`hamming_distance("GAGCCTACTAACGGGAT", "CATCGTAATGACGGCCT")`	7
`hamming_distance("AAAA", "AAAA")`	0
`hamming_distance("ACGT", "TGCA")`	4
`hamming_distance("ACGT", "ACG")`	`raise ValueError`

בדיקות נסתרות: מקרי קצה (הגשה)

כפתור **הגשה** מריץ גם את הבדיקות האלה. הן נועדו לתפוס קלטים ריקים, מקרי גבול, יעילות על קלט גדול וטיפול בשגיאות. כתבו את הפתרון כך שיעמוד בהן **לפני** שאתם מציצים בטבלה.

קריאה	תוצאה צפויה	מה נבדק
`hamming_distance("", "")`	0	קלט ריק
`hamming_distance("A", "G")`	1	תו בודד
`hamming_distance("GGACGGATTCTG", "AGGACGGATTCT")`	9	
`hamming_distance("AC", "ACGT")`	`raise ValueError`	קלט לא תקין: חייב לזרוק `ValueError`
`hamming_distance("A" * 10000, "A" * 9999 + "T")`	1	קלט גדול (10,000 תווים): יעילות

קוד התחלתי

כך נראה העורך כשפותחים את המודול. ממשו את גוף הפונקציה במקום `pass`.

```python
def hamming_distance(seq1, seq2):
    """
    Count point mutations between two aligned sequences of equal length.
    Raise ValueError if the lengths differ.

    >>> hamming_distance("GAGCCTACTAACGGGAT", "CATCGTAATGACGGCCT")
    7
    """
    # Write your code here
    pass
```

🔑 פתרון עיון מלא: `hamming_distance`

הפתרון שעובר את כל הבדיקות, הגלויות והנסתרות. מספרי השורות בהסבר מתייחסים לקוד שלמעלה.

```python
def hamming_distance(seq1, seq2):
    if len(seq1) != len(seq2):
        raise ValueError(
            f"Sequences must have equal length ({len(seq1)} != {len(seq2)})"
        )
    return sum(a != b for a, b in zip(seq1, seq2))
```

הסבר שורה אחר שורה

שורות 2–5 **בדיקת תקינות לפני הכול.** מרחק המינג מוגדר רק לרצפים באותו אורך, ולכן אם `len(seq1) != len(seq2)` זורקים `ValueError`. ההודעה היא f-string שמציגה את שני האורכים, מה שעוזר מאוד כשמאתרים באג בצנרת ניתוח. בלי הבדיקה הזו, `zip` היה עוצר בשקט בקלט הקצר ומחזיר תשובה שגויה.

שורה 6 **ספירת אי-התאמות בשורה אחת.** `zip(seq1, seq2)` מצמיד תו לתו: `('G','C'), ('A','A'), …`. הביטוי `a != b` הוא בוליאני, וב-Python `True` שווה 1 בחיבור, ולכן `sum(...)` סופר בדיוק את העמדות השונות. הגנרטור עצל: לא נבנית רשימה בזיכרון.

⌘ למה זה הפתרון האופטימלי

השוואה אחת לכל עמדה מיושרת, O(n) בזמן ו-O(1) בזיכרון נוסף. זו גם הגרסה האידיומטית ביותר ב-Python; הגרסה עם אינדקסים (`for i in range(len(s))`) נכונה אך פחות קריאה.

💡 איך הפתרון עומד במקרי הקצה הנסתרים

`""` מול `""` → 0 (אורכים שווים, אין עמדות); `"A"` מול `"G"` → 1; `"AC"` מול `"ACGT"` → `ValueError`; שני רצפים של 10,000 תווים שנבדלים רק בתו האחרון → 1.

מודול 4 · ביולוגיה × מדעי המחשב

חיפוש מוטיבים

בינוני | כ-40 דקות | אתרי קישור | חלון נע | חיפוש במחרוזות | `find_motif()`

מעבדה חזותית: חלון נע צעד-אחר-צעד עם מונה השוואות

בתוך הגנום מסתתרות "מילים" קצרות, באורך 6 עד 20 אותיות, שחלבונים מחפשים ונצמדים אליהן, בדיוק כמו Ctrl+F בעורך טקסט. מציאת כל המופעים של מילה כזו היא הצעד הראשון במיפוי האופן שבו גנים מופעלים ומכובים, והיא גם בעיה קלאסית במדעי המחשב: חיפוש תת-מחרוזת.

מה תלמדו

- הביולוגיה בשפה של מתכנתים: גורם שעתוק כ-Ctrl+F, ומוטיב כמחרוזת חיפוש
- מהם גורמי שעתוק ואיך הם מוצאים את אתרי הקישור שלהם
- רצפי קונצנזוס, ניוון (degeneracy) ומטריצות משקל-עמדה
- אלגוריתם החלון הנע והמחיר שלו, $O(n \cdot m)$
- למה כלים מובנים מפספסים התאמות חופפות, ואילו אלגוריתמים מהירים יותר קיימים (KMP, Boyer–Moore)

1 הביולוגיה בשפה של מתכנתים

גורם שעתוק הוא חלבון שמסתובב על ה-DNA ומחפש "מילה" קצרה, בדיוק כמו **Ctrl+F** בעורך טקסט או `grep` במסוף. כשהוא מוצא את המילה הוא נצמד אליה, וכך מסמן לתא: "את הגן שליד המילה הזאת צריך להפעיל (או לכבות)". המילה נקראת **מוטיב**, והמקום שבו היא נמצאה הוא **אתר קישור**. אם נחזור לדימוי ממודול 1, המוטיב יושב ב"כותרת" של קובץ הגן ואומר למערכת ההפעלה מתי לטעון אותו.

האלגוריתם שהמחשב מריץ נקרא **חלון נע**: מסגרת ברוחב המוטיב מונחת על תחילת המערך, משווים תו מול תו, ואז מזיזים את המסגרת תא אחד קדימה, וחוזר חלילה עד סוף המערך. בניגוד ל-Ctrl+F רגיל, כאן חייבים לדווח גם על מופעים **חופפים**, כי כל אחד מהם הוא מקום נפרד שבו חלבון יכול להיאחז. ולמה המילים מטושטשות? כי החלבון "סובל" כמה וריאציות של אותה מילה, כמו חיפוש עם תו ג'וקר.

איור Ctrl+F על DNA: החלון הנע, שלושה צעדים



למעלה: ה-DNA שבו מחפשים. מתחתיו שלושה צעדים של האלגוריתם: מסגרת ברוחב המוטיב מונחת על אינדקס i, התווים מושווים אחד-אחד (✓ תואם, ✗ לא), ואז המסגרת זזה תא אחד קדימה. שימו לב שההתאמות באינדקסים 1 ו-3 חופפות, ושתיהן נספרות.

מילון מונחים: ביולוגיה ↔ מדעי המחשב

בביולוגיה	במדעי המחשב	למה הדימוי עובד
גורם שעתוק	**תוכנית שמריצה Ctrl+F**	חלבון שסורק את ה-DNA ומחפש מילה קצרה שהוא יודע לזהות.
מוטיב	**מחרוזת החיפוש**	דפוס קצר (6–20 אותיות) בעל משמעות ביולוגית.
אתר קישור	**תוצאת החיפוש (אינדקס)**	המקום שבו המילה נמצאה והחלבון נצמד.
פרומוטור	**כותרת הקובץ / קובץ ההגדרות**	האזור שלפני הגן, שבו יושבות המילים שקובעות מתי להפעיל אותו.
מוטיב מנוון (IUPAC)	**תו ג'וקר / ביטוי רגולרי**	`W` = A או T: אותה מילה עם כמה איותים מותרים.
סריקת גנום	**חלון נע על מערך**	מסגרת בגודל קבוע שזזה תא אחד בכל צעד ומשווה תו מול תו.

⚡ בשורה אחת

מוטיב הוא מילה קצרה, גורם שעתוק הוא ה-Ctrl+F של התא, והקוד שלכם הוא חלון ברוחב המילה שזז תא אחד בכל פעם ומדווח על כל התאמה, גם חופפת.

2 גורמי שעתוק ואתרי קישור

נרד לפרטים: מי בדיוק מריץ את ה-Ctrl+F בתא, ואילו "מילים" הוא מחפש.

כמעט לכל תא בגופכם יש אותו גנום, ובכל זאת תא עצב ותא כבד מייצרים חלבונים שונים מאוד. אחת הסיבות העיקריות לכך היא **גורמי שעתוק** (transcription factors, TFs). החלבונים האלה נקשרים לרצפי DNA קצרים ליד גנים, שנקראים **אתרי קישור לגורמי שעתוק** (TFBS), ועוזרים לגייס את RNA פולימראז או לחסום אותו.

דפוס קצר וחוזר בעל תפקיד ביולוגי נקרא **מוטיב**. כמה מוטיבים מפורסמים:

מוטיב	קונצנזוס	תפקיד
TATA box	T A T A A A	רכיב בפרומוטור הליבה, כ-25–35 bp במעלה הזרם של גנים איקריוטיים רבים. נקשר על ידי TATA-binding protein (TBP).
E-box	C A C G T G	נקשר על ידי גורמי bHLH כמו MYC ו-CLOCK. שולט בשגשוג תאים ובשעון הצירקדי.
אתר EcoRI	G A A T T C	נחתך על ידי אנזים ההגבלה EcoRI. זהו פלינדרום: הוא זהה להשלמה ההפוכה של עצמו.
Shine–Dalgarno	A G G A G G	אתר קישור הריבוזום בחיידקים, ממש לפני קודון ההתחלה.

🧬 מוטיבים הם מטושטשים

אתרי קישור אמיתיים סובלים שונות, כך שגורם שעתוק עשוי להיקשר ל-`TATAAA`, ל-`TATATA` וגם ל-`TATAAT`. ביולוגים מתארים זאת בעזרת **קודי IUPAC** (`W` = A או T, כלומר `TATAWA`) או בעזרת **מטריצת משקל-עמדה (PWM)**, שנותנת ציון לכל בסיס בכל עמדה. כלים כמו MEME ו-HOMER מגלים מוטיבים; כלים כמו FIMO סורקים גנומים בחיפוש אחריהם. כולם נשענים על החיפוש המדויק שאתם עומדים לכתוב.

למה מופעים חופפים חשובים

DNA חזרתי כמו `ATATATAT` מכיל את `ATAT` בעמדות 0, 2 ו-4. ההתאמות האלה חופפות, וכל אחת מהן היא מקום נפרד שבו חלבון יכול להיקשר. כלי טוב לחיפוש מוטיבים חייב לדווח על כולן.

---

3 אלגוריתם החלון הנע

מציבים חלון בגודל המוטיב (m) בתחילת ה-DNA (שאורכו n). משווים את החלון למוטיב תו אחר תו, ואז מזיזים את החלון עמדה אחת קדימה. עוצרים כשהחלון היה גולש מעבר לסוף, כך שבסך הכול יש **n − m + 1** חלונות.

```python
def naive_search(text, pattern):
    n, m = len(text), len(pattern)
    hits = []
    for i in range(n - m + 1):          # every possible start position
        j = 0
        while j < m and text[i + j] == pattern[j]:
            j += 1                      # extend the match
        if j == m:                      # all m characters matched
            hits.append(i)
    return hits
```

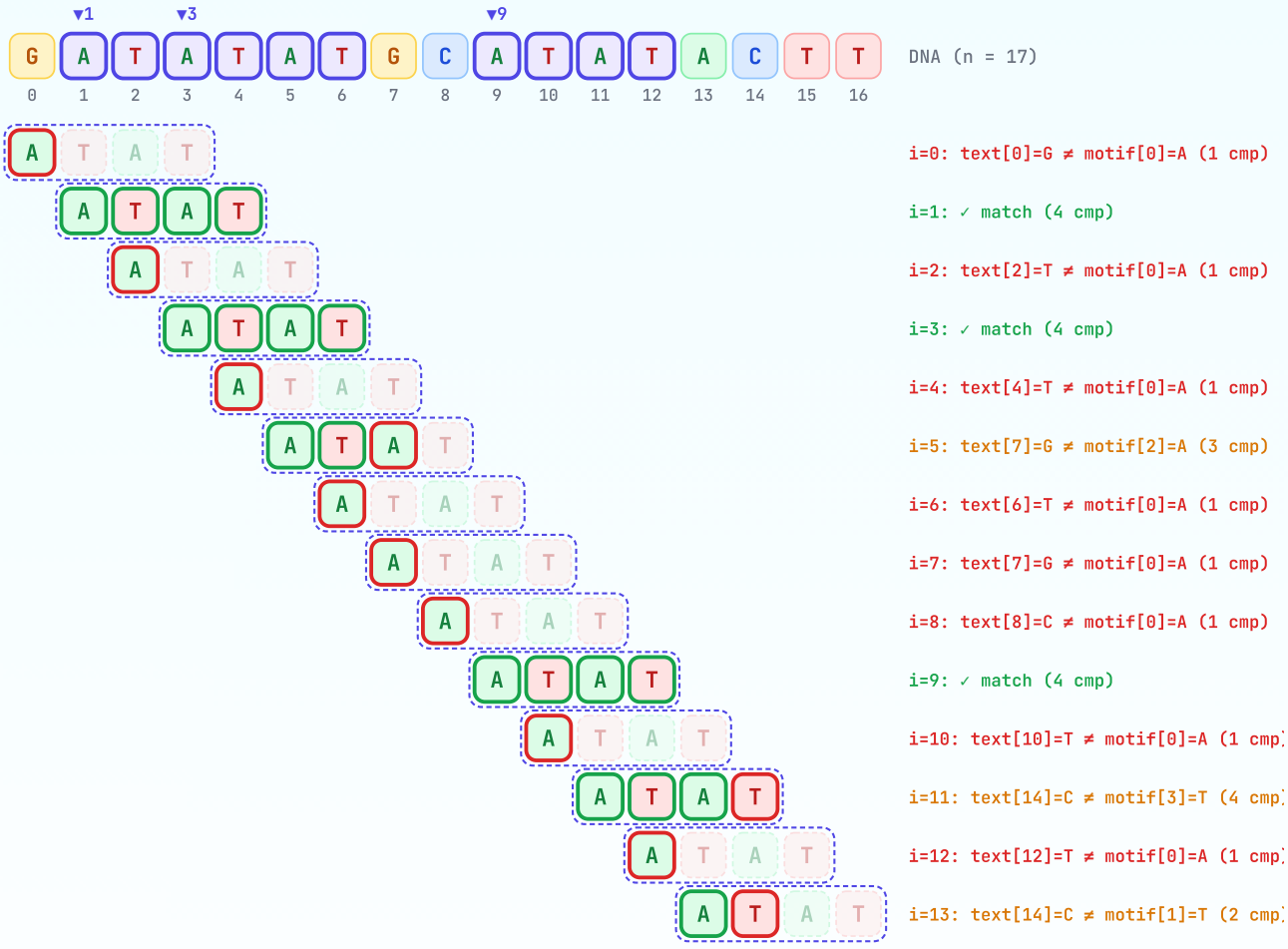
4 מעבדה חזותית: החלון הנע

צפו בכל השוואה. ירוק פירושו שהתווים תואמים; אדום פירושו אי-התאמה, שאחריה החלון קופץ קדימה. המונים מראים כמה עבודה האלגוריתם הנאיבי עושה. נסו רצף ומוטיב חזרתיים כמו `AAAAAAAA` ו-`AAAT` כדי לראות את המקרה הגרוע.

מעבדה חזותית · חיפוש מוטיב בחלון נע

החלון זז בסיס אחד בכל פעם. כל צעד משווה תו בודד.

סריקה מלאה: החלון (מוטיב ATAT, m = 4) מוצב בכל אחת מ-14 עמדות ההתחלה. ירוק: תו תואם; אדום: אי-התאמה שאחריה החלון נוטש מוקדם; אפור מקווקו: תווים שלא נבדקו כלל. מימין: תוצאת כל חלון ומספר ההשוואות שבוצעו בו.



חלונות (N−M+1)	השוואות	מקרה גרוע (N−M+1)·M	התאמות
14	29	56	[1, 3, 9]

הסריקה הסתיימה. 14 חלונות, **29** השוואות תווים (במקרה הגרוע 56; יציאות מוקדמות חסכו 27). ההתאמות בעמדות 1 ו-3 **חופפות** (ATATAT): כזו ש-`str.count` היה מפספס. התוצאה: `[1, 3, 9]`

בגרסה המקוונת: מקלידים DNA ומוטיב ולוחצים **סריקה** או **צעד**: כל השוואת תו מוצגת בנפרד, המונים מתעדכנים, וסמן ▼ מופיע מעל כל התאמה. נסו רצף ומוטיב חזרתיים (AAAAAAAA ו-AAAT) כדי לראות את המקרה הגרוע.

5 מימוש ב-Python

```python
dna, motif = "GATATATGCATATACTT", "ATAT"
m = len(motif)

# Slicing compares a whole window at once (still O(m) inside)
hits = [i for i in range(len(dna) - m + 1) if dna[i:i + m] == motif]
print(hits)       # [1, 3, 9]

# str.find with a moving start - jumps between matches using fast C code
hits, i = [], dna.find(motif)
while i != -1:
    hits.append(i)
    i = dna.find(motif, i + 1)    # +1, not +m, to allow overlaps!
```

⚠️ כלים מובנים שמדלגים בשקט על חפיפות

```python
"ATATAT".count("ATA")                  # 1  - non-overlapping!
import re
[m.start() for m in re.finditer("ATA", "ATATAT")]        # [0]
[m.start() for m in re.finditer("(?=ATA)", "ATATAT")]    # [0, 2]  lookahead fixes it
```

סיבוכיות ואלגוריתמים טובים יותר

החיפוש הנאיבי מבצע עד m השוואות לכל אחד מ-n − m + 1 החלונות, ולכן המקרה הגרוע הוא **O(n·m)**. עבור מוטיב באורך 20 מול גנום של 3 מיליארד בסיסים זה בסדר ב-C, אבל איטי ב-Python טהור. אלגוריתמים קלאסיים מהירים יותר:

אלגוריתם	רעיון	זמן
נאיבי / חלון נע	לנסות כל יישור אפשרי	O(n·m)
Knuth–Morris–Pratt	לחשב מראש טבלת כישלון, כך שאף תו בטקסט לא נקרא פעמיים	O(n + m)
Boyer–Moore	להשוות מהסוף להתחלה ולדלג קדימה בעזרת כללי "התו הרע" ו"הסיפא הטובה"	תת-לינארי בממוצע
Rabin–Karp	גיבוב מתגלגל (rolling hash) לכל חלון; מאמתים רק כשיש התאמת גיבוב	O(n + m) בתוחלת
Suffix array / FM-index	בונים אינדקס לגנום פעם אחת, ואז עונים מהר על כל שאילתה (כך BWA ו-Bowtie ממפים קריאות)	O(m) לשאילתה

זמן `O(n·m)`

המקרה הגרוע של האלגוריתם הנאיבי; בערך O(n) על DNA טיפוסי

זיכרון `O(k)`

k הוא מספר ההתאמות המוחזרות

💡 קואורדינטות מבוססות 0 לעומת מבוססות 1

Python מתחילה לספור מ-0. פורמטים ביולוגיים רבים סופרים מ-1 (GFF, VCF, SAM, וגם התשובות ב-Rosalind). קובצי BED מבוססי 0 וחצי-פתוחים, כמו חיתוכים ב-Python. באגים של "אחד יותר מדי" בקואורדינטות הם כנראה הבאג הנפוץ ביותר בגנומיקה, ולכן תמיד בדקו באיזו מוסכמה קובץ משתמש. במודול הזה, החזירו אינדקסים **מבוססי 0**.

---

6 דוגמאות קוד

לפני שתכתבו פתרון משלכם, הנה דוגמאות מוכנות שמשתמשות באותם כלים ורעיונות. מתחת לכל דוגמה מופיע **הפלט המלא** של הרצת הקוד (כפי שהתקבל מ-Python 3), ואחריו ההסבר המפורט. מומלץ להקליד את הקוד בעצמכם, להריץ אותו, ואז לשנות אותו ולבדוק איך הפלט משתנה. בגרסת האתר כל דוגמה ניתנת להרצה בלחיצת כפתור.

דוגמה 1 חיפוש אתרי הגבלה בפלסמיד

לפני שמשבטים גן לתוך פלסמיד, בודקים היכן אנזימי הגבלה יחתכו אותו. הדוגמה מוצאת את כל אתרי החיתוך של שלושה אנזימים נפוצים, ובודקת שהאתרים פלינדרומיים.

PYTHON

```python
def reverse_complement(seq):
    return seq.translate(str.maketrans("ACGT", "TGCA"))[::-1]

def is_palindrome(site):
    return site == reverse_complement(site)

ENZYMES = {"EcoRI": "GAATTC", "BamHI": "GGATCC", "HindIII": "AAGCTT"}
plasmid = "TTGAATTCGGATCCAAGCTTGCGAATTCAT"

for enzyme, site in ENZYMES.items():
    hits = []
    start = plasmid.find(site)
    while start != -1:
        hits.append(start)
        start = plasmid.find(site, start + 1)
    print(f"{enzyme:<8} {site}  palindrome={is_palindrome(site)}  positions={hits}")
```

```
$ python example.py
EcoRI    GAATTC  palindrome=True  positions=[2, 22]
BamHI    GGATCC  palindrome=True  positions=[8]
HindIII  AAGCTT  palindrome=True  positions=[14]
```

איך זה עובד, שלב אחר שלב

1. `reverse_complement` משתמשת בטבלת התרגום ממודול 1 ואז הופכת את סדר התווים.
2. `is_palindrome` בודקת אם אתר זהה להשלמה ההפוכה של עצמו. רוב אנזימי ההגבלה מסוג II מזהים אתרים פלינדרומיים, ולכן מספיק לחפש בגדיל אחד: ההתאמה בגדיל השני נמצאת בדיוק באותה עמדה.
3. `plasmid.find(site)` מחזיר את האינדקס של המופע הראשון, או `-1` אם אין כזה.

4. בלולאת ה-`while` ממשיכים לחפש מ-`start + 1`, ולא מ-`start + len(site)`, כדי לא לפספס מופעים חופפים.
5. `str.find` ממומש ב-C, ולכן הקפיצה בין התאמות מהירה הרבה יותר מבדיקה של כל חלון בלולאת Python.

דוגמה 2 מוטיבים מנוונים עם קודי IUPAC וביטויים רגולריים

גורמי שעתוק נקשרים למשפחה של רצפים דומים, ולא לרצף מדויק אחד. קודי IUPAC מאפשרים לתאר מוטיב מנוון, וביטוי רגולרי מאפשר לחפש אותו.

PYTHON

```python
import re

IUPAC = {
    "A": "A", "C": "C", "G": "G", "T": "T",
    "W": "[AT]", "S": "[CG]", "R": "[AG]", "Y": "[CT]", "N": "[ACGT]",
}

def motif_to_regex(motif):
    return "".join(IUPAC[ch] for ch in motif)

promoter = "GCTATAAAAGGCTATATAAAGCCTATAAATG"
motif = "TATAWA"
pattern = motif_to_regex(motif)
print("regex:", pattern)

for m in re.finditer(f"(?=({pattern}))", promoter):
    print(f"index {m.start():>2}: {m.group(1)}")
```

```
$ python example.py
regex: TATA[AT]A
index  2: TATAAA
index 12: TATATA
index 14: TATAAA
index 23: TATAAA
```

איך זה עובד, שלב אחר שלב

1. המילון `IUPAC` ממפה כל קוד למחלקת תווים של ביטוי רגולרי: `W` (מלשון weak, קשר חלש) הופך ל-`[AT]`, ו-`N` לכל אחד מארבעת הבסיסים.
2. `motif_to_regex("TATAWA")` מחליפה כל תו במוטיב ומחברת את התוצאות, כך שמתקבל הביטוי `TATA[AT]A`.
3. המבנה `(?=(...))` הוא lookahead: הוא בודק התאמה בלי "לצרוך" תווים, ולכן מנוע הביטויים מתקדם תו אחד בכל פעם ומוצא גם מופעים חופפים.
4. הקבוצה הפנימית `(...)` לוכדת את הרצף שהתאים בפועל, ו-`m.group(1)` מחזיר אותו. `m.start()` נותן את אינדקס ההתחלה.
5. שימו לב לפלט: מופיעים בו גם `TATAAA` וגם `TATATA`, שני רצפים שונים שמתאימים לאותו מוטיב מנוון. ההתאמות בעמדות 12 ו-14 חופפות, ושתיהן נמצאו.

דוגמה 3 המילים השכיחות ביותר (k-mers)

כשלא יודעים מראש מהו המוטיב, מתחילים בספירת כל המילים באורך k (k-mers) ומחפשים את אלה שחוזרות באופן חשוד. זהו הרעיון שמאחורי כלים לגילוי מוטיבים.

PYTHON

```python
from collections import Counter

def kmer_counts(seq, k):
    return Counter(seq[i:i + k] for i in range(len(seq) - k + 1))

dna = "ACGTTGCATGTCGCATGATGCATGAGAGCT"
counts = kmer_counts(dna, 4)

print("distinct 4-mers:", len(counts))
for kmer, n in counts.most_common(3):
    print(kmer, n)
```

```
$ python example.py
distinct 4-mers: 21
GCAT 3
CATG 3
TGCA 2
```

איך זה עובד, שלב אחר שלב

1. `range(len(seq) - k + 1)` מייצר את כל עמדות ההתחלה האפשריות, אותו מספר חלונות כמו באלגוריתם החלון הנע.
2. `seq[i:i + k]` חותך כל חלון, והגנרטור מעביר את החלונות ישר אל `Counter`, בלי לבנות רשימת ביניים.
3. `len(counts)` הוא מספר ה-k-mers **השונים** שנמצאו, ו-`counts.most_common(3)` מחזיר את שלושת השכיחים ביותר כזוגות `(kmer, count)`, ממוינים מהשכיח ביותר.
4. הסיבוכיות היא O(n·k): יש n חלונות, ובכל אחד מתבצעים חיתוך וגיבוב של מחרוזת באורך k.
5. באזור ה-ori של חיידקים, k-mers שחוזרים פעמים רבות בחלון קצר הם לרוב אתרי קישור של החלבון DnaA, שמתחיל את השכפול.

---

7 המשימה שלכם

ממשו את `find_motif(dna, motif)` כך שתחזיר **כל** אינדקס התחלה, כולל חפיפות. הבדיקות הנסתרות מכסות מוטיב ריק, מוטיב ארוך מה-DNA ומקרה גרוע באורך 1,000 בסיסים.

מודול 4 · אתגר קוד בסגנון LEETCODE

✎ התרגיל: `find_motif(dna, motif)`

בינוני | אתרי קישור | חלון נע | חיפוש במחרוזות | כ-40 דקות

בהינתן מחרוזת DNA `dna` ומחרוזת קצרה יותר `motif`, מצאו את **כל** המיקומים שבהם `motif` מופיעה כתת-מחרוזת של `dna`.

ממשו את `find_motif(dna, motif)` כך שתחזיר `list` של אינדקסי ההתחלה, **מבוססי 0**, בסדר עולה. יש לדווח על כל המופעים, כולל מופעים **חופפים**.

- אם `motif` לא מופיעה, או ארוכה מ-`dna`, החזירו `[]`.
- אם `motif` היא מחרוזת ריקה, החזירו `[]`.

דוגמאות

דוגמה 1

קלט: `dna = "GATATATGCATATACTT"`
`motif = "ATAT"`
פלט: `[1, 3, 9]`
הסבר: המופעים בעמדות 1 ו-3 חופפים (ATATAT), ושניהם נספרים.

דוגמה 2

קלט: `dna = "AAAA", motif = "AA"`
פלט: `[0, 1, 2]`

דוגמה 3

קלט: `dna = "ACGT", motif = "TTT"`
פלט: `[]`

אילוצים

- $0 \le \text{len(motif)} \le \text{len(dna)} \le 10^4$
- יש להחזיר `list` של `int` בסדר עולה
- האינדקסים מבוססי 0 (בבעיית SUBS של Rosalind הם מבוססי 1, ולכן שם מוסיפים 1)

רמזים

- יש `len(dna) - len(motif) + 1` חלונות אפשריים.
- `dna[i:i + m] == motif` בודק חלון אחד.
- טפלו במוטיב ריק לפני הלולאה הראשית.
- אל תשתמשו ב-`str.count` או ב-`re.finditer` רגיל: הם מדלגים על חפיפות.

בדיקות גלויות (הרצה)

אלה הבדיקות שכפתור **הרצה** באתר מריץ. הפתרון חייב לעבור את כולן.

קריאה	תוצאה צפויה
`find_motif("GATATATGCATATACTT", "ATAT")`	`[1, 3, 9]`
`find_motif("AAAA", "AA")`	`[0, 1, 2]`
`find_motif("ACGT", "TTT")`	`[]`

בדיקות נסתרות: מקרי קצה (הגשה)

כפתור **הגשה** מריץ גם את הבדיקות האלה. הן נועדו לתפוס קלטים ריקים, מקרי גבול, יעילות על קלט גדול וטיפול בשגיאות. כתבו את הפתרון כך שיעמוד בהן **לפני** שאתם מציצים בטבלה.

קריאה	תוצאה צפויה	מה נבדק
`find_motif("ACGT", "ACGT")`	`[0]`	
`find_motif("AC", "ACGT")`	`[]`	מוטיב ארוך מהרצף
`find_motif("ACGTACGT", "")`	`[]`	קלט ריק
`find_motif("GAATTCGAATTC", "GAATTC")`	`[0, 6]`	
`find_motif("TATAAAGGTATAAA", "TATAAA")`	`[0, 8]`	
`find_motif("A" * 1000, "AAA")`	`list(range(998))`	קלט גדול (1,000 תווים): יעילות

קוד התחלתי

כך נראה העורך כשפותחים את המודול. ממשו את גוף הפונקציה במקום `pass`.

```python
def find_motif(dna, motif):
    """
    Return all 0-based start indices where motif occurs in dna
    (overlapping occurrences included).

    >>> find_motif("GATATATGCATATACTT", "ATAT")
    [1, 3, 9]
    """
    # Write your code here
    pass
```

🔑 פתרון עיון מלא: `find_motif`

הפתרון שעובר את כל הבדיקות, הגלויות והנסתרות. מספרי השורות בהסבר מתייחסים לקוד שלמעלה.

```python
def find_motif(dna, motif):
    m = len(motif)
    if m == 0:
        return []
    # Slide a window of width m across dna; record every exact match.
    return [i for i in range(len(dna) - m + 1) if dna[i:i + m] == motif]
```

הסבר שורה אחר שורה

שורה 2 — `m` הוא אורך המוטיב. שומרים אותו במשתנה כי הוא משמש פעמיים: בחישוב מספר החלונות ובחיתוך.

שורות 3–4 — **מוטיב ריק.** בלי הבדיקה הזו, `dna[i:i + 0]` היה מחזיר `""` , שתמיד שווה למוטיב הריק, והפונקציה הייתה מחזירה את כל האינדקסים. לכן מחזירים `[]` מפורשות.

שורה 5 — ההערה מתארת את האלגוריתם: חלון ברוחב `m` מחליק לאורך ה-DNA, ונרשמת כל התאמה מדויקת.

שורה 6 — **החלון הנע ברשימת הבנה.** `range(len(dna) - m + 1)` מייצר את כל $n - m + 1$ עמדות ההתחלה שבהן החלון נכנס במלואו. אם המוטיב ארוך מה-DNA, הגבול שלילי, `range` ריק והתוצאה `[]` בלי בדיקה נוספת. `dna[i:i + m] == motif` משווה חלון שלם (עד `m` השוואות תווים, ב-C). מכיוון שהחלון זז עמדה אחת בכל פעם, מופעים **חופפים** נמצאים, והאינדקסים יוצאים בסדר עולה באופן טבעי.

⌘ למה זה הפתרון האופטימלי

$O(n \cdot m)$ במקרה הגרוע (DNA ומוטיב חזרתיים), אך בפועל כמעט לינארי, כי ההשוואה נעצרת באי-ההתאמה הראשונה. לגנום שלם משתמשים באינדקס (FM-index) או ב-KMP, אבל לקלט של עד 10^4 תווים זו הגרסה הפשוטה והנכונה. `str.count` ו-`re.finditer` רגיל פסולים כאן כי הם מדלגים על חפיפות.

💡 איך הפתרון עומד במקרי הקצה הנסתרים

`"ACGT"` , `"ACGT"` → `[0]` (חלון יחיד); `"AC"` → `[]` (מוטיב ארוך מהרצף); מוטיב ריק → `[]` ; `"A" * 1000` עם `"AAA"` → 998 אינדקסים, המקרה הגרוע.

מודול 5 · ביולוגיה × מדעי המחשב

תרגום RNA לחלבון

בינוני | כ-35 דקות | קודונים | מילונים (hash maps) | קוד גנטי | `translate_rna()`

מעבדה חזותית: ריבוזום שעובר על קודונים, שרשרת חומצות אמינו וטבלת הקוד הגנטי

mRNA הוא רק קובץ הוראות. כדי שיהפוך לחלבון, מפרש מולקולרי צריך לקרוא אותו שלוש אותיות בכל פעם ולחפש כל שלישייה בטבלת פקודות של 64 כניסות. במודול הזה תלמדו את הקוד הגנטי, ואיך מבנה הנתונים **מילון** הופך את התרגום לחיפוש מהיר בטבלה.

מה תלמדו

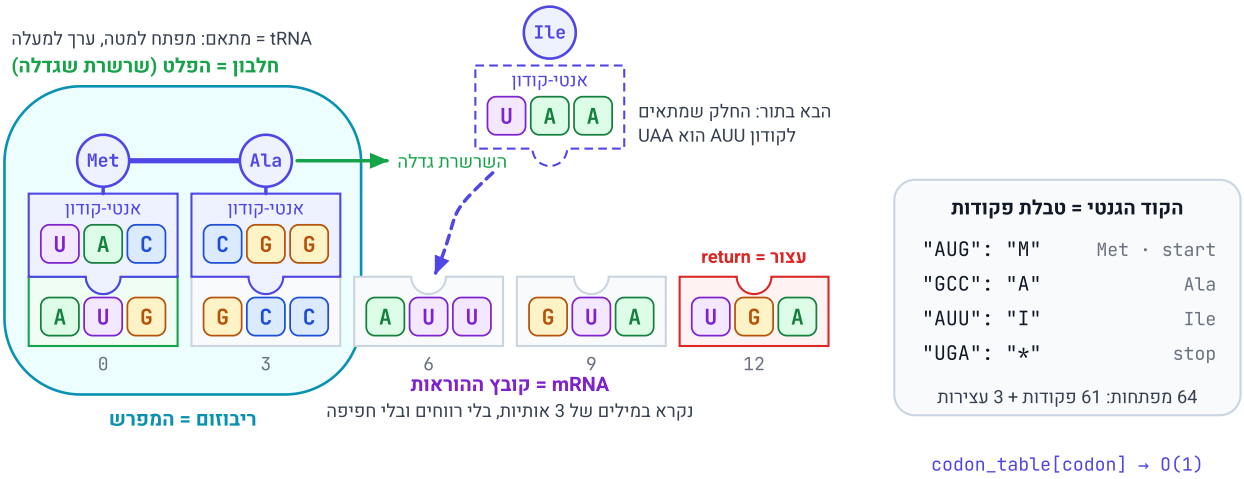
- הביולוגיה בשפה של מתכנתים: הריבוזום כמפרש, הקוד הגנטי כטבלת פקודות, tRNA כמתאם
- מהו קודון, ולמה צריך שלוש אותיות כדי לקודד 20 חומצות אמינו
- איך הריבוזום ומולקולות tRNA מתרגמים mRNA לחלבון
- מילונים (hash maps) ב-Python: חיפוש בזמן O(1) בממוצע
- איך לקרוא רצף במסגרת קריאה ולעצור בקודון עצירה

1 הביולוגיה בשפה של מתכנתים

אם mRNA הוא קובץ של הוראות, **הריבוזום** הוא המפרש (interpreter) שמריץ אותו. הוא קורא את הקובץ ב"מילים" קבועות של שלוש אותיות, **קודונים**, בדיוק כמו מעבד שקורא פקודות ברוחב קבוע. לכל קודון יש משמעות אחת, שמוגדרת ב**קוד הגנטי**: טבלת פקודות (opcode table) עם 64 שורות. 61 מהן אומרות "הוסף את חומצת האמינו X לשרשרת", ושלוש אומרות "עצור". הפקודה `AUG` היא גם נקודת הכניסה, ה-`main` של הגן, וגם פקודה רגילה שמוסיפה מתיונין.

מי שמבצע את החיפוש בטבלה הוא **tRNA**: מולקולת מתאם שבקצה אחד יש לה "מפתח" (אנטי-קודון) שמתאים לקודון כמו חלק בפאזל, ובקצה השני "ערך", חומצת האמינו הנכונה. אין כאן קסם: זה `dict` שמומש בכימיה, ו-`codon_table[codon]` ב-Python עושה בדיוק אותו דבר. התוצר הוא **חלבון**: שרשרת של 20 סוגי אבני בניין (חומצות אמינו) שמתקפלת למכונה עובדת. כשהריבוזום פוגש קודון עצירה הוא משחרר את השרשרת, כמו `return` שמסיים פונקציה.

איור קודונים כחלקי פאזל: המפרש, טבלת הפקודות והמתאם



ה-mRNA מחולק ל"מילים" של שלוש אותיות (קודונים), וכל קודון הוא שקע בפאזל. לכל שקע מתאים רק חלק אחד: מולקולת tRNA שבתחתיתה אנטי-קודון משלים (המפתח) ובראשה חומצת האמינו המתאימה (הערך). הריבוזום מחבר את חומצות האמינו לשרשרת, קודון אחר קודון, עד שהוא פוגש קודון עצירה.

מילון מונחים: ביולוגיה ↔ מדעי המחשב

בביולוגיה	במדעי המחשב	למה הדימוי עובד
קודון	**פקודה (opcode) ברוחב 3 תווים**	המפרש קורא תמיד שלוש אותיות, ולכן אין צורך ברווחים או במפרידים.
הקוד הגנטי	**טבלת פקודות / `dict`**	64 מפתחות אפשריים, ולכל אחד ערך אחד: חומצת אמינו או "עצור".
tRNA	**מתאם מפתח ↔ ערך**	צד אחד מזהה את הקודון, הצד השני נושא את חומצת האמינו המתאימה.
ריבוזום	**מפרש / מכונה וירטואלית**	עובר על הקובץ פקודה אחר פקודה ומבצע: מוסיף אבן בניין לשרשרת.
חומצת אמינו	**אבן בניין (20 סוגים)**	הפלט של כל פקודה; רצף של אבני בניין הוא התוכנית המורכבת.
קודון התחלה AUG	**`main()` / נקודת כניסה**	מסמן מאיפה להתחיל לקרוא, וגם מוסיף את חומצת האמינו הראשונה (Met).
קודון עצירה	**`return` / halt**	מסיים את הפונקציה ומשחרר את התוצר; מה שאחריו לא מורץ.

⚡ בשורה אחת

הריבוזום הוא מפרש שקורא פקודות של 3 אותיות, מחפש כל אחת ב-dict של 64 כניסות (דרך ה-tRNA), ומוסיף אבן בניין לחלבון עד שהוא פוגש return.

2 הקוד הגנטי: שלישיות של אותיות

מכאן נפרט: למה הפקודות הן בדיוק שלוש אותיות, ומה כתוב בכל אחת מ-64 השורות בטבלה.

חלבונים בנויים מ-20 סוגים של **חומצות אמינו**, אבל ל-RNA יש רק 4 אותיות. כמה אותיות צריך כדי לקודד כל חומצת אמינו? אות אחת נותנת 4 צירופים בלבד, ושתי אותיות נותנות $4^2 = 16$, וזה עדיין לא מספיק. שלוש אותיות נותנות $4^3 = 64$ צירופים, יותר ממספיק. כל שלישייה כזו נקראת **קודון**.

ב-1961 גילו מרשל נירנברג ויוהאן מתאי שהרצף `UUU` מקודד לחומצת האמינו פנילאלנין, ועד 1966 פוענח הקוד כולו. מתברר שהקוד כמעט **אוניברסלי**: אותה טבלה משמשת חיידקים, צמחים ובני אדם, עם חריגים קטנים בלבד (למשל במיטוכונדריה).

- **61 קודונים** מקודדים חומצות אמינו, ולכן לרוב חומצות האמינו יש יותר מקודון אחד. אומרים שהקוד מנוון (degenerate): למשל, ל-Leu (L) יש שישה קודונים שונים.
- **3 קודוני עצירה** (`UAA`, `UAG`, `UGA`) אינם מקודדים אף חומצת אמינו. הם מסמנים לריבוזום לשחרר את החלבון. בטבלאות מסמנים אותם ב-`*`.
- **קודון ההתחלה** `AUG` מקודד מתיונין (M), ובאותו זמן מסמן היכן התרגום מתחיל.

1st \ 2nd	U		C		A		G	
U	UUU	Phe **F**	UCU	Ser **S**	UAU	Tyr **Y**	UGU	Cys **C**
	UUC	Phe **F**	UCC	Ser **S**	UAC	Tyr **Y**	UGC	Cys **C**
	UUA	Leu **L**	UCA	Ser **S**	UAA	Stop *	UGA	Stop *
	UUG	Leu **L**	UCG	Ser **S**	UAG	Stop *	UGG	Trp **W**
C	CUU	Leu **L**	CCU	Pro **P**	CAU	His **H**	CGU	Arg **R**
	CUC	Leu **L**	CCC	Pro **P**	CAC	His **H**	CGC	Arg **R**
	CUA	Leu **L**	CCA	Pro **P**	CAA	Gln **Q**	CGA	Arg **R**
	CUG	Leu **L**	CCG	Pro **P**	CAG	Gln **Q**	CGG	Arg **R**
A	AUU	Ile **I**	ACU	Thr **T**	AAU	Asn **N**	AGU	Ser **S**
	AUC	Ile **I**	ACC	Thr **T**	AAC	Asn **N**	AGC	Ser **S**
	AUA	Ile **I**	ACA	Thr **T**	AAA	Lys **K**	AGA	Arg **R**
	AUG	Met **M**	ACG	Thr **T**	AAG	Lys **K**	AGG	Arg **R**
G	GUU	Val **V**	GCU	Ala **A**	GAU	Asp **D**	GGU	Gly **G**
	GUC	Val **V**	GCC	Ala **A**	GAC	Asp **D**	GGC	Gly **G**
	GUA	Val **V**	GCA	Ala **A**	GAA	Glu **E**	GGA	Gly **G**
	GUG	Val **V**	GCG	Ala **A**	GAG	Glu **E**	GGG	Gly **G**

הקוד הגנטי הסטנדרטי. השורה היא הבסיס הראשון, העמודה היא הבסיס השני, ובכל תא מופיעים ארבעת הקודונים לפי הבסיס השלישי. קודון ההתחלה מסומן בירוק, קודוני העצירה באדום.

🧬 למה הבסיס השלישי "סלחני"?

שימו לב שבטבלה, לקודונים שנבדלים רק בבסיס השלישי יש לרוב אותה חומצת אמינו: למשל, כל ארבעת הקודונים `GCU`, `GCC`, `GCA` ו-`GCG` מקודדים Ala. זו תופעת ה-wobble: ה-tRNA נצמד בצורה פחות הדוקה לבסיס השלישי. כתוצאה מכך, מוטציות רבות בעמדה השלישית הן סינונימיות, כלומר שקטות, כפי שראיתם במודול 3.

איך הריבוזום עובד

הריבוזום נצמד ל-mRNA ומתקדם לאורכו קודון אחר קודון, בכיוון 5′→3′. לכל קודון מגיעה מולקולת **tRNA** שנושאת בקצה אחד אנטי-קודון משלים, ובקצה השני את חומצת האמינו המתאימה. הריבוזום מחבר כל חומצת אמינו חדשה לשרשרת בקשר פפטידי. כשמגיע קודון עצירה, אין tRNA שמתאים לו: חלבון שנקרא גורם שחרור נקשר במקומו, והחלבון המוכן משתחרר.

⌘ מסגרת קריאה (reading frame)

הקוד נקרא בשלישיות שאינן חופפות ובלי רווחים, ולכן חשוב מאיפה מתחילים. הרצף `AUGGCCUAA` נקרא כ-`AUG GCC UAA` (Met ,Ala, עצירה) אם מתחילים מאינדקס 0, אבל כ-`UGG CCU` (Trp ,Pro) אם מתחילים מאינדקס 1. במודול הזה מתרגמים תמיד מאינדקס 0; במודול הבא תחפשו את מסגרת הקריאה הנכונה בעצמכם.
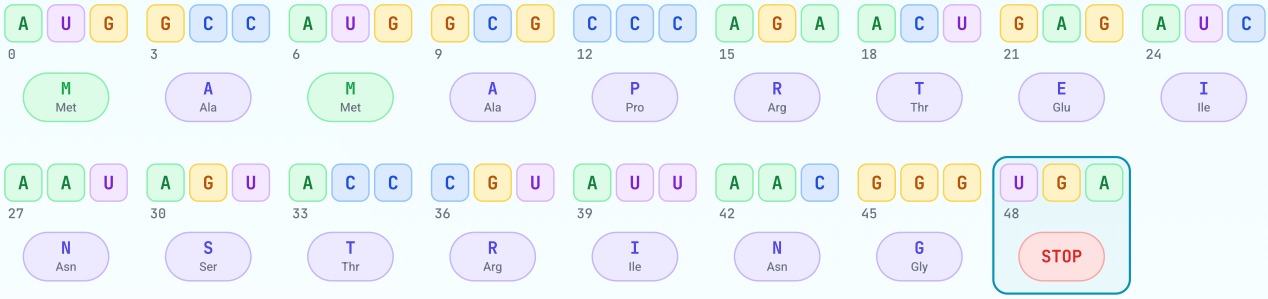
---

3 מעבדה חזותית: הריבוזום בפעולה

הזינו רצף mRNA ולחצו **תרגום**. הריבוזום עובר קודון אחר קודון, וכל קודון מתורגם לחומצת אמינו. בקודון העצירה התרגום נעצר, והקודונים שאחריו לא נקראים כלל.

מעבדה חזותית · תרגום

הריבוזום קורא את ה-mRNA קודון אחר קודון, ומחבר לשרשרת את חומצת האמינו של כל קודון.

הרצף מחולק ל-17 קודונים (המספר מתחת לכל קודון הוא האינדקס שבו הוא מתחיל). מתחת לכל קודון: חומצת האמינו שה-tRNA מביא. ירוק: מתיונין (AUG). אדום: קודון עצירה, שבו הריבוזום משחרר את החלבון.



חומצות אמינו 16 — חלבון `MAMAPRTEINSTRING`

קודון 1: `AUG → Met (M)` · **מתיונין, חומצת האמינו הראשונה בכמעט כל חלבון.**

קודון עצירה UGA: אין tRNA שמתאים לו, והחלבון משתחרר. הקודונים שאחריו (אם היו) לא נקראים. התוצאה: `"MAMAPRTEINSTRING"`

בגרסה המקוונת: מזינים mRNA ולוחצים **תרגום** או **צעד**: הריבוזום (מסגרת תכולה) עובר קודון אחר קודון, בכל צעד "צץ" תג חומצת אמינו ושורת החלבון גדלה. בקודון עצירה התרגום נעצר והקודונים שאחריו מעומעמים; שארית של אות או שתיים בסוף מוצגת כקודון חלקי.

4 מילונים: טבלאות חיפוש ב-Python

הקוד הגנטי הוא בדיוק מה שמילון (`dict`) נועד לייצג: התאמה בין **מפתח** (קודון) ל**ערך** (חומצת אמינו). מילון ב-Python ממומש כ**טבלת גיבוב** (hash table), ולכן חיפוש מפתח לוקח בממוצע זמן קבוע, **O(1)**, בלי קשר לגודל המילון.

```python
codon_table = {"AUG": "M", "GCC": "A", "UAA": "*"}

codon_table["AUG"]            # 'M'    -> lookup by key
codon_table["UUU"] = "F"      # add (or overwrite) a key
"GCC" in codon_table          # True   -> membership test on KEYS
len(codon_table)              # 4

codon_table["XYZ"]            # KeyError: 'XYZ'
codon_table.get("XYZ", "?")   # '?'    -> default instead of an exception

for codon, aa in codon_table.items():   # iterate over (key, value) pairs
    print(codon, "->", aa)
```

איך טבלת גיבוב עובדת

כשמכניסים מפתח, Python מחשבת עבורו ערך גיבוב (`hash(key)`): מספר שקובע באיזה "תא" בזיכרון הערך יישמר. בחיפוש מחושב שוב אותו ערך גיבוב, ו-Python ניגשת ישר לתא הנכון במקום לסרוק את כל הטבלה. לכן המפתחות חייבים להיות בלתי ניתנים לשינוי: מחרוזות, מספרים ו-tuples מתאימים, ורשימות לא.

פעולה	רשימה של זוגות	מילון
חיפוש חומצת האמינו של קודון	O(n)	O(1)
בדיקת שייכות עם `in`	O(n)	O(1)
הוספת זוג חדש	O(1)	O(1)

```python
# Slow: scan a list of pairs until the codon is found - O(n) per lookup
pairs = [("UUU", "F"), ("UUC", "F"), ("UUA", "L")]   # ... 64 pairs
def lookup_list(codon):
    for key, value in pairs:
        if key == codon:
            return value

# Fast: one hash computation, then jump straight to the slot - O(1) average
table = dict(pairs)
table["UUA"]                  # 'L'

# Dict comprehension: invert the table (amino acid -> list of codons)
by_amino_acid = {}
for codon, aa in table.items():
    by_amino_acid.setdefault(aa, []).append(codon)
print(by_amino_acid)          # {'F': ['UUU', 'UUC'], 'L': ['UUA']}
```

זמן	זיכרון
O(n)	O(n)
n/3 חיפושים במילון, כל אחד O(1) בממוצע	רשימת חומצות האמינו של החלבון

⚠️ מלכודות נפוצות

- **DNA במקום RNA**: בטבלה יש `U`, לא `T`. חיפוש של `"ATG"` יזרוק `KeyError`. שעתקו קודם, כמו במודול 1.
- **קודון חלקי בסוף**: אם אורך הרצף אינו מתחלק ב-3, השארית (אות אחת או שתיים) אינה קודון. `range(0, len(rna) - 2, 3)` עוצר בזמן.
- **שכחה לעצור**: מה שבא אחרי קודון העצירה כבר אינו חלק מהחלבון.

---

5 דוגמאות קוד

לפני שתכתבו פתרון משלכם, הנה דוגמאות מוכנות שמשתמשות באותם כלים ורעיונות. מתחת לכל דוגמה מופיע **הפלט המלא** של הרצת הקוד (כפי שהתקבל מ-Python 3), ואחריו ההסבר המפורט. מומלץ להקליד את הקוד בעצמכם, להריץ אותו, ואז לשנות אותו ולבדוק איך הפלט משתנה. בגרסת האתר כל דוגמה ניתנת להרצה בלחיצת כפתור.

דוגמה 1 בניית הקוד הגנטי והיפוך המילון

כמה רצפי RNA שונים יכולים לקודד את אותו חלבון? בגלל שהקוד מנוון, התשובה היא מכפלה של מספרי הקודונים. הדוגמה בונה את כל 64 הקודונים בשלוש שורות קוד, וסופרת כמה קודונים יש לכל חומצת אמינו.

PYTHON

```python
from collections import Counter

bases = "UCAG"
amino_acids = "FFLLSSSSYY**CC*WLLLLPPPPHHQQRRRRIIIMTTTTNNKKSSRRVVVVAAAADDEEGGGG"
codons = [a + b + c for a in bases for b in bases for c in bases]
CODON_TABLE = dict(zip(codons, amino_acids))

codons_per_aa = Counter(CODON_TABLE.values())
print("codons:", len(CODON_TABLE))
print("L:", codons_per_aa["L"], "| W:", codons_per_aa["W"], "| stop:",
codons_per_aa["*"])

protein = "MA"
total = codons_per_aa["*"]          # every protein ends with one stop codon
for aa in protein:
    total *= codons_per_aa[aa]
print(f"RNA strings that encode {protein!r}: {total}")
```

```
$ python example.py
codons: 64
L: 6 | W: 1 | stop: 3
RNA strings that encode 'MA': 12
```

איך זה עובד, שלב אחר שלב

1. רשימת ההבנה עם שלוש לולאות מקוננות, `for a ... for b ... for c`, מייצרת את כל 4 × 4 × 4 = 64 הקודונים בסדר הקבוע של טבלת הקוד: `UUU`, `UUC`, `UUA` וכן הלאה.
2. המחרוזת `amino_acids` מסודרת באותו סדר בדיוק. `zip(codons, amino_acids)` מצמיד כל קודון לאות שלו, ו-`dict(...)` הופך את הזוגות למילון.
3. `Counter(CODON_TABLE.values())` סופר כמה פעמים כל ערך מופיע במילון, כלומר כמה קודונים יש לכל חומצת אמינו. ל-L יש 6, ל-W רק אחד, ולעצירה 3.
4. לפי עקרון הכפל, מספר רצפי ה-RNA שמקודדים חלבון הוא מכפלת מספרי הקודונים של כל חומצות האמינו שלו, כפול 3 עבור קודון העצירה בסוף. עבור `"MA"`: 12 = 3 × 4 × 1.
5. עבור חלבון אמיתי באורך מאות חומצות אמינו המספר עצום, ולכן בבעיית MRNA של Rosalind מחשבים אותו מודולו 1,000,000.

דוגמה 2 תרגום בשלוש מסגרות הקריאה

אותו רצף mRNA נותן שלושה חלבונים שונים לגמרי, לפי האינדקס שממנו מתחילים לקרוא. הדוגמה מתרגמת את שלוש המסגרות בלי לעצור, כדי שתראו היכן נופלים קודוני העצירה (`*`).

PYTHON

```python
bases = "UCAG"
amino_acids = "FFLLSSSSYY**CC*WLLLLPPPPHHQQRRRRIIIMTTTTNNKKSSRRVVVVAAAADDEEGGGG"
codons = [a + b + c for a in bases for b in bases for c in bases]
CODON_TABLE = dict(zip(codons, amino_acids))

rna = "AUGGCCAUGGCGCCCAGAACUGAGAUCAAUAGUACCCGUAUUAACGGGUGA"

for frame in range(3):
    peptide = "".join(
        CODON_TABLE[rna[i:i + 3]] for i in range(frame, len(rna) - 2, 3)
    )
    print(f"frame {frame}: {peptide}")
```

```
$ python example.py
frame 0: MAMAPRTEINSTRING*
frame 1: WPWRPELRSIVPVLTG
frame 2: GHGAQN*DQ*YPY*RV
```

איך זה עובד, שלב אחר שלב

1. שלוש השורות הראשונות בונות את `CODON_TABLE` כמו בדוגמה הקודמת, כך שהדוגמה עומדת בפני עצמה.
2. `range(frame, len(rna) - 2, 3)` מתחיל באינדקס `frame` (0, 1 או 2) ומתקדם בצעדים של 3. החסרת 2 מבטיחה שכל חיתוך `rna[i:i + 3]` יהיה קודון שלם.
3. ביטוי הגנרטור בתוך `"".join(...)` מחפש כל קודון במילון ומחבר את התוצאות למחרוזת אחת, בלי לבנות רשימת ביניים.
4. במסגרת 0 מקבלים את החלבון האמיתי, `MAMAPRTEINSTRING`, ואחריו `*`. במסגרת 2 קודוני העצירה מפוזרים לאורך הרצף ושוברים אותו לקטעים קצרים: כך נראית מסגרת "סגורה". במסגרת 1 דווקא אין קודון עצירה, אבל גם אין בה `M`, כלומר אין קודון התחלה. מסגרת בלי עצירות עדיין אינה גן.

5 זה בדיוק הרעיון שמאחורי חיפוש ORFs, הנושא של המודול הבא: מחפשים מסגרת שבה יש קטע ארוך בלי `*` .

דוגמה 3 תרגום בטוח: DNA, אותיות קטנות וקודונים לא מוכרים

נתונים אמיתיים מבולגנים. הגרסה הזו של התרגום מקבלת גם DNA, גם אותיות קטנות וגם בסיסים עמומים (`N`), ומשתמשת ב- `dict.get` כדי לא לקרוס על קודון שאינו בטבלה.

PYTHON

```python
CODON_TABLE = {"AUG": "M", "GCC": "A", "UUU": "F", "UAA": "*"}   # excerpt

def safe_translate(seq, table, unknown="X"):
    rna = seq.upper().replace("T", "U")
    protein = []
    for i in range(0, len(rna) - 2, 3):
        aa = table.get(rna[i:i + 3], unknown)
        if aa == "*":
            break
        protein.append(aa)
    return "".join(protein)

print(safe_translate("AUGGCCUUU", CODON_TABLE))
print(safe_translate("atgNNNgcc", CODON_TABLE))    # DNA, lowercase, ambiguous
print(safe_translate("AUGUAAGCC", CODON_TABLE))    # stops at UAA
```

```
$ python example.py
MAF
MXA
M
```

איך זה עובד, שלב אחר שלב

1 `seq.upper().replace("T", "U")` מנרמל את הקלט בשרשרת אחת: הופך לאותיות גדולות ומשעתק DNA ל-RNA. שתי המתודות מחזירות מחרוזות חדשות.

2 `table.get(codon, unknown)` מחזיר את חומצת האמינו אם הקודון קיים, ואחרת את ערך ברירת המחדל `"X"` , הסימון המקובל לחומצת אמינו לא ידועה. כך `NNN` לא גורם ל- `KeyError` .

3 הפרמטר `unknown="X"` הוא ארגומנט ברירת מחדל: אפשר לקרוא לפונקציה בלעדיו, או להעביר ערך אחר כמו `"?"` .

4 בקודון העצירה `break` יוצא מהלולאה, ולכן בקריאה השלישית מקבלים רק `"M"` : הקודון `GCC` שאחרי `UAA` לא מתורגם.

5 בתרגיל הקלט תמיד נקי (RNA באותיות גדולות), אז אפשר להשתמש ישירות ב- `CODON_TABLE[codon]` .

6 המשימה שלכם

ממשו את `translate_rna(rna_sequence)` בעזרת המילון `CODON_TABLE` , שכבר מוגדר עבורכם בקוד ההתחלתי. הבדיקות הנסתרות כוללות קודון עצירה באמצע הרצף, קודון חלקי בסוף, רצף ללא קודון עצירה ורצף ארוך מאוד.

מודול 5 · אתגר קוד בסגנון LEETCODE

התרגיל: `translate_rna(rna_sequence)`

בינוני | קודונים | מילונים (hash maps) | קוד גנטי | כ-35 דקות

בקוד ההתחלתי מוגדר המילון `CODON_TABLE`, שממפה כל אחד מ-64 הקודונים לקוד בן אות אחת של חומצת אמינו. קודוני העצירה ממופים ל-`"*"`.

ממשו את `translate_rna(rna_sequence)` כך שתתרגם את מחרוזת ה-RNA לחלבון:

- קראו את הרצף בקודונים של 3 אותיות, החל מאינדקס 0.
- החליפו כל קודון בחומצת האמינו שלו והחזירו את התוצאה כמחרוזת.
- עצרו בקודון העצירה הראשון. קודון העצירה עצמו **אינו** נכלל בחלבון.
- אם בסוף נשארות אות אחת או שתיים שאינן מרכיבות קודון שלם, התעלמו מהן.

דוגמאות

דוגמה 1

קלט: `rna_sequence = "AUGGCCAUGGCGCCCAGAACUGAGAUCAAUAGUACCCGUAUUAACGGGUGA"`

פלט: `"MAMAPRTEINSTRING"`

הסבר: הקודון האחרון, UGA, הוא קודון עצירה ולכן אינו מתורגם.

דוגמה 2

קלט: `rna_sequence = "AUGUUUUAAGCC"`

פלט: `"MF"`

הסבר: התרגום נעצר ב-UAA; הקודון GCC שאחריו לא נקרא.

דוגמה 3

קלט: `rna_sequence = "AUGGC"`

פלט: `"M"`

הסבר: האותיות GC בסוף אינן קודון שלם.

אילוצים

- $0 \le \text{len(rna_sequence)} \le 3 \cdot 10^4$
- הרצף מכיל רק את האותיות הגדולות `A`, `C`, `G`, `U`
- יש להחזיר `str`; אם אין חומצות אמינו, מחזירים מחרוזת ריקה

רמזים

- `range(0, len(rna_sequence) - 2, 3)` נותן את תחילתו של כל קודון שלם.
- `CODON_TABLE[codon]` מחזיר את חומצת האמינו בזמן O(1).
- `break` יוצא מהלולאה ברגע שמגיעים ל-`"*"`.
- אספו את חומצות האמינו ברשימה, וחברו אותן בסוף עם `"".join(...)`.

בדיקות גלויות (הרצה)

אלה הבדיקות שכפתור **הרצה** באתר מריץ. הפתרון חייב לעבור את כולן.

קריאה	תוצאה צפויה
`translate_rna("AUGGCCAUGGCGCCCAGAACUGAGAUCAAU…"` (מחרוזת באורך 51)	`"MAMAPRTEINSTRING"`
`translate_rna("AUGUUUUAAGCC")`	`"MF"`
`translate_rna("AUGGC")`	`"M"`

בדיקות נסתרות: מקרי קצה (הגשה)

כפתור **הגשה** מריץ גם את הבדיקות האלה. הן נועדו לתפוס קלטים ריקים, מקרי גבול, יעילות על קלט גדול וטיפול בשגיאות. כתבו את הפתרון כך שיעמוד בהן **לפני** שאתם מציצים בטבלה.

קריאה	תוצאה צפויה	מה נבדק
`translate_rna("")`	`""`	קלט ריק
`translate_rna("UAG")`	`""`	
`translate_rna("GCUGCUGCU")`	`"AAA"`	
`translate_rna("UUUUUCUUAUUG")`	`"FFLL"`	
`translate_rna("AUGUGGUGAUGG")`	`"MW"`	
`translate_rna("UUUUUCUUAUUGUCUUCCUCAUCGUAUUAC…"` (מחרוזת באורך 183)	`"FFLLSSSSYYCCWLLLLPPPPHHQQRRRRI…"` (מחרוזת באורך 61)	
`translate_rna("AUG" + "GCU" * 3000 + "UAGCCC")`	`"M" + "A" * 3000`	קלט גדול (9,009 תווים): יעילות

קוד התחלתי

כך נראה העורך כשפותחים את המודול. ממשו את גוף הפונקציה במקום `pass`.

PYTHON

```python
# The standard genetic code: codon -> one-letter amino acid ("*" = stop)
CODON_TABLE = {
    "UUU": "F", "UUC": "F", "UUA": "L", "UUG": "L",
    "UCU": "S", "UCC": "S", "UCA": "S", "UCG": "S",
    "UAU": "Y", "UAC": "Y", "UAA": "*", "UAG": "*",
    "UGU": "C", "UGC": "C", "UGA": "*", "UGG": "W",
    "CUU": "L", "CUC": "L", "CUA": "L", "CUG": "L",
    "CCU": "P", "CCC": "P", "CCA": "P", "CCG": "P",
    "CAU": "H", "CAC": "H", "CAA": "Q", "CAG": "Q",
    "CGU": "R", "CGC": "R", "CGA": "R", "CGG": "R",
    "AUU": "I", "AUC": "I", "AUA": "I", "AUG": "M",
    "ACU": "T", "ACC": "T", "ACA": "T", "ACG": "T",
    "AAU": "N", "AAC": "N", "AAA": "K", "AAG": "K",
    "AGU": "S", "AGC": "S", "AGA": "R", "AGG": "R",
    "GUU": "V", "GUC": "V", "GUA": "V", "GUG": "V",
    "GCU": "A", "GCC": "A", "GCA": "A", "GCG": "A",
    "GAU": "D", "GAC": "D", "GAA": "E", "GAG": "E",
    "GGU": "G", "GGC": "G", "GGA": "G", "GGG": "G",
}


def translate_rna(rna_sequence):
    """
    Translate an RNA string (read from index 0) into a protein string.
    Stop at the first stop codon; ignore a trailing partial codon.

    >>> translate_rna("AUGUUUUAAGCC")
    'MF'
    """
    # Write your code here
    pass
```

🔑 פתרון עיון מלא: `translate_rna`

הפתרון שעובר את כל הבדיקות, הגלויות והנסתרות. מספרי השורות בהסבר מתייחסים לקוד שלמעלה.

```python
# CODON_TABLE is defined at the top of the starter code.

def translate_rna(rna_sequence):
    protein = []
    for i in range(0, len(rna_sequence) - 2, 3):
        amino_acid = CODON_TABLE[rna_sequence[i:i + 3]]
        if amino_acid == "*":        # stop codon: release the protein
            break
        protein.append(amino_acid)
    return "".join(protein)
```

הסבר שורה אחר שורה

שורה 1 — ההערה מזכירה ש-`CODON_TABLE`, המילון שממפה 64 קודונים לאותיות של חומצות אמינו (`"*"` לעצירה), מוגדר בראש הקוד ההתחלתי. הפתרון נשען עליו, ואינו בונה אותו מחדש.

שורה 4 — **רשימת צובר.** אוספים את חומצות האמינו ברשימה ומחברים בסוף. זה יעיל יותר מחיבור חוזר של מחרוזות עם `+=`.

שורה 5 — **קריאה במסגרת 0, קודון אחר קודון.** `range(0, len(rna_sequence) - 2, 3)` מייצר 0, 3, 6, ..., אבל עוצר כך שתמיד נשארות לפחות 3 אותיות: החסרת 2 מבטיחה שלא ייחתך קודון חלקי בסוף (אות אחת או שתיים מיותרות פשוט לא נקראות).

שורה 6 — **חיפוש במילון.** `rna_sequence[i:i + 3]` חותך קודון, ו-`CODON_TABLE[...]` מחזיר את חומצת האמינו בזמן O(1) בממוצע (טבלת גיבוב). הקלט נקי לפי האילוצים, ולכן אין צורך ב-`.get`.

שורות 7–8 — **קודון עצירה.** כשהערך הוא `"*"`, `break` יוצא מהלולאה מיד, בדיוק כמו ריבוזום שמשחרר את החלבון. קודון העצירה עצמו לא נוסף לרשימה, וכל מה שאחריו לא נקרא.

שורה 9 — כל קודון רגיל מוסיף את חומצת האמינו שלו לרשימה.

שורה 10 — `"".join(protein)` מחבר את הרשימה למחרוזת אחת במעבר יחיד. אם לא נוספה אף חומצת אמינו (למשל רצף ריק או `"UAG"`), מתקבלת מחרוזת ריקה, כנדרש.

⌘ למה זה הפתרון האופטימלי

n/3 חיפושים במילון, כל אחד O(1), ולכן O(n) בזמן; הזיכרון הוא רשימת חומצות האמינו, O(n). חיפוש ברשימה של זוגות היה O(64) לכל קודון, פי עשרות איטי יותר.

💡 איך הפתרון עומד במקרי הקצה הנסתרים

`""` → `""` (ה-`range` ריק); `""` → `"UAG"` (עצירה מיידית); `"AUGGC"` → `"M"` (קודון חלקי בסוף מתעלמים ממנו); `"AUGUGGUGAUGG"` → `"MW"` (ה-UGG שאחרי UGA לא נקרא); כל 61 הקודונים המקודדים ברצף אחד → 61 אותיות; רצף של 9,000 בסיסים → 3,001 אותיות.

מודול 6 · ביולוגיה × מדעי המחשב

מסגרות קריאה פתוחות (ORFs)

בינוני-קשה | כ-45 דקות | קודוני התחלה ועצירה | חיתוך מתקדם | חיזוי גנים | `find_first_orf()`

מעבדה חזותית: שלוש מסגרות קריאה עם קודוני התחלה/עצירה וסורק ORF

בגנום אמיתי אין שלט שאומר "כאן מתחיל גן". כמו בקובץ בינארי, הכול תלוי מאיפה מתחילים לקרוא. אחת הדרכים העיקריות למצוא גנים חדשים היא לחפש **מסגרות קריאה פתוחות**: בלוקים של קוד שמתחילים בקודון התחלה ונמשכים עד קודון עצירה. במודול הזה תכתבו את הגרסה הבסיסית של כלי לחיזוי גנים.

מה תלמדו

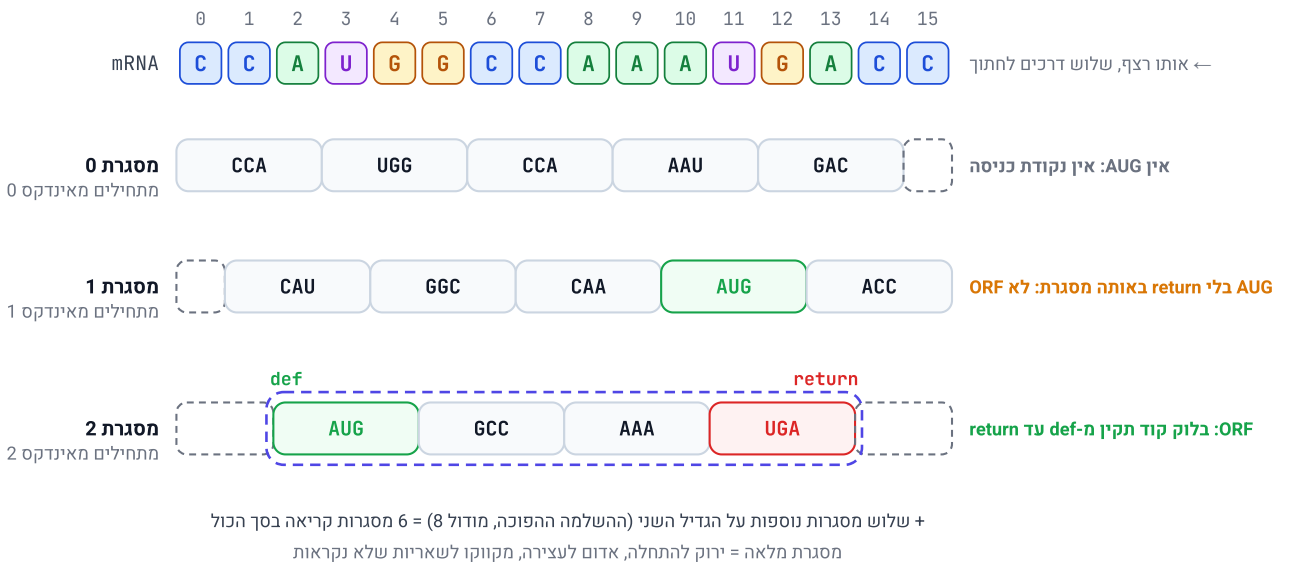
- הביולוגיה בשפה של מתכנתים: מסגרות קריאה כיישור בתים, ORF כבלוק קוד תקין
- קודוני התחלה ועצירה, ומה מגדיר ORF
- שש מסגרות הקריאה של מולקולת DNA
- למה ORF ארוך הוא סימן לגן אמיתי
- חיתוך מתקדם, לולאות עם צעד 3 וחיפוש תת-מחרוזות

1 הביולוגיה בשפה של מתכנתים

קובץ בינארי הוא רצף אחד ארוך של ביטים, ואם מתחילים לקרוא אותו מהבית הלא נכון, כל ה"פקודות" יוצאות ג'יבריש. ל-mRNA יש בדיוק אותה בעיה: הריבוזום קורא שלוש אותיות בכל פעם, ולכן יש שלוש דרכים לחתוך את אותו רצף לקודונים, לפי האות שבה מתחילים (אינדקס 0, 1 או 2). כל חיתוך כזה נקרא **מסגרת קריאה**, ורק אחת מהן נותנת את החלבון הנכון.

מסגרת קריאה פתוחה (ORF) היא בלוק קוד תקין: מתחיל בקודון ההתחלה `AUG` (ה-`def` או ה-`main` של הגן), ממשיך בקודונים שלמים ומסתיים בקודון עצירה (ה-`return`) באותה מסגרת. ל-DNA יש שני גדילים שנקראים בכיוונים הפוכים, ולכן בסך הכול יש שש מסגרות לבדוק: שלוש בכל כיוון. ולמה ORF ארוך מרמז על גן אמיתי? כי ברצף אקראי "return" צץ בממוצע כל 20 פקודות; מי שמצא 300 פקודות רצופות בלי עצירה, כנראה מצא קוד שמישהו (האבולוציה) כתב בכוונה. כלים לחיזוי גנים מתחילים בדיוק מהסריקה הזאת.

איור מסגרות קריאה כיישור בתים: אותו רצף, שלוש "תוכניות" שונות



אותו mRNA נחתך לקודונים משלושה היסטים: 0, 1 ו-2. רק במסגרת 2 מופיע בלוק שלם: קודון התחלה (AUG, הירוק), גוף של קודונים מלאים, וקודון עצירה (UGA, האדום) באותה מסגרת. זה ORF: הקטע שהריבוזום באמת יריץ. במסגרת 1 יש AUG אבל אין עצירה באותה מסגרת, ובמסגרת 0 אין אפילו נקודת כניסה.

מילון מונחים: ביולוגיה ↔ מדעי המחשב

בביולוגיה	במדעי המחשב	למה הדימוי עובד
מסגרת קריאה	**יישור בתים (byte alignment)**	אותו רצף ביטים, שלושה היסטים התחלה, שלוש "תוכניות" שונות לגמרי.
קודון התחלה AUG	**`def` / נקודת כניסה**	מכאן המפרש מתחיל לקרוא; בלי זה אין מה להריץ.
קודון עצירה	**`return`**	סוף הבלוק; חייב להיות באותה מסגרת, אחרת הוא נקרא כפקודה רגילה.
ORF	**בלוק קוד תקין (פונקציה שלמה)**	מ-def עד return, בלי return באמצע; מועמד להיות גן.
שש מסגרות	**3 היסטים × 2 כיווני קריאה**	ל-DNA שני גדילים, וכל אחד נקרא בכיוון שלו, ולכן בודקים את שניהם.
ORF ארוך	**קוד שנכתב בכוונה**	ברצף אקראי עצירה מופיעה כל ~20 קודונים; מאות קודונים בלי עצירה הם אות ולא רעש.

⚡ בשורה אחת

מסגרת קריאה היא ההיסט שממנו חותכים לשלשות, ו-ORF הוא בלוק קוד שמתחיל ב-(AUG) def ונגמר ב-return (קודון עצירה) באותה מסגרת. ככל שהבלוק ארוך יותר, כך פחות סביר שהוא מקרי.

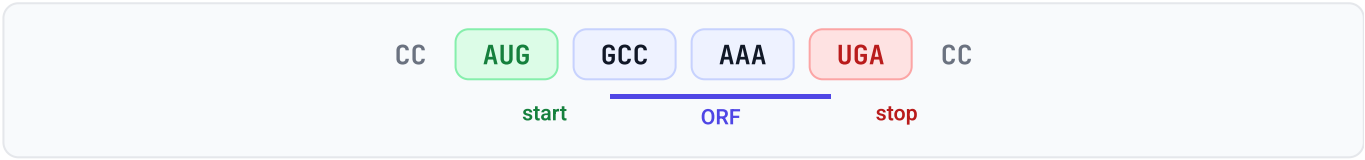
2 התחלה ועצירה

ועכשיו ההגדרה המדויקת של "בלוק קוד תקין" בשפת הביולוגיה.

במודול הקודם תרגמתם רצף מאינדקס 0. בתא אמיתי הריבוזום מחפש את **קודון ההתחלה** `AUG`, ומשם קורא בשלישיות עד שהוא פוגש אחד משלושת **קודוני העצירה**: `UAA`, `UAG` או `UGA`.

מסגרת קריאה פתוחה (ORF, Open Reading Frame) היא קטע רצף ש:

- מתחיל בקודון `AUG`;
- ממשיך בקודונים שלמים, בלי קודון עצירה באמצע;
- מסתיים בקודון עצירה **באותה מסגרת קריאה**.



> ⚠️ **עצירה באותה מסגרת**
>
> ברצף `AUGAUAAGCUAG` מופיע `UAA` באינדקסים 4–6, אבל הוא לא באותה מסגרת כמו ה-`AUG` שבאינדקס 0: המרחק ביניהם, 4, אינו מתחלק ב-3. בקריאה בשלישיות מקבלים `AUG AUA AGC UAG`, וה-ORF מסתיים רק ב-`UAG`.

שש מסגרות קריאה

לכל רצף יש 3 מסגרות קריאה אפשריות: מתחילים מאינדקס 0, 1 או 2. אינדקס 3 כבר מחזיר אותנו למסגרת 0. ל-DNA יש שני גדילים, ולכן כשמחפשים גנים בודקים גם את 3 המסגרות של ההשלמה ההפוכה (מודול 1), ובסך הכול **6 מסגרות קריאה**.

למה ORF ארוך הוא סימן לגן?

ברצף אקראי, 3 מתוך 64 הקודונים הם קודוני עצירה. הסיכוי שקודון אקראי *אינו* קודון עצירה הוא 61/64, והסיכוי לשרשרת של k קודונים בלי עצירה הוא $(61/64)^k$. לכן ORF אקראי נמשך בממוצע כ-20 קודונים בלבד, וכמעט אף פעם לא מגיע ל-100. חלבונים אמיתיים ארוכים בדרך כלל ממאות חומצות אמינו, ולכן ORF ארוך מאוד הוא מועמד טוב לגן. על העיקרון הזה מבוססים כלים כמו NCBI ORFfinder, Prodigal ו-GeneMark, שמחפשים גנים בחיידקים.

סיכוי ל-k קודונים בלי עצירה — ברצף אקראי

$$P(k) = (61/64)^k \quad \rightarrow \quad P(100) \approx 0.008$$

אורך ממוצע לפני עצירה — התפלגות גאומטרית

$$E[\text{length}] = (61/64) \,/\, (3/64) = 61/3 \approx 20.3 \text{ codons}$$

🧬 ובתאים איקריוטיים?

בתאים איקריוטיים גנים מפוצלים לאקסונים ולאינטרונים, וה-ORF השלם מופיע רק ב-mRNA הבוגר, אחרי שתהליך החיבור (splicing) הסיר את האינטרונים. לכן חיזוי גנים בגנום האדם דורש מודלים מורכבים יותר, ונתוני ביטוי כמו RNA-seq.

---

3 מעבדה חזותית: מחפשים ORF

השורה העליונה היא הרצף, ושלוש השורות שמתחתיו מציגות אותו כקודונים בכל אחת משלוש מסגרות הקריאה. קודוני `AUG` מסומנים בירוק, קודוני עצירה באדום, ואזורי ORF מוצללים. לחצו **סריקה** כדי לצפות באלגוריתם של התרגיל: הוא מחפש `AUG`, קורא קדימה בשלישיות, ואם אין קודון עצירה באותה מסגרת, הוא ממשיך ל-`AUG` הבא.

מעבדה חזותית · מסגרות קריאה ו-ORF

שלוש מסגרות קריאה של אותו רצף. AUG בירוק, קודוני עצירה באדום, ואזורי ORF מוצללים.

השורה העליונה היא הרצף; שלוש השורות שמתחתיו מציגות אותו כקודונים בכל אחת משלוש מסגרות הקריאה. ה-AUG באינדקס 2 (מסגרת 2) לא נסגר באף קודון עצירה, ולכן אינו ORF; ה-AUG באינדקס 6 (מסגרת 0) נסגר ב-UGA, וזה ה-ORF הראשון.



קודון התחלה AUG · קודון עצירה · ORF · ה-ORF הראשון (תשובת התרגיל)

מעקב אחרי האלגוריתם של התרגיל, צעד אחר צעד:

1. אינדקס 0: `GGA` אינו AUG. ממשיכים.
2. אינדקס 1: `GAU` אינו AUG. ממשיכים.
3. **נמצא AUG באינדקס 2.** קוראים קדימה בשלישיות, במסגרת 2.
4. קודון באינדקס 5: `AAU` אינו קודון עצירה. ממשיכים 3 עמדות קדימה.
5. קודון באינדקס 8: `GCC` אינו קודון עצירה. ממשיכים 3 עמדות קדימה.
6. קודון באינדקס 11: `CUG` אינו קודון עצירה. ממשיכים 3 עמדות קדימה.
7. קודון באינדקס 14: `AGG` אינו קודון עצירה. ממשיכים 3 עמדות קדימה.
8. קודון באינדקס 17: `CCA` אינו קודון עצירה. ממשיכים 3 עמדות קדימה.
9. קודון באינדקס 20: `UGU` אינו קודון עצירה. ממשיכים 3 עמדות קדימה.
10. קודון באינדקס 23: `UUG` אינו קודון עצירה. ממשיכים 3 עמדות קדימה.
11. קודון באינדקס 26: `GCU` אינו קודון עצירה. ממשיכים 3 עמדות קדימה.
12. קודון באינדקס 29: `AAG` אינו קודון עצירה. ממשיכים 3 עמדות קדימה.
13. **הגענו לסוף הרצף בלי קודון עצירה במסגרת 2.** ה-AUG באינדקס 2 אינו ORF תקין, ולכן ממשיכים לחפש מאינדקס 3.
14. אינדקס 3: `UGA` אינו AUG. ממשיכים.
15. אינדקס 4: `GAA` אינו AUG. ממשיכים.
16. אינדקס 5: `AAU` אינו AUG. ממשיכים.

17. **נמצא AUG באינדקס 6.** קוראים קדימה בשלישיות, במסגרת 0.
18. קודון באינדקס 9: `CCC` אינו קודון עצירה. ממשיכים 3 עמדות קדימה.
19. ✓ **קודון עצירה UGA באינדקס 12!** ה-ORF הראשון הוא אינדקסים 6–14 (3 קודונים). התוצאה: `"AUGCCCUGA"`

בגרסה המקוונת: מקלידים RNA ולוחצים **סריקה** או **צעד**: הסמן בודק כל אינדקס, קופץ בשלישיות ממסגרת ה-AUG שנמצא, וכשנמצא קודון עצירה ה-ORF מודגש. ORF שנכשל מחזיר את החיפוש לאינדקס הבא.

---

4 חיתוך מתקדם ולולאות עם צעד

שלושה כלים יעשו את רוב העבודה: חיתוך עם צעד, `range` עם צעד, ו-`str.find` עם נקודת התחלה.

```python
rna = "CCAUGGCCAAAUGACC"

rna[2:5]                      # 'AUG'    -> one codon
rna[2::3]                     # 'AGAUC'  -> first base of every codon in frame 2
[rna[i:i + 3] for i in range(2, len(rna) - 2, 3)]
                              # ['AUG', 'GCC', 'AAA', 'UGA']  -> codons in frame 2

rna.find("AUG")               # 2        -> first AUG
rna.find("AUG", 3)            # 10       -> next AUG, searching from index 3
```

כדי לעבור על **כל** המופעים, מקדמים את נקודת ההתחלה בעצמכם בלולאת `while`:

```python
i = rna.find("AUG")
while i != -1:
    print("AUG at", i, "-> frame", i % 3)
    i = rna.find("AUG", i + 1)            # +1 so overlapping matches are not skipped
# AUG at 2 -> frame 2
# AUG at 10 -> frame 1
```

ול-Python יש מבנה פחות מוכר שמתאים בדיוק לשאלה "האם מצאנו עצירה?": `for ... else`. בלוק ה-`else` רץ רק אם הלולאה הסתיימה **בלי** `break`.

```python
STOP_CODONS = {"UAA", "UAG", "UGA"}

start = 2                                 # an AUG we found
for i in range(start + 3, len(rna) - 2, 3):
    codon = rna[i:i + 3]
    if codon in STOP_CODONS:
        print("stop", codon, "at", i, "->", rna[start:i + 3])
        break
else:
    print("no in-frame stop codon")
# stop UGA at 11 -> AUGGCCAAAUGA
```

⌘ כמה זה עולה?

הגישה הישירה (לכל `AUG`, לקרוא קדימה עד קודון עצירה) היא $O(n^2)$ במקרה הגרוע: חשבו על `AUGAUGAUG...` בלי קודון עצירה, שבו כל `AUG` נקרא עד הסוף. אפשר לרדת ל-$O(n)$: עוברים פעם אחת על כל מסגרת, זוכרים את ה-`AUG` הפתוח הראשון, וכשמגיעים לקודון עצירה סוגרים את ה-ORF. בתרגיל הגישה הישירה מספיקה, אבל כלים לגנומים שלמים חייבים את הגרסה הלינארית.

זמן $O(n^2)$
הגישה הישירה במקרה הגרוע; $O(n)$ עם מעבר אחד לכל מסגרת

זיכרון $O(n)$
מחרוזת ה-ORF המוחזרת

5 דוגמאות קוד

לפני שתכתבו פתרון משלכם, הנה דוגמאות מוכנות שמשתמשות באותם כלים ורעיונות. מתחת לכל דוגמה מופיע **הפלט המלא** של הרצת הקוד (כפי שהתקבל מ-Python 3), ואחריו ההסבר המפורט. מומלץ להקליד את הקוד בעצמכם, להריץ אותו, ואז לשנות אותו ולבדוק איך הפלט משתנה. בגרסת האתר כל דוגמה ניתנת להרצה בלחיצת כפתור.

דוגמה 1 כל ה-ORFs בשש מסגרות הקריאה של DNA

זו הגרסה ה"מחקרית" של הרעיון: הדוגמה עוברת על שני הגדילים של מולקולת DNA ועל שלוש המסגרות בכל גדיל,

ומדפיסה כל ORF שנמצא. היא עובדת ישירות על DNA, ולכן קודוני העצירה נכתבים עם `T`.

```python
def reverse_complement(dna):
    return dna.translate(str.maketrans("ACGT", "TGCA"))[::-1]

STOPS = {"TAA", "TAG", "TGA"}

def orfs_in_frame(seq, frame):
    orfs, start = [], None
    for i in range(frame, len(seq) - 2, 3):
        codon = seq[i:i + 3]
        if start is None and codon == "ATG":
            start = i
        elif start is not None and codon in STOPS:
            orfs.append(seq[start:i + 3])
            start = None
    return orfs

dna = ("AGCCATGTAGCTAACTCAGGTTACATGGGGATGACCCCGCGACTTGGATTAG"
       "AGTCTCTTTTGGAATAAGCCTGAATGATCCGAGTAGCATCTCAG")

for strand, seq in (("+", dna), ("-", reverse_complement(dna))):
    for frame in range(3):
        for orf in orfs_in_frame(seq, frame):
            print(f"strand {strand} frame {frame}: {orf} ({len(orf) // 3} codons)")
```

```
$ python example.py
strand + frame 0: ATGGGGATGACCCCGCGACTTGGATTAGAGTCTCTTTTGGAATAA (15 codons)
strand + frame 1: ATGTAG (2 codons)
strand - frame 1: ATGTAA (2 codons)
strand - frame 2:
ATGCTACTCGGATCATTCAGGCTTATTCCAAAAGAGACTCTAATCCAAGTCGCGGGGTCATCCCCATGTAACCTGAGTTAG (27
codons)
```

איך זה עובד, שלב אחר שלב

1. `reverse_complement` היא אותה פונקציה ממודול 1. היא נותנת את הגדיל השני, כתוב בכיוון 5′→3′ שלו, כך שאפשר לסרוק אותו בדיוק כמו את הגדיל הראשון.
2. `orfs_in_frame` עוברת על מסגרת אחת **פעם אחת בלבד**, בצעדים של 3. המשתנה `start` זוכר אם יש כרגע `ATG` "פתוח": `None` פירושו שעדיין לא נמצא קודון התחלה.
3. כשאין ORF פתוח ונמצא `ATG`, שומרים את מיקומו. כשיש ORF פתוח ומגיע קודון עצירה, חותכים `seq[start:i + 3]` (כולל קודון העצירה), מוסיפים לרשימה וסוגרים עם `start = None`. זו בדיוק הגרסה הלינארית, O(n), שתוארה למעלה.
4. שימו לב: מכיוון ש-`start` נקבע רק כשאין ORF פתוח, `ATG` פנימי לא פותח ORF חדש. הפונקציה מחזירה את ה-ORF הארוך ביותר שמתחיל בכל אזור, וזה מה שרוב כלי החיזוי מחפשים.
5. הלולאה החיצונית עוברת על שני הגדילים (`"+"` ו-`"-"`) ועל שלוש המסגרות, ובסך הכול 6 מסגרות קריאה. `len(orf) // 3` הוא מספר הקודונים, כולל קודון העצירה.

דוגמה 2 כמה ארוך ORF אקראי? סימולציה

האם החישוב התאורטי (בממוצע כ-20 קודונים לפני עצירה) באמת נכון? הדוגמה מייצרת אלפי רצפים אקראיים ומודדת אותם. זו טכניקה חשובה במחקר: לבדוק טענה הסתברותית בעזרת סימולציה.

PYTHON

```python
import random

random.seed(42)                      # same "random" numbers on every run
STOPS = {"UAA", "UAG", "UGA"}
BASES = "ACGU"

def random_orf_length():
    length = 0
    while True:
        codon = "".join(random.choice(BASES) for _ in range(3))
        if codon in STOPS:
            return length
        length += 1

lengths = [random_orf_length() for _ in range(5000)]

print(f"mean length: {sum(lengths) / len(lengths):.1f} codons (theory: {61 / 3:.1f})")
print("longest of 5,000 random ORFs:", max(lengths), "codons")
print("share longer than 100 codons:", sum(n > 100 for n in lengths) / len(lengths))
```

```
$ python example.py
mean length: 19.9 codons (theory: 20.3)
longest of 5,000 random ORFs: 162 codons
share longer than 100 codons: 0.0054
```

איך זה עובד, שלב אחר שלב

1. `random.seed(42)` קובע את נקודת ההתחלה של מחולל המספרים האקראיים. כך כל הרצה נותנת בדיוק אותן תוצאות, וזה חשוב כדי שתוצאות מחקר יהיו ניתנות לשחזור.
2. `random_orf_length` מגרילה קודון אחר קודון: `random.choice(BASES)` בוחר בסיס אקראי, והגנרטור בתוך `"".join` מחבר שלושה כאלה לקודון.
3. `while True` רץ עד ש-`return` מסיים את הפונקציה. זו לולאה שאינה אינסופית בפועל, כי בכל צעד יש סיכוי של 3/64 להגריל קודון עצירה.
4. רשימת ההבנה מריצה 5,000 ניסויים. `sum(n > 100 for n in lengths)` סופר כמה ORFs עברו את 100 הקודונים, כי `True` נחשב 1, בדיוק כמו במודול 3.
5. השוו את התוצאות לתאוריה: הממוצע צריך להיות קרוב ל-61/3 ≈ 20.3, ושיעור ה-ORFs שעוברים 100 קודונים צריך להיות פחות מאחוז אחד (בתאוריה $(61/64)^{101} \approx 0.8\%$; עם 5,000 ניסויים בלבד יוצא מעט פחות, בגלל רעש סטטיסטי). מכאן ש-ORF של 300 קודונים כמעט בוודאות אינו צירוף מקרים.

6 המשימה שלכם

ממשו את `find_first_orf(rna)`, שמחזירה את ה-ORF הראשון ברצף, כולל קודון העצירה. הבדיקות הנסתרות כוללות `AUG` בלי קודון עצירה, קודון עצירה שאינו באותה מסגרת, רצף ריק ורצפים ארוכים.

מודול 6 · אתגר קוד בסגנון LEETCODE

✎ התרגיל: `find_first_orf(rna)`

בינוני-קשה | קודוני התחלה ועצירה | חיתוך מתקדם | חיזוי גנים | כ-45 דקות

בהינתן מחרוזת RNA `rna`, מצאו את **מסגרת הקריאה הפתוחה הראשונה** ברצף, והחזירו אותה כמחרוזת.

- עברו על עמדות ההתחלה לפי הסדר, מהאינדקס הנמוך לגבוה, וחפשו את הקודון `AUG`.
- מכל `AUG` קראו קדימה בקודונים של 3 אותיות, באותה מסגרת, עד קודון העצירה הראשון: `UAA`, `UAG` או `UGA`.
- אם נמצא קודון עצירה, החזירו את הקטע מה-`AUG` ועד סוף קודון העצירה, **כולל** קודון העצירה.
- אם ל-`AUG` מסוים אין קודון עצירה באותה מסגרת, הוא אינו ORF תקין: המשיכו ל-`AUG` הבא.
- אם אין ברצף אף ORF תקין, החזירו מחרוזת ריקה `""`.

דוגמאות

דוגמה 1

קלט: `rna = "CCAUGGCCUAAGG"`

פלט: `"AUGGCCUAA"`

הסבר: ה-AUG נמצא באינדקס 2, ואחריו GCC ו-UAA.

דוגמה 2

קלט: `rna = "GGAUGAAUGCCCUGAGG"`

פלט: `"AUGCCCUGA"`

הסבר: ל-AUG שבאינדקס 2 אין קודון עצירה באותה מסגרת (AUG AAU GCC CUG AGG), ולכן ממשיכים ל-AUG שבאינדקס 6.

דוגמה 3

קלט: `rna = "AUGAUAAGCUAG"`

פלט: `"AUGAUAAGCUAG"`

הסבר: ה-UAA שבאינדקס 4 אינו באותה מסגרת, ולכן ה-ORF מסתיים ב-UAG.

אילוצים

- $0 \le \text{len(rna)} \le 10^4$
- הרצף מכיל רק את האותיות הגדולות `A`, `C`, `G`, `U`
- יש להחזיר `str` שמתחיל ב-`AUG` ומסתיים בקודון עצירה, או `""`

רמזים

- 💡 `for start in range(len(rna) - 2)` עובר על כל עמדות ההתחלה האפשריות.
- 💡 מ-`AUG` שבאינדקס `start`, הקודונים הבאים מתחילים ב-`start + 3`, `start + 6` וכן הלאה: `range(start + 3, len(rna) - 2, 3)`.
- 💡 `return` מתוך הלולאה הפנימית מסיים את הפונקציה מיד כשנמצא ה-ORF הראשון.
- 💡 שימו לב לחיתוך: `rna[start:i + 3]` כולל את קודון העצירה.

בדיקות גלויות (הרצה)

אלה הבדיקות שכפתור **הרצה** באתר מריץ. הפתרון חייב לעבור את כולן.

קריאה	תוצאה צפויה
`find_first_orf("CCAUGGCCUAAGG")`	`"AUGGCCUAA"`
`find_first_orf("GGAUGAAUGCCCUGAGG")`	`"AUGCCCUGA"`
`find_first_orf("AUGAUAAGCUAG")`	`"AUGAUAAGCUAG"`

בדיקות נסתרות: מקרי קצה (הגשה)

כפתור **הגשה** מריץ גם את הבדיקות האלה. הן נועדו לתפוס קלטים ריקים, מקרי גבול, יעילות על קלט גדול וטיפול בשגיאות. כתבו את הפתרון כך שיעמוד בהן **לפני** שאתם מציצים בטבלה.

קריאה	תוצאה צפויה	מה נבדק
`find_first_orf("")`	`""`	קלט ריק
`find_first_orf("AUGCCC")`	`""`	
`find_first_orf("UAAUAGUGA")`	`""`	
`find_first_orf("AUGUAA")`	`"AUGUAA"`	
`find_first_orf("AUGAUGUAA")`	`"AUGAUGUAA"`	
`find_first_orf("CAUGA")`	`""`	
`find_first_orf("GGAUGAAUGCCCUGAGGCCAUGUUUGGCUAAGC")`	`"AUGCCCUGA"`	
`find_first_orf("AUG" + "GCU" * 3000 + "UGA")`	`"AUG" + "GCU" * 3000 + "UGA"`	קלט גדול (9,006 תווים): יעילות
`find_first_orf("AUG" * 600)`	`""`	קלט גדול (1,800 תווים): יעילות
`find_first_orf("AUG" * 600 + "UAG")`	`"AUG" * 600 + "UAG"`	קלט גדול (1,803 תווים): יעילות

קוד התחלתי

כך נראה העורך כשפותחים את המודול. ממשו את גוף הפונקציה במקום `pass`.

PYTHON

```python
START_CODON = "AUG"
STOP_CODONS = {"UAA", "UAG", "UGA"}


def find_first_orf(rna):
    """
    Return the first open reading frame in rna: from the first AUG that has
    an in-frame stop codon, up to and including that stop codon.
    Return "" if there is no valid ORF.

    >>> find_first_orf("GGAUGAAUGCCCUGAGG")
    'AUGCCCUGA'
    """
    # Write your code here
    pass
```

🔑 פתרון עיון מלא: `find_first_orf`

הפתרון שעובר את כל הבדיקות, הגלויות והנסתרות. מספרי השורות בהסבר מתייחסים לקוד שלמעלה.

```python
START_CODON = "AUG"
STOP_CODONS = {"UAA", "UAG", "UGA"}


def find_first_orf(rna):
    for start in range(len(rna) - 2):
        if rna[start:start + 3] != START_CODON:
            continue
        # Read forward codon by codon, in the same frame as this AUG
        for i in range(start + 3, len(rna) - 2, 3):
            if rna[i:i + 3] in STOP_CODONS:
                return rna[start:i + 3]
        # No in-frame stop for this AUG: try the next one
    return ""
```

הסבר שורה אחר שורה

שורות 2-1 קבועים ברמת המודול: קודון ההתחלה, וקבוצת (`set`) קודוני העצירה. בדיקת שייכות בקבוצה היא O(1), והשמות הופכים את הקוד לקריא.

שורה 6 **הלולאה החיצונית** עוברת על כל עמדת התחלה אפשרית לקודון: `range(len(rna) - 2)` מבטיח שהחיתוך `rna[start:start + 3]` תמיד באורך 3. עוברים על כל האינדקסים, לא רק על כפולות של 3, כי ה-AUG יכול להיות בכל אחת משלוש המסגרות.

שורות 8-7 אם שלוש האותיות באינדקס הנוכחי אינן `AUG` , `continue` מדלג לאינדקס הבא. זו הדרך הקריאה להימנע מקינון עמוק.

שורות 10-9 **הלולאה הפנימית קוראת קדימה באותה מסגרת.** היא מתחילה ב-`start + 3` (הקודון שאחרי ה-AUG) ומתקדמת בצעדים של 3, ושוב `len(rna) - 2` מבטיח קודונים שלמים בלבד.

שורות 12-11 **מצאנו קודון עצירה באותה מסגרת.** `rna[i:i + 3] in STOP_CODONS` בודק את הקודון הנוכחי. ברגע שנמצא, `return rna[start:i + 3]` מחזיר את הקטע מה-AUG ועד סוף קודון העצירה, **כולל** אותו (לכן `i + 3` ולא `i`). ה-`return` מתוך הלולאה המקוננת מסיים את הפונקציה מיד: זהו ה-ORF הראשון.

שורה 13 אם הלולאה הפנימית הסתיימה בלי `return` , ל-AUG הזה אין קודון עצירה במסגרת שלו, והוא אינו ORF תקין. ההערה מסבירה שממשיכים ל-AUG הבא: הלולאה החיצונית פשוט מתקדמת ל-`start + 1` .

שורה 14 אם אף AUG לא נסגר בקודון עצירה, מחזירים מחרוזת ריקה.

⌘ למה זה הפתרון האופטימלי

הגישה הישירה היא $O(n^2)$ במקרה הגרוע (למשל `"AUG" * 600` בלי עצירה: כל AUG נקרא עד הסוף), אבל לקלט של עד 10^4 תווים זה מהיר מאוד, והקוד פשוט ונכון. בכלים לגנומים שלמים עוברים לגרסה הלינארית מהשיעור: מעבר אחד לכל מסגרת, שזוכר את ה-AUG הפתוח הראשון.

💡 איך הפתרון עומד במקרי הקצה הנסתרים

`"AUGCCC"` → `""` (אין עצירה); `"CAUGA"` → `""` (ה-UGA חופף ל-AUG ולא באותה מסגרת, ואין מספיק אותיות); `"AUGUAA"` → `"AUGUAA"` (ה-ORF הקצר ביותר); `"AUGAUGUAA"` → כל הרצף (ה-AUG הראשון מנצח, גם אם יש AUG פנימי); `"GGAUGAAUGCCCUGAGG…"` → ה-AUG באינדקס 2 נכשל ועוברים לאינדקס 6; `"AUG" * 600` → `""`, ועם `"UAG"` בסוף → כל הרצף.

מודול 7 · ביולוגיה × מדעי המחשב

ניקוד יישור רצפים

קשה | כ-50 דקות | הומולוגיה | פונקציות ניקוד | zip על כמה רצפים | `score_alignment()`

מעבדה חזותית: ניקוד עמודה-אחר-עמודה עם התאמה/אי-התאמה/פער וגרף מצטבר

כשמוצאים גן חדש, השאלה הראשונה היא כמעט תמיד: למה הוא דומה? **יישור רצפים** הוא ה-diff של הביולוגיה: הוא מסדר שני רצפים זה מול זה כך שעמדות בעלות מוצא משותף יעמדו באותה עמודה, ו**פונקציית ניקוד** הופכת את ההבדלים למספר אחד. זה הלב של כלים כמו BLAST, שמורצים מיליוני פעמים בכל יום.

מה תלמדו

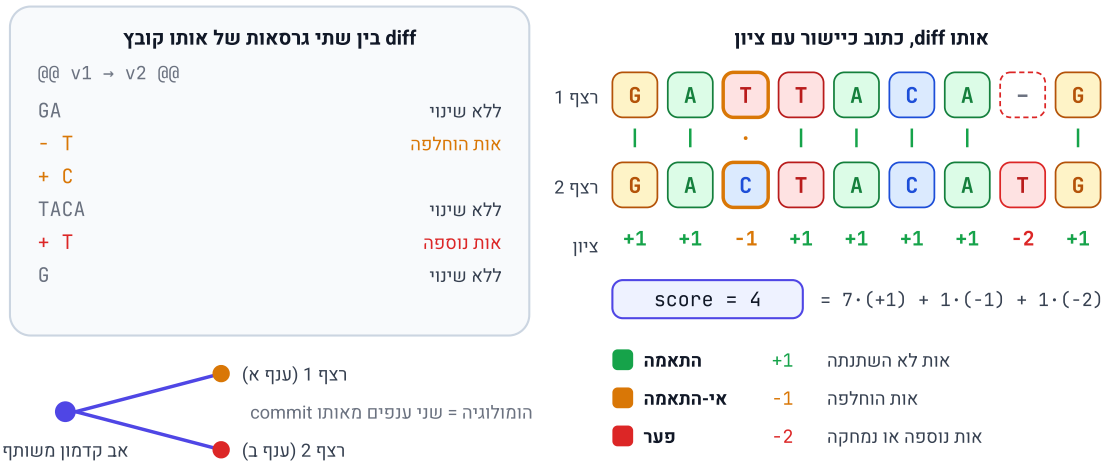
- הביולוגיה בשפה של מתכנתים: יישור כ-diff, הומולוגיה כשני ענפים מאותו commit
- למה מיישרים רצפים: הומולוגיה, זהות ודמיון
- ניקוד של התאמות, אי-התאמות ופערים (gaps)
- מטריצות החלפה, ויישור גלובלי לעומת יישור מקומי
- מעבר על כמה רצפים במקביל ואלגוריתמי ניקוד איטרטיביים

1 הביולוגיה בשפה של מתכנתים

כל מי שעבד עם `git` מכיר את הרעיון: שני קבצים שיצאו מאותו מקור, ו-`diff` שמראה מה השתנה ביניהם. **יישור רצפים** הוא ה-diff של הביולוגיה. שני גנים שירשו מאב קדמון משותף (שני ענפים מאותו commit; ביולוגים אומרים **הומולוגיים**) נבדלים זה מזה בשלושה סוגי עריכה: אות שהוחלפה (**אי-התאמה**), אות שנוספה או נמחקה (**פער**, המקף ביישור), ואותיות שלא נגעו בהן (**התאמה**).

פונקציית הניקוד הופכת את ה-diff למספר אחד: נקודה על כל התאמה, קנס על כל החלפה, וקנס גדול יותר על כל פער, כי הכנסה או השמטה הן אירוע נדיר יותר מהחלפת אות. ככל שהציון גבוה יותר, כך סביר יותר ששני הרצפים באמת קרובים, ולא דומים במקרה. בסוף המודול תכתבו את הפונקציה שמחשבת את הציון הזה עמודה אחר עמודה; במודול 11 תלמדו איך למצוא את היישור הטוב ביותר מבין כל האפשרויות.

איור יישור רצפים הוא diff עם ציון



משמאל: שתי גרסאות של אותו "קובץ" (שני רצפים שירשו מאב קדמון משותף) והעריכות ביניהן: את שהוחלפה, את שנוספה. מימין: אותן עריכות כתובות כיישור, עמודה לכל עמדה, עם מקף במקום האת החסרה. כל עמודה מקבלת ציון, והסכום הוא ציון היישור.

מילון מונחים: ביולוגיה ↔ מדעי המחשב

בביולוגיה	במדעי המחשב	למה הדימוי עובד
רצפים הומולוגיים	**שני ענפים מאותו commit**	לשניהם מקור משותף; ההבדלים הם העריכות שהצטברו מאז הפיצול.
יישור	`diff` **תו-אחר-תו**	מסדר את שני הרצפים כך שכל עמודה מכילה את אותה עמדה מקורית.
התאמה	**שורה ללא שינוי**	אותו תו בשני הרצפים: מקבל נקודה.
אי-התאמה	**שורה שנערכה**	תו שהוחלף במוטציה: קנס קטן.
פער (-)	**שורה שנוספה או נמחקה**	הכנסה או השמטה; קנס גדול יותר, כי זה אירוע נדיר יותר.
ציון היישור	**סכום נקודות וקנסות**	מספר אחד שמסכם כמה טוב היישור; גבוה = קרבה אמיתית.

בשורה אחת

יישור הוא diff בין שני קבצים שהתפצלו מאותו מקור, והציון הוא סכום של נקודות על שורות זהות וקנסות על עריכות, כשהכנסה/מחיקה (פער) עולה יותר מהחלפת אות.

---

2 למה מיישרים רצפים?

נתחיל מהשאלה הביולוגית: מה בעצם מנסים לשחזר כשמיישרים שני רצפים.

שני רצפים שמקורם באב קדמון משותף נקראים **הומולוגיים**. לאורך הדורות הם צברו התמרות, הכנסות והשמטות (מודול 3). יישור מנסה לשחזר את ההיסטוריה הזו: אילו עמדות מקבילות זו לזו, והיכן התרחשו שינויים.

- **חיזוי תפקוד**: אם חלבון לא מוכר דומה מאוד לאנזים ידוע, סביר שגם התפקוד שלו דומה.

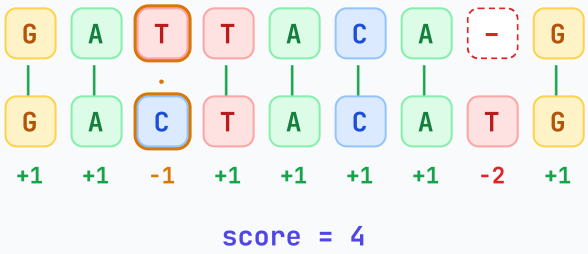
- **אבולוציה ופילוגנטיקה**: כמות ההבדלים ביישור משמשת לבניית עצים, כמו מטריצת המרחקים שבניתם במודול 3.
- **רפואה וגנטיקה**: מיפוי קריאות ריצוף לגנום ייחוס הוא יישור, וכך מזהים את המוטציות של מטופל.
- **אזורים שמורים**: יישור מראה אילו אזורים כמעט לא השתנו בין מינים, ולכן כנראה חשובים לתפקוד.

> 🧬 **הומולוגיה היא כן או לא**
>
> הומולוגיה היא קביעה על היסטוריה: לשני רצפים יש אב קדמון משותף, או שאין להם. לכן לא נכון לומר "75% הומולוגיה". מה שמודדים ביישור הוא **זהות** (אחוז העמדות הזהות) או **דמיון** (שמתחשב גם בהחלפות בין חומצות אמינו בעלות תכונות דומות), ומהם מסיקים אם סביר שהרצפים הומולוגיים.

---

3 איך מנקדים יישור

יישור כותבים כשתי שורות באותו אורך. המקף `-` מסמן **פער** (gap): הכנסה ברצף אחד, או השמטה ברצף השני. את הציון מחשבים עמודה אחר עמודה, ואז מסכמים:



סוג עמודה	ציון לדוגמה	משמעות
התאמה (match)	+1	אותו תו בשתי השורות
אי-התאמה (mismatch)	−1	התמרה: תווים שונים
פער (gap)	−2	הכנסה או השמטה; בדרך כלל נענש בחומרה רבה יותר

בתרגיל של המודול הזה תשתמשו בסכמה הפשוטה ביותר: +1 להתאמה ו-1− לאי-התאמה. כלים אמיתיים משתמשים בסכמות עשירות יותר:

- **מטריצות החלפה**: לחלבונים משתמשים במטריצות כמו BLOSUM62 ו-PAM250, שבהן לכל זוג חומצות אמינו יש ציון משלו. החלפה בין שתי חומצות אמינו דומות (למשל Leu ו-Ile) נענשת פחות מהחלפה בין חומצות שונות מאוד. הציונים הם log-odds: עד כמה סביר לראות את הזוג ברצפים הומולוגיים לעומת צירוף מקרי.
- **ב-DNA** נותנים לפעמים לטרנזיציות עונש קטן יותר מאשר לטרנסברזיות (מודול 3).
- **עונש פער אפיני** (affine gap): פתיחת פער עולה הרבה (למשל 10−), והארכתו עולה מעט (1−). כך פער ארוך אחד עדיף על הרבה פערים קצרים, וזה מתאים לביולוגיה: הכנסה אחת של 6 בסיסים סבירה יותר משש הכנסות נפרדות.

יישור גלובלי ויישור מקומי

הציון רק מעריך יישור נתון. כדי למצוא את היישור הטוב ביותר מבין כל האפשרויות, משתמשים ב**תכנון דינמי** (dynamic programming):

אלגוריתם	מה הוא מוצא	זמן
Needleman–Wunsch (1970)	יישור **גלובלי** של שני הרצפים לכל אורכם. מתאים להשוואת שני גנים או חלבונים באורך דומה.	O(n·m)
Smith–Waterman (1981)	יישור **מקומי**: הקטע הדומה ביותר בתוך רצפים ארוכים, למשל דומיין שמור בתוך חלבון.	O(n·m)
BLAST (1990)	יישורים מקומיים בקירוב, מהר מאוד, מול מאגר שלם כמו GenBank.	היוריסטי, מהיר בהרבה בפועל

כל האלגוריתמים האלה משתמשים, כאבן בניין, בפונקציית ניקוד כמו זו שתכתבו עכשיו.

---

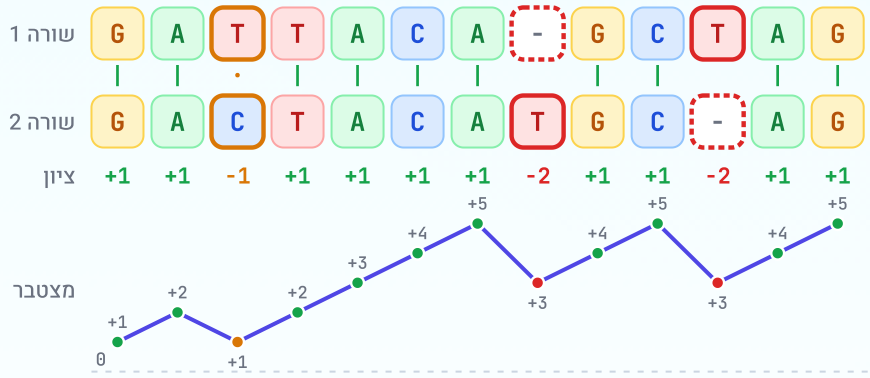
4 מעבדה חזותית: ניקוד יישור

ערכו את שתי השורות (השתמשו ב-`-` לפערים) ושנו את ערכי הניקוד. כל עמודה מקבלת ציון, והגרף שמתחת מציג את **הציון המצטבר** לאורך היישור. לחצו **סריקה** כדי לצפות בלולאה שמחשבת את הסכום עמודה אחר עמודה, ונסו להזיז פער כדי לראות איך הציון משתנה.

מעבדה חזותית · ניקוד יישור

כל עמודה מקבלת ציון. הסכום המצטבר הוא ציון היישור כולו.

כל עמודה מקבלת ציון לפי הסכמה התאמה +1 · אי-התאמה −1 · פער −2. הגרף שמתחת מציג את הסכום המצטבר עמודה אחר עמודה: כל התאמה מעלה אותו, וכל פער מוריד אותו בחדות.



ציון	אחוז זהות	ציון בתרגיל (+...	התאמות	אי-התאמות	פערים
+5	76.9%	+7	10	1	2

ציון היישור בסכמה הנוכחית: **+5**. בסכמת התרגיל, שבה כל תו (כולל `-`) מושווה כפי שהוא, הציון הוא **+7**. 13 עמודות, ופעולת חיבור אחת לכל עמודה, ולכן **O(n)**.

בגרסה המקוונת: עורכים את שתי השורות (עם `-` לפערים), משנים את ערכי ההתאמה, אי-ההתאמה והפער, ולוחצים **סריקה** כדי לצפות בצובר מתקדם עמודה אחר עמודה. הזזת פער משנה מיד את הציון ואת הגרף.

5 מעבר על כמה רצפים במקביל

במודול 3 עברתם על שני רצפים עם `zip`. הפונקציה הזו מקבלת כל מספר של רצפים, ומחזירה בכל צעד עמודה: tuple עם תו אחד מכל רצף.

```python
seq1, seq2, seq3 = "GATTACA", "GACTACA", "GATTGCA"

for column in zip(seq1, seq2, seq3):
    print(column)        # ('G', 'G', 'G'), ('A', 'A', 'A'), ('T', 'C', 'T'), ...

alignment = [seq1, seq2, seq3]
columns = list(zip(*alignment))     # * unpacks the list into separate arguments
print(len(columns), columns[2])     # 7 ('T', 'C', 'T')
```

אלגוריתם ניקוד איטרטיבי

ניקוד הוא שוב תבנית הצובר: מתחילים מאפס, ובכל עמודה מוסיפים את הציון שלה. כדאי להפריד את כלל הניקוד מהלולאה, כך שאפשר להחליף סכמה בלי לגעת בשאר הקוד:

```python
MATCH, MISMATCH, GAP = 1, -1, -2

def column_score(a, b):
    if a == "-" or b == "-":
        return GAP
    return MATCH if a == b else MISMATCH

def score(seq1, seq2, scorer=column_score):
    total = 0
    for a, b in zip(seq1, seq2):
        total += scorer(a, b)
    return total

print(score("GATTACA", "GACTACA"))    # 5
```

> ⌘ **פונקציות כערכים**
>
> ב-Python אפשר להעביר פונקציה כפרמטר, כמו `scorer=column_score` בדוגמה. כך מחליפים כלל ניקוד (למשל BLOSUM62) בלי לשכתב את הלולאה. כך בנויות גם ספריות כמו Biopython, למשל `Bio.Align.PairwiseAligner`.

> ⚠ **ציון גבוה אינו בהכרח יישור טוב**
>
> הציון תלוי באורך: יישור ארוך צובר יותר נקודות גם אם הוא פחות טוב באופן יחסי. לכן מסתכלים גם על **אחוז הזהות**, ובכלים כמו BLAST משתמשים ב-E-value: כמה יישורים עם ציון כזה היינו מצפים למצוא במקרה, במאגר בגודל הזה.

זמן	זיכרון
עמודה אחת לכל עמדה ביישור	צובר אחד בלבד
O(n)	O(1)

6 דוגמאות קוד

לפני שתכתבו פתרון משלכם, הנה דוגמאות מוכנות שמשתמשות באותם כלים ורעיונות. מתחת לכל דוגמה מופיע **הפלט המלא** של הרצת הקוד (כפי שהתקבל מ-Python 3), ואחריו ההסבר המפורט. מומלץ להקליד את הקוד בעצמכם, להריץ אותו, ואז לשנות אותו ולבדוק איך הפלט משתנה. בגרסת האתר כל דוגמה ניתנת להרצה בלחיצת כפתור.

דוגמה 1 דוח יישור מלא: ציון, פערים ואחוז זהות

זה הפלט שכלי יישור מציגים: שתי השורות, שורת סימונים ביניהן וסיכום. הדוגמה משתמשת בסכמה עם עונש פער, ומונה כל סוג עמודה בנפרד.

PYTHON

```python
def alignment_report(top, bottom, match=1, mismatch=-1, gap=-2):
    if len(top) != len(bottom):
        raise ValueError("rows of an alignment must have equal length")
    score = matches = mismatches = gaps = 0
    marks = []
    for a, b in zip(top, bottom):
        if a == "-" or b == "-":
            score += gap
            gaps += 1
            marks.append(" ")
        elif a == b:
            score += match
            matches += 1
            marks.append("|")
        else:
            score += mismatch
            mismatches += 1
            marks.append(".")
    identity = 100 * matches / len(top) if top else 0.0
    print(top)
    print("".join(marks))
    print(bottom)
    print(f"score={score}  matches={matches}  mismatches={mismatches}  "
          f"gaps={gaps}  identity={identity:.1f}%")

alignment_report("GATTACA-GCT", "GACTACAAG-T")
```

```
$ python example.py
GATTACA-GCT
||.|||| | |
GACTACAAG-T
score=3  matches=8  mismatches=1  gaps=2  identity=72.7%
```

איך זה עובד, שלב אחר שלב

1. הפונקציה מתחילה בבדיקת תקינות: בשתי שורות של יישור חייב להיות אותו מספר עמודות. זה אותו דפוס הגנתי שתממשו בתרגיל.
2. `score = matches = mismatches = gaps = 0` מאתחל ארבעה צוברים בשורה אחת. בכל עמודה מתעדכנים הציון ואחד המונים.
3. סדר התנאים חשוב: קודם בודקים פער (`"-"` באחת השורות), ורק אחר כך התאמה. אחרת, שני פערים זה מול זה היו נספרים בטעות כהתאמה.
4. הרשימה `marks` בונה את שורת הסימונים: `|` להתאמה, `.` לאי-התאמה ורווח לפער, כמו בפלט של BLAST ו-EMBOSS.
5. `identity` מחושב רק אם השורות אינן ריקות (ביטוי תנאי בשורה אחת), כדי להימנע מחלוקה באפס, כמו בתכולת GC במודול 2.
6. הפרמטרים `match=1, mismatch=-1, gap=-2` הם ארגומנטי ברירת מחדל, כך שקל לנסות סכמות אחרות: `alignment_report(a, b, gap=-5)`.

דוגמה 2 יישור מרובה: רצף קונצנזוס ועמודות שמורות

ביישור מרובה (MSA) של כמה רצפים, `zip(*alignment)` מחזיר את העמודות. מכל עמודה מחשבים את הבסיס השכיח

ואת מידת השימור, ומקבלים רצף קונצנזוס ושורת סימון בסגנון Clustal.

```python
from collections import Counter

alignment = [
    "ATGCTAGCTAG",
    "ATGCTGGCTAG",
    "ATGATAGCTGG",
    "ATGCTAGTTAG",
]

consensus, marks = [], []
for column in zip(*alignment):
    base, count = Counter(column).most_common(1)[0]
    consensus.append(base)
    share = count / len(column)
    marks.append("*" if share == 1 else ":" if share >= 0.75 else " ")

for row in alignment:
    print(row)
print("".join(marks))
print("consensus:", "".join(consensus))
print("fully conserved columns:", [i for i, m in enumerate(marks) if m == "*"])
```

```
$ python example.py
ATGCTAGCTAG
ATGCTGGCTAG
ATGATAGCTGG
ATGCTAGTTAG
**:*:*:*:*
consensus: ATGCTAGCTAG
fully conserved columns: [0, 1, 2, 4, 6, 8, 10]
```

איך זה עובד, שלב אחר שלב

1. `zip(*alignment)` פורק את רשימת הרצפים לארבעה ארגומנטים נפרדים, ולכן בכל צעד מתקבלת עמודה אחת: tuple עם התו מכל רצף.
2. `Counter(column).most_common(1)` מחזיר רשימה עם הזוג השכיח ביותר, `(base, count)`. האינדקס `[0]` שולף את הזוג, והפירוק `base, count = ...` פותח אותו לשני משתנים.
3. `share` הוא החלק של הרצפים שמסכימים עם הבסיס השכיח. הביטוי התנאי המשורשר בוחר `*` לעמודה שמורה לגמרי, `:` לשמורה ברובה (75% לפחות), ורווח לשאר.
4. הדפסת השורות זו מעל זו, עם שורת הסימונים מתחתיהן, יוצרת תצוגה כמו של Clustal Omega: אפשר לראות במבט אחד היכן הרצפים שונים.
5. רצף הקונצנזוס הוא "הנציג" של המשפחה. מעמודות שמורות לגמרי נוטים לבנות פריימרים ולחפש אתרים חיוניים לתפקוד.

דוגמה 3 מציאת המיקום הטוב ביותר לקריאה קצרה

ממפי קריאות מחפשים היכן בגנום קריאה קצרה "מתיישבת" הכי טוב. הדוגמה מחליקה קריאה לאורך רצף ייחוס, מנקדת כל היסט בלי פערים ובוחרת את הציון הגבוה ביותר. זה שילוב של החלון הנע ממודול 4 עם פונקציית ניקוד.

PYTHON

```python
def score(a, b):
    return sum(1 if x == y else -1 for x, y in zip(a, b))

reference = "TTGACCGATTACAGGCATGCA"
read = "GATTACA"
offsets = range(len(reference) - len(read) + 1)

scores = [score(reference[o:o + len(read)], read) for o in offsets]
best = max(offsets, key=lambda o: scores[o])

print("scores:", scores)
print(f"best offset {best}: {reference[best:best + len(read)]} (score {scores[best]})")
```

```
$ python example.py
scores: [-5, -5, -3, -5, -5, -5, 7, -5, -5, -5, -3, -5, -5, -5, 1]
best offset 6: GATTACA (score 7)
```

איך זה עובד, שלב אחר שלב

1. `score` היא הגרסה הקצרה של כלל הניקוד: ביטוי תנאי בתוך גנרטור, ו- `sum` מסכם את כל העמודות.
2. `offsets` הם כל המיקומים שבהם הקריאה נכנסת במלואה לתוך הייחוס: $n - m + 1$ מיקומים, בדיוק כמו מספר החלונות במודול 4.
3. לכל היסט חותכים חלון מהייחוס באורך הקריאה ומנקדים אותו מולה. כך מתקבלת רשימת ציונים, אחד לכל היסט.
4. `max(offsets, key=lambda o: scores[o])` מחזיר את ההיסט עם הציון הגבוה ביותר, ולא את הציון עצמו. הפרמטר `key` קובע לפי מה משווים, ו- `lambda` היא פונקציה קצרה ואנונימית.
5. הציון 7 באופסט 6 הוא התאמה מושלמת. ממפים אמיתיים, כמו BWA ו-Bowtie, מאיצים את החיפוש בעזרת אינדקס (FM-index) ומאפשרים גם פערים, אבל הרעיון של "לנקד ולבחור את הטוב ביותר" זהה.

---

7 המשימה שלכם

ממשו את `score_alignment(seq1, seq2)`: +1 לכל התאמה ו-1− לכל אי-התאמה. הבדיקות הנסתרות כוללות יישורים ריקים, פערים, רצפי חלבון, אורכים שונים (שדורשים `ValueError`) ויישור ארוך במיוחד.

מודול 7 · אתגר קוד בסגנון LEETCODE

התרגיל: `score_alignment(seq1, seq2)`

קשה | הומולוגיה | פונקציות ניקוד | zip על כמה רצפים | כ-50 דקות

נתונים שני רצפים **מיושרים**, כמחרוזות באורך שווה: התו באינדקס `i` ב-`seq1` עומד מול התו באינדקס `i` ב-`seq2`.

ממשו את `score_alignment(seq1, seq2)` כך שתחזיר את ציון היישור כ-`int`:

- **+1** לכל עמודה שבה התווים זהים (התאמה).
- **−1** לכל עמודה שבה התווים שונים (אי-התאמה).
- כל תו מושווה כפי שהוא, כולל `-`: בתרגיל הזה פער הוא פשוט עוד תו. (בכלים אמיתיים, ובמעבדה החזותית, לפערים יש עונש נפרד.)
- אם האורכים שונים, הרצפים אינם יישור תקין: זרקו `ValueError`.

דוגמאות

דוגמה 1

קלט: `seq1 = "GATTACA", seq2 = "GATTACA"`

פלט: `7`

הסבר: שבע התאמות.

דוגמה 2

קלט: `seq1 = "GATTACA", seq2 = "GCTTAGA"`

פלט: `3`

הסבר: חמש התאמות ושתי אי-התאמות: $5 - 2 = 3$.

דוגמה 3

קלט: `seq1 = "AAAA", seq2 = "TTTT"`

פלט: `-4`

הסבר: ארבע אי-התאמות.

אילוצים

- $0 \le$ `len(seq1), len(seq2)` $\le 10^5$
- הרצפים יכולים להכיל כל תו: בסיסים, חומצות אמינו או `-`
- יש להחזיר `int`, ולזרוק `ValueError` אם `len(seq1) != len(seq2)`

רמזים

- התחילו בבדיקת האורכים, וזרקו `raise ValueError("...")` אם הם שונים.
- `zip(seq1, seq2)` נותן את עמודות היישור אחת אחרי השנייה.
- צוברים `score = 0`, ובכל עמודה `score += MATCH` או `score += MISMATCH`.
- יישור ריק הוא יישור תקין, והציון שלו 0.

בדיקות גלויות (הרצה)

אלה הבדיקות שכפתור **הרצה** באתר מריץ. הפתרון חייב לעבור את כולן.

קריאה	תוצאה צפויה
`score_alignment("GATTACA", "GATTACA")`	`7`
`score_alignment("GATTACA", "GCTTAGA")`	`3`
`score_alignment("AAAA", "TTTT")`	`-4`
`score_alignment("ACGT", "ACG")`	`raise ValueError`

בדיקות נסתרות: מקרי קצה (הגשה)

כפתור **הגשה** מריץ גם את הבדיקות האלה. הן נועדו לתפוס קלטים ריקים, מקרי גבול, יעילות על קלט גדול וטיפול בשגיאות. כתבו את הפתרון כך שיעמוד בהן **לפני** שאתם מציצים בטבלה.

קריאה	תוצאה צפויה	מה נבדק
`score_alignment("", "")`	`0`	קלט ריק
`score_alignment("A", "A")`	`1`	תו בודד
`score_alignment("A", "C")`	`-1`	תו בודד
`score_alignment("AC-GT", "ACAGT")`	`3`	
`score_alignment("MKTAYIAK", "MKTAHIAK")`	`6`	
`score_alignment("ACG", "ACGT")`	`raise ValueError`	קלט לא תקין: חייב לזרוק `ValueError`
`score_alignment("ACGT" * 2500, "ACGA" * 2500)`	`5000`	קלט גדול (10,000 תווים): יעילות

קוד התחלתי

כך נראה העורך כשפותחים את המודול. ממשו את גוף הפונקציה במקום `pass`.

PYTHON

```python
MATCH = 1
MISMATCH = -1


def score_alignment(seq1, seq2):
    """
    Score two aligned sequences of equal length:
    +1 for every matching column, -1 for every mismatch.
    Raise ValueError if the lengths differ.

    >>> score_alignment("GATTACA", "GCTTAGA")
    3
    """
    # Write your code here
    pass
```

🔑 פתרון עיון מלא: `score_alignment`

הפתרון שעובר את כל הבדיקות, הגלויות והנסתרות. מספרי השורות בהסבר מתייחסים לקוד שלמעלה.

```python
MATCH = 1
MISMATCH = -1


def score_alignment(seq1, seq2):
    if len(seq1) != len(seq2):
        raise ValueError("Aligned sequences must have equal length")
    score = 0
    for a, b in zip(seq1, seq2):
        score += MATCH if a == b else MISMATCH
    return score
```

הסבר שורה אחר שורה

שורות 1–2 קבועי הניקוד ברמת המודול. הפרדת כלל הניקוד מהלולאה מאפשרת להחליף סכמה (למשל BLOSUM62) בלי לגעת בשאר הקוד.

שורות 6–7 **בדיקת תקינות.** שתי שורות של יישור חייבות להיות באותו אורך; אחרת זורקים `ValueError`, בדיוק כמו במרחק המינג (מודול 3). הבדיקה קודמת ללולאה, כדי ש-`zip` לא יחתוך בשקט את הקלט הארוך.

שורה 8 **הצובר.** הציון מתחיל מאפס, כי יישור ריק הוא יישור תקין בציון 0.

שורות 9–10 **עמודה אחר עמודה.** `zip(seq1, seq2)` מחזיר בכל צעד עמודה אחת: זוג התווים שעומדים זה מול זה. הביטוי התנאי `MATCH if a == b else MISMATCH` בוחר את ציון העמודה, ו-`+=` מוסיף אותו לצובר. בתרגיל הזה `-` הוא פשוט עוד תו: `"-"` מול `"A"` היא אי-התאמה, ו-`"-"` מול `"-"` היא התאמה.

שורה 11 מחזירים את הסכום כ-`int`.

⌘ למה זה הפתרון האופטימלי

עמודה אחת לכל עמדה ביישור, $O(n)$ בזמן ו-$O(1)$ בזיכרון נוסף. אפשר לקצר ל-`sum(MATCH if a == b else MISMATCH for a, b in zip(seq1, seq2))`, אבל הגרסה עם הצובר המפורש היא זו שמרחיבים בקלות לעונשי פער ולמטריצות החלפה.

💡 איך הפתרון עומד במקרי הקצה הנסתרים

`""`, `""` → 0; `"A"`, `"C"` → −1; `"AC-GT"`, `"ACAGT"` → 3 (ארבע התאמות ואי-התאמה אחת של הפער); רצפי חלבון `"MKTAYIAK"`, `"MKTAHIAK"` → 6; אורכים שונים → `ValueError`; שני רצפים של 10,000 תווים → 5,000.

מודול 8 · ביולוגיה × מדעי המחשב

השלמה הפוכה (Reverse Complement)

בינוני-קשה | כ-40 דקות | אנטי-מקביליות | היפוך מחרוזות | טבלאות תרגום | `reverse_complement()`

מעבדה חזותית: השלמה ואז היפוך, עם תוויות 5′/3′ וזיהוי פלינדרומים

ל-DNA יש שני גדילים, וכל אחד מהם נושא את אותו מידע בצורת מראה. כשרצף נשמר במאגר, נכתב בו רק גדיל אחד, ואת השני צריך לחשב. במודול הזה תבינו למה החישוב הזה הוא map ועוד reverse, ולא map בלבד, ותכתבו את אחת הפונקציות השימושיות ביותר בביואינפורמטיקה.

מה תלמדו

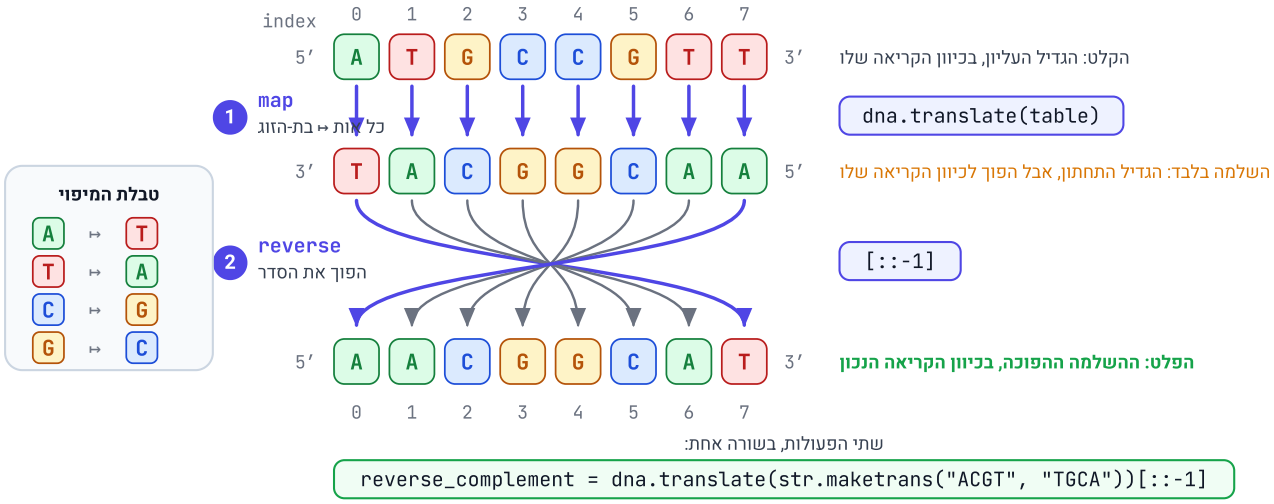
- הביולוגיה בשפה של מתכנתים: שני גדילים כשני מערכים ראש-לזנב, השלמה הפוכה כ-map ו-reverse
- שני גדילים, כיווניות 5′→3′, ומה פירוש "אנטי-מקביליים"
- למה את הגדיל השני קוראים בסדר הפוך
- היפוך מחרוזות וחיתוך עם צעד שלילי
- טבלאות תרגום (`str.maketrans`) ומיפוי תווים

1 הביולוגיה בשפה של מתכנתים

DNA נשמר בשני גדילים שמונחים "ראש לזנב": הגדיל השני הוא אותו מידע, אבל כתוב במראה. דמיינו שני מערכים שמחוברים זה לזה, כאשר האינדקס 0 של אחד יושב מול האינדקס האחרון של השני. לכל גדיל יש "כיוון קריאה" טבעי (מ-5′ ל-3′), כמו קובץ שקוראים תמיד מההתחלה לסוף. כדי לקבל את הגדיל השני בכיוון הקריאה שלו, צריך שני צעדים: להחליף כל אות בבת-זוגה (**השלמה**: `A↔T`, `C↔G`), ולהפוך את סדר המערך (**היפוך**).

בשפה של תכנות זה צינור (pipeline) קצר של שתי פעולות סטנדרטיות: `map` עם טבלת מיפוי, ואז `reverse`. ב-Python: `dna.translate(table)[::-1]`. הסדר לא משנה, אבל שתי הפעולות חובה: השלמה בלי היפוך נותנת מחרוזת שקוראים מהכיוון הלא נכון, וכל חיפוש או תרגום עליה ייתן תוצאות שגויות. בהמשך תראו למה הכיווניות הזאת היא תוצאה של המבנה הכימי של הגדיל, ואיפה היא פוגשת אתכם בנתוני ריצוף אמיתיים.

איור השלמה הפוכה כצינור של שתי פעולות: map ואז reverse



שלב 1 (map): כל אות בגדיל העליון מוחלפת בבת-הזוג שלה לפי טבלת המיפוי, וכך מתקבל הגדיל התחתון, אבל כתוב בכיוון הקריאה של העליון. שלב 2 (reverse): הופכים את סדר התאים (האינדקס 0 עובר לסוף), וכך הגדיל התחתון נכתב בכיוון הקריאה שלו. סדר שני השלבים לא משנה, אבל שניהם חובה.

מילון מונחים: ביולוגיה ↔ מדעי המחשב

בביולוגיה	במדעי המחשב	למה הדימוי עובד
שני גדילים	**שני מערכים ראש-לזנב**	אותו מידע פעמיים, בצורת מראה; האינדקס הראשון של אחד מול האחרון של השני.
כיוון 5′→3′	**כיוון הקריאה של הקובץ**	כל גדיל נקרא תמיד מההתחלה שלו לסוף שלו; המכונות בתא לא יודעות לקרוא הפוך.
השלמה	**`map` עם טבלת מיפוי**	החלפה תו-לתו לפי טבלה קבועה: A↔T, C↔G.
היפוך	**`reverse` / `[::-1]`**	הפיכת סדר התאים במערך.
השלמה הפוכה	**צינור: map ואז reverse**	שתי הפעולות יחד; מה שמאגרי מידע וכלי מיפוי קריאות עושים כל הזמן.
פלינדרום ביולוגי	**מחרוזת ששווה להשלמה ההפוכה שלה**	לא היפוך רגיל של מחרוזת, אלא היפוך + השלמה; למשל GAATTC.

⚡ בשורה אחת

הגדיל השני הוא המערך שלכם במראה: החליפו כל אות בבת-זוגה (map) והפכו את הסדר (reverse). ב-Python זה translate ואז [::-1].

2 סליל כפול, שני כיוונים

> מה שמאחורי הדימוי של "ראש לזנב": הכימיה שמקנה לכל גדיל כיוון.

כל נוקלאוטיד מחובר לשכנו דרך קבוצת פוספט, שמקשרת בין פחמן 3′ של סוכר אחד לפחמן 5′ של הסוכר הבא. לכן לכל גדיל יש "ראש" ו"זנב": **קצה 5′**, עם קבוצת פוספט חופשית, ו**קצה 3′**, עם קבוצת הידרוקסיל חופשית. לפי המוסכמה, רצפים נכתבים תמיד **מ-5′ ל-3′**.

שני הגדילים בסליל הם **אנטי-מקביליים**: הם פונים לכיוונים הפוכים. כשגדיל אחד רץ משמאל לימין מ-5′ ל-3′, הגדיל שמולו רץ מ-3′ ל-5′. רק בסידור הזה הבסיסים מתיישבים זה מול זה ויוצרים קשרי מימן.

```
5′ גדיל עליון   A T G C C G T T  3′
3′ גדיל תחתון   T A C G G C A A  5′
                ↓ קוראים את הגדיל התחתון בכיוון 3′→5′
5′ השלמה הפוכה  A A C G G C A T  3′
```

כדי לכתוב את הגדיל השני לפי המוסכמה, מ-5′ ל-3′, צריך שני צעדים: **להשלים** כל בסיס (A↔T, C↔G) ו**להפוך** את הסדר. התוצאה נקראת **ההשלמה ההפוכה** (reverse complement). סדר הצעדים לא משנה: אפשר להשלים ואז להפוך, או להפוך ואז להשלים.

> ⚠️ **טעות נפוצה: השלמה בלי היפוך**
>
> השלמה בלבד (`TACGGCAA`) נותנת את הגדיל השני, אבל כתוב מ-3′ ל-5′. אם תחפשו בו מוטיב או תתרגמו אותו, תקבלו תוצאות שגויות לגמרי, כי ריבוזומים, פולימראזות וגורמי שעתוק קוראים תמיד מ-5′ ל-3′.

> 🧬 **תוצאה של אנטי-מקביליות: מקטעי אוקזאקי**
>
> DNA פולימראז יכול להאריך גדיל רק בכיוון 3′→5′. במזלג השכפול, גדיל אחד (הגדיל המוביל) נבנה ברצף אחד, אבל הגדיל השני (הגדיל המפגר) נבנה במקטעים קצרים, מקטעי אוקזאקי, שמחוברים אחר כך על ידי אנזים ליגאז. כל זה קורה רק משום ששני הגדילים פונים לכיוונים הפוכים.

איפה זה פוגש אתכם במחקר

- **קריאות ריצוף**: מכשיר ריצוף קורא מקטעים משני הגדילים באקראי. כדי למפות קריאה לגנום, מנסים גם אותה וגם את ההשלמה ההפוכה שלה. בקובצי SAM, קריאה שמופתה לגדיל ההפוך מסומנת בדגל 16.
- **תכנון פריימרים**: הפריימר ההפוך (reverse primer) ב-PCR הוא ההשלמה ההפוכה של הרצף בקצה האזור שרוצים להגביר.
- **גנים על הגדיל השלילי**: בערך מחצית מהגנים יושבים על הגדיל "השני". קובצי GFF מסמנים זאת ב- `-` בעמודת ה-strand.
- **פלינדרומים**: אתרי הגבלה כמו `GAATTC` שווים להשלמה ההפוכה של עצמם (מודול 4).

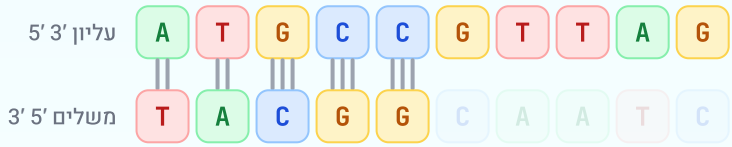
3 מעבדה חזותית: משלימים והופכים

הקלידו רצף ולחצו **הדגמה**. בשלב הראשון כל בסיס מקבל את המשלים שלו, והגדיל התחתון נבנה בכיוון 5′→3′. בשלב השני קוראים את הגדיל התחתון מהסוף להתחלה, וכותבים אותו מחדש בכיוון 3′→5′. נסו גם פלינדרום, למשל `GAATTC`.

מעבדה חזותית · השלמה הפוכה

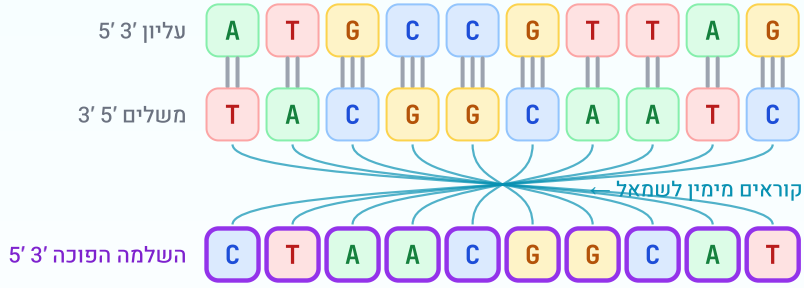
שלב 1: משלימים כל בסיס. שלב 2: קוראים את הגדיל המשלים מהסוף להתחלה.

שלב 1 · השלמה: לכל בסיס בגדיל העליון (5′→3′) מצמידים את המשלים שלו (A↔T, C↔G). הגדיל המשלים נבנה מתחתיו, אבל כתוב מ-3′ ל-5′. כאן הושלמו 5 העמדות הראשונות.



שלב 1 · השלמה, עמדה 4: `C ↔ G`. עד כה: `TACGG` (בכיוון 3′→5′).

שלב 2 · היפוך: קוראים את הגדיל המשלים מהסוף להתחלה, כלומר מקצה ה-5′ שלו, וכותבים אותו מחדש משמאל לימין. הקווים מראים מאיזו עמדה בגדיל המשלים מגיע כל בסיס בתוצאה: result[k] = complement[n − 1 − k].



סיום. ההשלמה ההפוכה: `CTAACGGCAT` שימו לב שהיא שונה גם מהרצף המקורי וגם מההשלמה בלבד (TACGGCAATC).

לשם השוואה, הפלינדרום `GAATTC` (אתר EcoRI): ההשלמה שלו היא `CTTAAG`, ובקריאה הפוכה מתקבל שוב `GAATTC`: **הרצף שווה להשלמה ההפוכה של עצמו.**

בגרסה המקוונת: מקלידים רצף ולוחצים **הדגמה** או **צעד**: תחילה הגדיל המשלים נבנה בסיס אחר בסיס, ואחר כך כל בסיס בתוצאה "נשלף" מהקצה הרחוק של הגדיל המשלים. נסו פלינדרום כמו GAATTC.

4 היפוך מחרוזות ומיפוי תווים

היפוך ומיפוי הם שתי פעולות נפרדות, ול-Python יש כלי טוב לכל אחת מהן:

```python
dna = "ATGCCGTT"

dna[::-1]                          # 'TTGCCGTA'  -> slice with step -1
"".join(reversed(dna))             # 'TTGCCGTA'  -> reversed() gives an iterator

COMPLEMENT = {"A": "T", "T": "A", "C": "G", "G": "C"}
"".join(COMPLEMENT[b] for b in dna)              # 'TACGGCAA'  (complement only)
"".join(COMPLEMENT[b] for b in reversed(dna))    # 'AACGGCAT'  (reverse complement)
```

טבלאות תרגום

`str.maketrans(from, to)` בונה טבלה שממפה כל תו במחרוזת הראשונה לתו שבאותו מקום במחרוזת השנייה. `str.translate` מחיל את הטבלה במעבר אחד שממומש ב-C, ולכן הוא מהיר בהרבה מלולאת Python.

```python
table = str.maketrans("ACGT", "TGCA")
table                              # {65: 84, 67: 71, 71: 67, 84: 65}  (Unicode code points)
"ATGCCGTT".translate(table)        # 'TACGGCAA'
"ATGCNNGT".translate(table)        # 'TACGNNCA'  -> characters not in the table pass through

# Lowercase too, still in a single pass:
table = str.maketrans("ACGTacgt", "TGCAtgca")
```

⚠️ למה לא שרשרת של `replace`?

הניסיון האינטואיטיבי, `dna.replace("A", "T").replace("T", "A")`, שובר את הרצף: ה-`replace` השני הופך בחזרה ל-A גם את ה-T שנוצרו בשלב הראשון. למשל, `"AAAACCCGGT"` הופך ל-`'AAAACCCGGA'`. המיפוי חייב לקרות בבת אחת לכל תו, וזה בדיוק מה ש-`translate` ומילון עושים.

גישה	זמן	הערה
`translate` ואז `[::-1]`	O(n)	הכי מהירה: שני מעברים, שניהם ב-C
מילון וגנרטור עם `join`	O(n)	ברורה וגמישה, אבל איטית יותר כי כל תו עובר בלולאת Python
לולאה עם `+=` למחרוזת	O(n²)	במקרה הגרוע: כל חיבור עלול להעתיק את כל המחרוזת

💡 תכונה שימושית לבדיקה

השלמה הפוכה היא אינוולוציה: הפעלה כפולה מחזירה את הרצף המקורי. `reverse_complement(reverse_complement(s)) == s` היא בדיקה מצוינת לקוד שלכם, ובדיוק את סוג התכונות האלה בודקות ספריות של בדיקות מבוססות-תכונות (property-based testing) כמו Hypothesis.

זמן O(n)
כל בסיס ממופה פעם אחת, וההיפוך הוא מעבר נוסף

זיכרון O(n)
מחרוזת חדשה באותו אורך

5 דוגמאות קוד

לפני שתכתבו פתרון משלכם, הנה דוגמאות מוכנות שמשתמשות באותם כלים ורעיונות. מתחת לכל דוגמה מופיע **הפלט** של הרצת הקוד (כפי שהתקבל מ-Python 3), ואחריו ההסבר המפורט. מומלץ להקליד את הקוד בעצמכם, להריץ אותו, ואז לשנות אותו ולבדוק איך הפלט משתנה. בגרסת האתר כל דוגמה ניתנת להרצה בלחיצת כפתור.

דוגמה 1 איתור פלינדרומים ביולוגיים

אתרי הגבלה רבים הם רצפים ששווים להשלמה ההפוכה של עצמם. הדוגמה סורקת רצף ומדפיסה כל קטע באורך 4 עד 12 שהוא פלינדרום כזה, בדומה לבעיית REVP ב-Rosalind.

PYTHON

```python
def reverse_complement(dna):
    return dna.translate(str.maketrans("ACGT", "TGCA"))[::-1]

dna = "TCAATGCATGCGGGTCTATATGCAT"

for length in range(4, 13):
    for i in range(len(dna) - length + 1):
        site = dna[i:i + length]
        if site == reverse_complement(site):
            print(f"position {i + 1:>2}  length {length:>2}  {site}")
```

```
$ python example.py
position  5  length  4  TGCA
position  7  length  4  CATG
position 17  length  4  TATA
position 18  length  4  ATAT
position 21  length  4  TGCA
position  4  length  6  ATGCAT
position  6  length  6  GCATGC
position 20  length  6  ATGCAT
```

איך זה עובד, שלב אחר שלב

1. `reverse_complement` כתובה כאן בשורה אחת: `translate` משלים את כל הבסיסים, ו-`[::-1]` הופך את הסדר.
2. הלולאה החיצונית עוברת על אורכי הקטעים, 4 עד 12 (`range(4, 13)` לא כולל את 13). הלולאה הפנימית היא חלון נע: כל עמדת התחלה אפשרית לקטע באורך הזה.
3. `site == reverse_complement(site)` בודק את תכונת הפלינדרום. שימו לב שפלינדרום ביולוגי שונה מפלינדרום במילים: `GCATGC` אינו נקרא אותו דבר מהסוף, אבל ההשלמה ההפוכה שלו שווה לו.
4. ההדפסה משתמשת ב-`i + 1`, כי Rosalind סופרת עמדות מ-1 (זוכרים את מודול 4?).
5. פלינדרום ביולוגי תמיד באורך זוגי: באורך אי-זוגי, הבסיס האמצעי היה צריך להיות המשלים של עצמו, וזה בלתי אפשרי. לכן כל האורכים בפלט זוגיים.

דוגמה 2 חיפוש מוטיב בשני הגדילים ותרגום קואורדינטות

גורם שעתוק יכול להיקשר לאותו מוטיב על כל אחד משני הגדילים. הדוגמה מחפשת את תיבת ה-TATA גם בגדיל העליון וגם בהשלמה ההפוכה, ומתרגמת כל התאמה בגדיל ההפוך בחזרה לקואורדינטות של הגדיל העליון.

PYTHON

```python
def reverse_complement(dna):
    return dna.translate(str.maketrans("ACGT", "TGCA"))[::-1]

def find_all(seq, motif):
    hits, i = [], seq.find(motif)
    while i != -1:
        hits.append(i)
        i = seq.find(motif, i + 1)
    return hits

genome = "GGTATAAACCGTTTATACCA"
motif = "TATAAA"
n, m = len(genome), len(motif)

print("+ strand:", find_all(genome, motif))
for j in find_all(reverse_complement(genome), motif):
    start = n - (j + m)          # map back to plus-strand coordinates
    print(f"- strand: index {j} on the reverse complement "
          f"= plus-strand {start}..{start + m - 1} ({genome[start:start + m]})")
```

```
$ python example.py
+ strand: [2]
- strand: index 3 on the reverse complement = plus-strand 11..16 (TTTATA)
```

איך זה עובד, שלב אחר שלב

1. `find_all` היא לולאת ה-`find` ממודול 4, שמוצאת גם מופעים חופפים.
2. החיפוש בגדיל העליון מוצא את `TATAAA` באינדקס 2.
3. החיפוש בהשלמה ההפוכה מוצא התאמה נוספת: ברצף המקורי היא נראית כמו `TTTATA`, שהיא ההשלמה ההפוכה של המוטיב. על הגדיל השני זהו `TATAAA` רגיל.
4. תרגום הקואורדינטות הוא החלק העדין: אינדקס `j` בהשלמה ההפוכה סופר מהקצה השני של הגנום. הקטע מסתיים בעמדה $n - 1 - j$ של הגדיל העליון, ולכן מתחיל ב-$n - (j + m)$.
5. באגים בתרגום קואורדינטות בין גדילים נפוצים מאוד בצנרות ניתוח. בדיקה פשוטה: `genome[start:start + m]` צריך להיות שווה ל-`reverse_complement(motif)`.

דוגמה 3 k-mers קנוניים: ספירה בלי תלות בגדיל

אותה מולקולת DNA יכולה להיקרא מכל אחד משני הגדילים. כדי שהספירה לא תהיה תלויה בגדיל שנקרא, כלים כמו

Jellyfish ו-Mash סופרים כל k-mer יחד עם ההשלמה ההפוכה שלו, תחת נציג אחד: ה"קנוני". זה גם גשר למודול הבא.

```python
from collections import Counter

def reverse_complement(dna):
    return dna.translate(str.maketrans("ACGT", "TGCA"))[::-1]

def canonical(kmer):
    return min(kmer, reverse_complement(kmer))

read_plus = "ATGGCGTA"
read_minus = reverse_complement(read_plus)   # same molecule, other strand
k = 3

for name, read in (("plus ", read_plus), ("minus", read_minus)):
    kmers = [read[i:i + k] for i in range(len(read) - k + 1)]
    counts = Counter(canonical(x) for x in kmers)
    print(name, read, "->", sorted(counts.items()))
```

```
$ python example.py
plus  ATGGCGTA → [('ACG', 1), ('ATG', 1), ('CCA', 1), ('CGC', 1), ('GCC', 1), ('GTA', 1)]
minus TACGCCAT → [('ACG', 1), ('ATG', 1), ('CCA', 1), ('CGC', 1), ('GCC', 1), ('GTA', 1)]
```

איך זה עובד, שלב אחר שלב

1. `canonical` מחזירה את הקטן מבין ה-k-mer וההשלמה ההפוכה שלו, לפי סדר אלפביתי (`min` על מחרוזות משווה מילונית).
2. `read_minus` היא אותה מולקולה כפי שהייתה נקראת מהגדיל השני. ה-k-mers הגולמיים שלה שונים לגמרי מאלה של `read_plus`.
3. רשימת ההבנה מפרקת כל קריאה ל-k-mers חופפים, ו-`Counter` סופר אותם אחרי ההמרה לצורה הקנונית.
4. `sorted(counts.items())` ממיין את הזוגות כדי שיהיה קל להשוות. שתי השורות בפלט זהות: הספירה הקנונית לא תלויה בגדיל.
5. שימו לב: ל-k אי-זוגי אין k-mer ששווה להשלמה ההפוכה של עצמו, ולכן כל זוג מקבל נציג אחד ברור. זו אחת הסיבות שכלים בוחרים k אי-זוגי.

6 המשימה שלכם

ממשו את `reverse_complement(dna)`. הבדיקות הנסתרות כוללות מחרוזת ריקה, בסיסים בודדים, פלינדרומים ורצף ארוך מאוד.

מודול 8 · אתגר קוד בסגנון LEETCODE

✎ התרגיל: `reverse_complement(dna)`

בינוני-קשה | אנטי-מקביליות | היפוך מחרוזות | טבלאות תרגום | כ-40 דקות

ההשלמה ההפוכה של מחרוזת DNA `s` מתקבלת בשני צעדים: הופכים את סדר הסמלים ב- `s`, ואז מחליפים כל סמל במשלים שלו: `A` ↔ `T` ו- `C` ↔ `G`. (אפשר גם בסדר ההפוך: קודם להשלים ואז להפוך.)

ממשו את `reverse_complement(dna)` כך שתחזיר את ההשלמה ההפוכה כמחרוזת.

דוגמאות

דוגמה 1

קלט: `dna = "AAAACCCGGT"`

פלט: `"ACCGGGTTTT"`

הסבר: היפוך נותן TGGCCCAAAA, וההשלמה שלו היא ACCGGGTTTT.

דוגמה 2

קלט: `dna = "ATGCCGTT"`

פלט: `"AACGGCAT"`

דוגמה 3

קלט: `dna = "GAATTC"`

פלט: `"GAATTC"`

הסבר: אתר EcoRI הוא פלינדרום: הוא שווה להשלמה ההפוכה של עצמו.

אילוצים

- $0 \le \text{len(dna)} \le 10^5$
- `dna` מכילה רק את האותיות הגדולות `A`, `C`, `G`, `T`
- יש להחזיר `str`

רמזים

💡 `dna[::-1]` הופך מחרוזת.

💡 מילון `{"A": "T", ...}` או `str.maketrans` ממפים כל בסיס למשלים שלו.

💡 אל תשתמשו בשרשרת של `replace`: ה- `replace` השני ישנה גם את התווים שנוצרו בראשון.

בדיקות גלויות (הרצה)

אלה הבדיקות שכפתור **הרצה** באתר מריץ. הפתרון חייב לעבור את כולן.

קריאה	תוצאה צפויה
`reverse_complement("AAAACCCGGT")`	`"ACCGGGTTTT"`
`reverse_complement("ATGCCGTT")`	`"AACGGCAT"`
`reverse_complement("GAATTC")`	`"GAATTC"`

בדיקות נסתרות: מקרי קצה (הגשה)

כפתור **הגשה** מריץ גם את הבדיקות האלה. הן נועדו לתפוס קלטים ריקים, מקרי גבול, יעילות על קלט גדול וטיפול בשגיאות. כתבו את הפתרון כך שיעמוד בהן **לפני** שאתם מציצים בטבלה.

קריאה	תוצאה צפויה	מה נבדק
`reverse_complement("")`	`""`	קלט ריק
`reverse_complement("A")`	`"T"`	תו בודד
`reverse_complement("C")`	`"G"`	תו בודד
`reverse_complement("TTTT")`	`"AAAA"`	
`reverse_complement("ACGT")`	`"ACGT"`	פלינדרום
`reverse_complement("GGGCCCATAC")`	`"GTATGGGCCC"`	
`reverse_complement("ATGGCCTAGC" * 1000)`	`"GCTAGGCCAT" * 1000`	קלט גדול (10,000 תווים): יעילות

קוד התחלתי

כך נראה העורך כשפותחים את המודול. ממשו את גוף הפונקציה במקום `pass`.

PYTHON

```python
def reverse_complement(dna):
    """
    Return the reverse complement of a DNA string (A<->T, C<->G, then reverse).

    >>> reverse_complement("AAAACCCGGT")
    'ACCGGGTTTT'
    """
    # Write your code here
    pass
```

🔑 פתרון עיון מלא: `reverse_complement`

הפתרון שעובר את כל הבדיקות, הגלויות והנסתרות. מספרי השורות בהסבר מתייחסים לקוד שלמעלה.

```python
COMPLEMENT = str.maketrans("ACGT", "TGCA")


def reverse_complement(dna):
    # Complement every base in one C-speed pass, then read it backwards.
    return dna.translate(COMPLEMENT)[::-1]
```

הסבר שורה אחר שורה

שורה 1 — **טבלת התרגום נבנית פעם אחת**, ברמת המודול, ולא בכל קריאה. `str.maketrans("ACGT", "TGCA")` ממפה כל תו במחרוזת הראשונה לתו שבאותו מקום בשנייה: A→T, C→G, G→C, T→A. המיפוי קורה בבת אחת לכל תו, ולכן אין את הבאג של שרשרת `replace`.

שורה 5 — ההערה מסכמת את שני הצעדים: השלמה במעבר אחד במהירות C, ואז קריאה לאחור.

שורה 6 — `dna.translate(COMPLEMENT)` מחזיר את הגדיל המשלים (כתוב מ-3′ ל-5′). החיתוך `[::-1]`, עם צעד −1, הופך את סדר התווים ומחזיר מחרוזת חדשה: ההשלמה ההפוכה, כתובה מ-5′ ל-3′ לפי המוסכמה. סדר שני הצעדים לא משנה את התוצאה.

⌘ למה זה הפתרון האופטימלי

שני מעברים על המחרוזת, שניהם ממומשים ב-C: O(n) בזמן ו-O(n) בזיכרון (המחרוזת החדשה). הגרסה עם מילון וגנרטור נכונה אך איטית פי כמה, כי כל תו עובר בלולאת Python; לולאה עם `+=` עלולה להיות $O(n^2)$.

💡 איך הפתרון עומד במקרי הקצה הנסתרים

`""` → `""`; `"A"` → `"T"`; `"AAAA"` → `"TTTT"`; פלינדרומים כמו `"ACGT"` ו-`"GAATTC"` מוחזרים כפי שהם; רצף של 10,000 בסיסים מטופל במעבר אחד. תכונה שימושית לבדיקה עצמית: הפעלה כפולה מחזירה את הקלט.

מודול 9 · ביולוגיה × מדעי המחשב

תדירויות k-mer

קשה | כ-45 דקות | הרכבת גנומים | חלון נע | ספירה במילון | `count_kmers()`

מעבדה חזותית: חלון k נע עם טבלת תדירויות ממוינת

k-mer הוא פשוט "מילה" באורך k שחותכים מתוך הרצף: ה-n-gram של DNA. הרעיון הפשוט הזה מניע חלק גדול מהביואינפורמטיקה המודרנית: הרכבת גנומים ממקטעים קצרים, זיהוי מינים בדגימות מיקרוביום, הערכת גודל של גנום ובקרת איכות של נתוני ריצוף. במודול הזה תכתבו את פונקציית הספירה שעומדת בבסיס כל אלה.

מה תלמדו

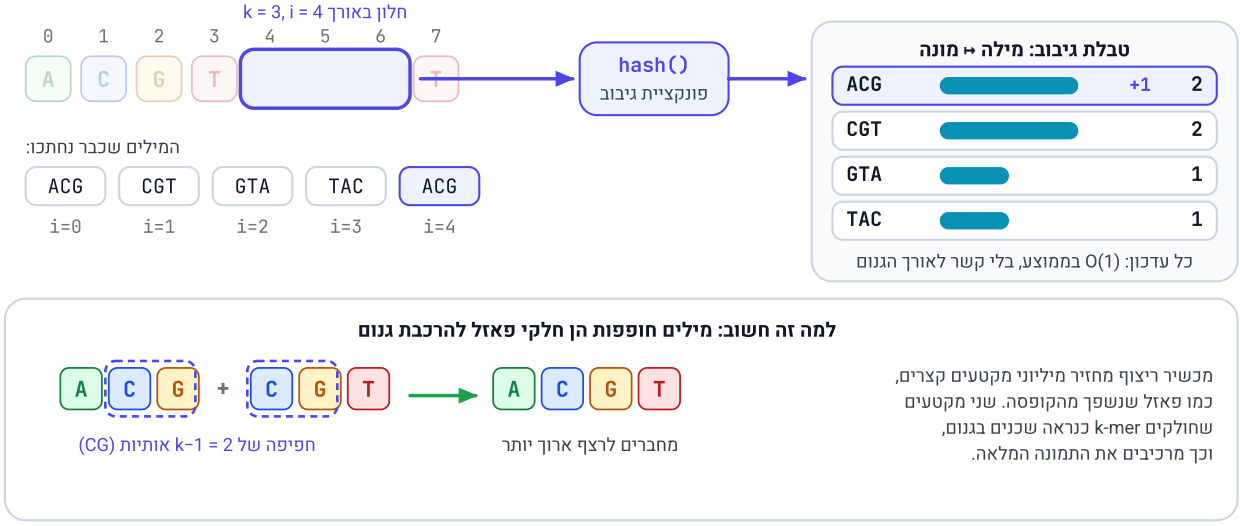
- הביולוגיה בשפה של מתכנתים: k-mers כ-n-grams, ספירה בטבלת גיבוב, הרכבה כפאזל
- מהו k-mer, וכמה k-mers יש ברצף באורך n
- למה k-mers הם הבסיס להרכבת גנומים (גרפי de Bruijn)
- ספירה במילון: `dict` , `get` , `defaultdict` ו-`Counter`
- איך בוחרים k: הפשרה בין ייחודיות לעמידות בפני שגיאות

1 הביולוגיה בשפה של מתכנתים

k-mer הוא פשוט "מילה" באורך k שחותכים מתוך הרצף, ומאחר שאין רווחים ב-DNA, חותכים את כל המילים האפשריות: מכל אינדקס, k אותיות קדימה. זה בדיוק ה-n-gram שמנועי חיפוש ומערכות השלמה אוטומטית מחשבים על טקסט, או ה-shingles שתוכנות לזיהוי העתקות משוות בין מסמכים. אותו רעיון של חלון נע ממודול 4, רק שהפעם לא מחפשים מילה אחת אלא סופרים את כולן.

הספירה עצמה היא שימוש קלאסי ב**טבלת גיבוב** (המילון של Python): כל חלון נשלח לפונקציית גיבוב, שמפנה אותו לתא במילון, ושם המונה עולה באחד. בלי קשר לאורך הגנום, כל עדכון עולה זמן קבוע בממוצע. ולמה זה חשוב לביולוגים? כי מכשירי ריצוף לא קוראים גנום שלם אלא מיליוני מקטעים קצרים, ו-k-mers הם חלקי הפאזל: שני מקטעים שחולקים k-mer, סביר שמגיעים מאותו מקום בגנום, וכך מרכיבים את התמונה המלאה. אותה ספירה משמשת גם לזיהוי מינים בדגימה, להערכת גודל הגנום ולתפיסת שגיאות ריצוף.

איור k-mers כ-n-grams: חותכים מילים, סופרים בטבלת גיבוב, מרכיבים פאזל



למעלה: חלון באורך 3 זז תא אחד בכל צעד וחותך "מילה"; כל מילה עוברת דרך פונקציית גיבוב אל התא שלה בטבלה, ושם המונה עולה באחד (ACG כבר נראה פעמיים). למטה: שתי מילים שחופפות ב-k−1 אותיות אפשר לחבר, וזה הרעיון שמאחורי הרכבת גנומים ממקטעים קצרים.

מילון מונחים: ביולוגיה ↔ מדעי המחשב

בביולוגיה	במדעי המחשב	למה הדימוי עובד
k-mer	**n-gram / shingle**	תת-מחרוזת באורך קבוע; כל האינדקסים, כולל חפיפות.
הרכב k-mers	**היסטוגרמה של מילים**	כמה פעמים כל מילה הופיעה; טביעת אצבע של הרצף.
ספירה	**טבלת גיבוב (`dict`, `Counter`)**	מפתח = המילה, ערך = מונה; עדכון בזמן קבוע בממוצע.
קריאת ריצוף	**חלק פאזל**	מקטע קצר ואקראי של הגנום; לבד הוא לא אומר הרבה.
הרכבת גנום	**חיבור פאזל לפי קצוות חופפים**	מילים שחופפות ב-k−1 אותיות מצביעות על שכנות בגנום (גרף de Bruijn).
בחירת k	**גודל ה-n-gram**	קצר = דו-משמעי, ארוך = שביר לשגיאות; בוחרים באמצע, ובדרך כלל אי-זוגי.

⚡ בשורה אחת

k-mers הם ה-n-grams של DNA: חותכים כל מילה באורך k, סופרים אותן במילון, ומשתמשים בחפיפות ביניהן כדי להרכיב גנום ממקטעים קצרים.

2 מהו k-mer?

ההגדרה המדויקת, וכמה מילים כאלה יש בכלל.

k-mer הוא כל תת-מחרוזת רציפה באורך k. ברצף באורך n יש בדיוק **n − k + 1** חלונות, כמו בחלון הנע של מודול 4. לעומת זאת, מספר ה-k-mers האפשריים באלפבית של 4 אותיות הוא 4^k: 16 עבור k = 2, 65,536 עבור k = 8, ויותר מ-4 טריליון עבור k = 21.



שישה חלונות באורך 3 ברצף באורך 8: 8 − 3 + 1 = 6. `ACG` ו-`CGT` מופיעים פעמיים כל אחד.

הרכב ה-k-mers (k-mer composition) של רצף הוא רשימת כל ה-k-mers שלו, עם מספר המופעים של כל אחד. זה בדיוק מה שתחזירו בתרגיל: מילון שבו המפתח הוא ה-k-mer, והערך הוא הספירה.

3 למה k-mers חשובים?

- **הרכבת גנומים**: מכשירי ריצוף קוראים מיליוני מקטעים קצרים. אסמבלרים כמו SPAdes ו-Velvet מפרקים את הקריאות ל-k-mers ובונים מהם גרף de Bruijn: כל k-mer הוא קשת בין התחילית לסיפא שלו, שאורכן k − 1. מסלול שעובר בכל הקשתות משחזר את הגנום.
- **זיהוי מינים ומטאגנומיקה**: כלים כמו Kraken2 מחזיקים מאגר של k-mers (למשל באורך 35) שבו כל k-mer ממופה למין שלו. כך אפשר לסווג מיליוני קריאות בדקות, בלי יישור.
- **הערכת גודל הגנום**: היסטוגרמה של ספירות ה-k-mers ("k-mer spectrum") מראה פסגה סביב עומק הריצוף. כלים כמו GenomeScope מסיקים ממנה את גודל הגנום, את רמת ההטרוזיגוטיות ואת כמות הרצפים החוזרים.
- **תיקון שגיאות**: k-mer שמופיע פעם אחת בלבד בנתונים עמוקים הוא כנראה שגיאת ריצוף. על העיקרון הזה עובדים כלים לתיקון שגיאות.
- **חתימת הרכב**: לכל גנום יש "טביעת אצבע" של תדירויות k-mers קצרים (למשל 4-mers), שמשמשת לקיבוץ (binning) של רצפים לפי המין שממנו הגיעו.

🧬 בחירת k היא פשרה

אם k קטן מדי, אותו k-mer מופיע במקומות רבים בגנום, ואי אפשר לדעת מאיפה הוא הגיע. אם k גדול מדי, כל שגיאת ריצוף אחת הורסת k k-mers, ויש פחות חפיפה בין קריאות. בגנום האדם ($3{\cdot}10^9$ בסיסים) רוב ה-k-mers באורך 31 ומעלה ייחודיים, ולכן ערכים כמו 21, 31 ו-51 נפוצים. מקובל לבחור k אי-זוגי, כדי ש-k-mer לעולם לא יהיה שווה להשלמה ההפוכה של עצמו (מודול 8).

---

4 מעבדה חזותית: ספירת k-mers

בחרו רצף וערך של k ולחצו **סריקה**. החלון זז עמדה אחת בכל צעד, והספירה של ה-k-mer שבתוכו עולה ב-1. הטבלה ממוינת לפי שכיחות. נסו להגדיל את k ולראות איך כמעט כל k-mer הופך לייחודי.

מעבדה חזותית · ספירת k-mers

חלון באורך k זז לאורך הרצף, ובכל צעד הספירה של ה-k-mer שבתוכו עולה ב-1.

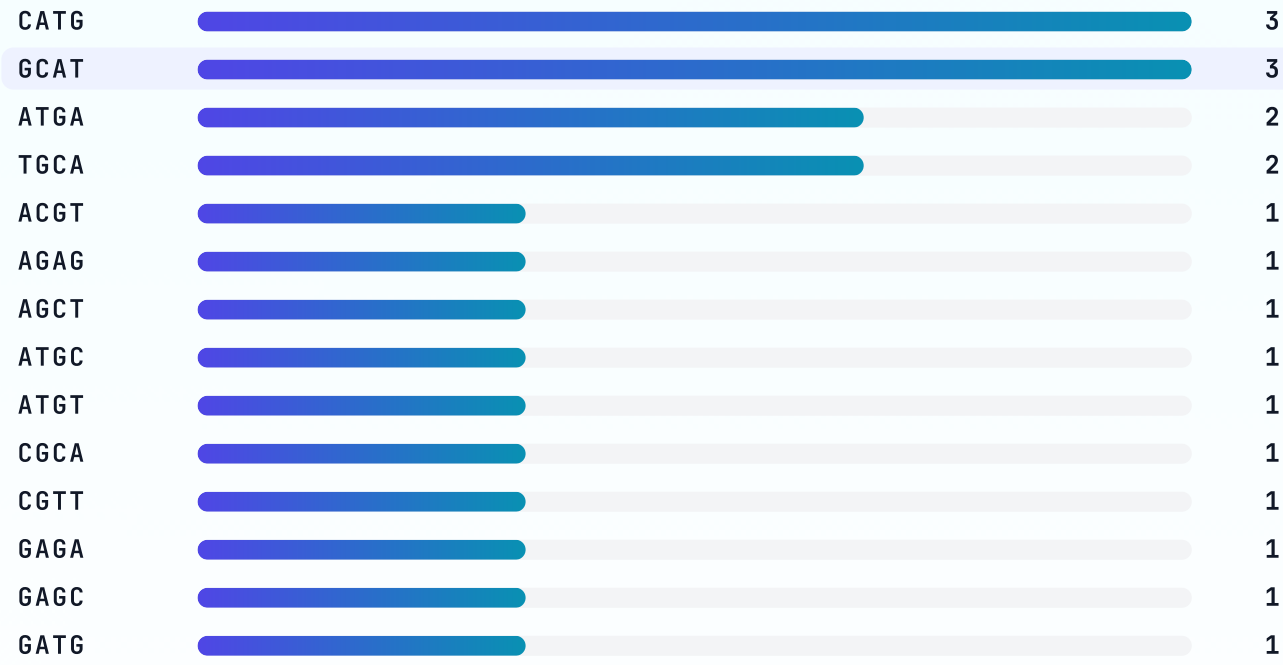
הרצף (n = 30) עם חלון באורך k = 4 בעמדה i = 5. בכל צעד החלון זז עמדה אחת ימינה, והספירה של ה-k-mer שבתוכו עולה ב-1.



חלון 5: `dna[5:9]` = `GCAT` ← מפתח חדש במילון, ספירה 1.

חלונות N − K + 1	K-MERS שונים	אפשריים 4^K	השכיח ביותר
27	21	256	CATG ×3

טבלת התדירויות בסוף הסריקה, ממוינת מהשכיח לנדיר (בתיקו, לפי האלפבית). השורה המודגשת היא ה-k-mer של החלון שלמעלה.



שמופיעים פעם אחת k-mers ועוד 7

הסריקה הסתיימה: 21 k-mers שונים מתוך 27 חלונות. התוצאה היא המילון `{"CATG": 3, "GCAT": 3, "ATGA": 2, …}`

בגרסה המקוונת: בוחרים רצף וערך k (1–8) ולוחצים **סריקה**, **צעד** או **ספירה מלאה**. הטבלה הממוינת מתעדכנת בכל צעד. הגדלת k מראה איך כמעט כל k-mer הופך לייחודי.

5 ספירה במילון

ספירת תדירויות היא שימוש קלאסי במילון: המפתח הוא הפריט, והערך הוא מספר הפעמים שראינו אותו. יש כמה דרכים לכתוב את זה, מהמפורשת ועד הקצרה ביותר:

```python
dna, k = "ACGTACGT", 3
windows = [dna[i:i + k] for i in range(len(dna) - k + 1)]
# ['ACG', 'CGT', 'GTA', 'TAC', 'ACG', 'CGT']

# 1) Explicit check
counts = {}
for kmer in windows:
    if kmer in counts:
        counts[kmer] += 1
    else:
        counts[kmer] = 1

# 2) dict.get with a default
counts = {}
for kmer in windows:
    counts[kmer] = counts.get(kmer, 0) + 1

# 3) defaultdict: a missing key starts at int() == 0
from collections import defaultdict
counts = defaultdict(int)
for kmer in windows:
    counts[kmer] += 1

# 4) Counter: does it all, plus most_common()
from collections import Counter
counts = Counter(windows)
print(counts)        # Counter({'ACG': 2, 'CGT': 2, 'GTA': 1, 'TAC': 1})
```

⌘ כמה זה עולה?

כל עדכון של `counts[kmer]` הוא חיפוש בטבלת גיבוב, O(1) בממוצע (מודול 5). ספירת כל ה-k-mers לוקחת O(n·k): יש $n - k + 1$ חלונות, וחיתוך וגיבוב של כל k-mer עולים O(k). המילון שומר רק k-mers שבאמת הופיעו, ולא את כל 4^k האפשרויות, ולכן הוא חסכוני גם כש-k גדול.

⚠ מקרי קצה

- **k גדול מאורך הרצף**: אין אף חלון, ולכן מחזירים מילון ריק. `range` עם גבול שלילי פשוט לא רץ, אבל כדאי לחשוב על זה במפורש.
- `k ≤ 0`: אין k-mers באורך 0 או באורך שלילי. שימו לב שהחיתוך `dna[i:i + 0]` מחזיר מחרוזת ריקה, ולכן בלי בדיקה מפורשת תקבלו מפתח `""` שגוי.
- `Counter` הוא תת-מחלקה של `dict`, ולכן מותר להחזיר אותו בתרגיל.

זמן `O(n·k)`
n − k + 1 חלונות, וחיתוך וגיבוב של O(k) לכל אחד

זיכרון `O(n·k)`
לכל היותר n − k + 1 מפתחות שונים, כל אחד באורך k

---

6 דוגמאות קוד

לפני שתכתבו פתרון משלכם, הנה דוגמאות מוכנות שמשתמשות באותם כלים ורעיונות. מתחת לכל דוגמה מופיע **הפלט המלא** של הרצת הקוד (כפי שהתקבל מ-Python 3), ואחריו ההסבר המפורט. מומלץ להקליד את הקוד בעצמכם, להריץ אותו, ואז לשנות אותו ולבדוק איך הפלט משתנה. בגרסת האתר כל דוגמה ניתנת להרצה בלחיצת כפתור.

דוגמה 1 גרף de Bruijn: הצעד הראשון בהרכבת גנום

אסמבלרים לא משווים קריאות זו לזו ישירות. הם מפרקים אותן ל-k-mers, ומחברים כל תחילית לסיפא שלה. הדוגמה בונה

את רשימת השכנויות של הגרף מארבע קריאות קצרות וחופפות.

```python
from collections import defaultdict

reads = ["ATGGCGT", "GGCGTGC", "CGTGCAA", "TGCAATG"]
k = 4

graph = defaultdict(set)
for read in reads:
    for i in range(len(read) - k + 1):
        kmer = read[i:i + k]
        graph[kmer[:-1]].add(kmer[1:])     # edge: prefix -> suffix

for node in sorted(graph):
    print(node, "->", ", ".join(sorted(graph[node])))
```

```
$ python example.py
AAT → ATG
ATG → TGG
CAA → AAT
CGT → GTG
GCA → CAA
GCG → CGT
GGC → GCG
GTG → TGC
TGC → GCA
TGG → GGC
```

איך זה עובד, שלב אחר שלב

1. ארבע הקריאות חופפות זו לזו, כמו מקטעים אמיתיים מאותו אזור בגנום. אף אחת מהן לבדה לא מכילה את כל הרצף.
2. `defaultdict(set)` יוצר קבוצה ריקה בפעם הראשונה שניגשים למפתח חדש. בחרנו ב-`set` ולא ב-`list`, כי אותו k-mer מופיע בכמה קריאות וחפיפות כפולות לא מוסיפות מידע.
3. כל k-mer הופך לקשת: מהתחילית שלו, `kmer[:-1]`, אל הסיפא שלו, `kmer[1:]`. שתיהן באורך $k - 1 = 3$.
4. ההדפסה ממוינת כדי שהפלט יהיה קבוע. עקבו אחרי החצים:
   `ATG → TGG → GGC → GCG → CGT → GTG → TGC → GCA → CAA → AAT → ATG`. זה מעגל, כלומר הקריאות כנראה הגיעו מ-DNA מעגלי, כמו פלסמיד.
5. באסמבלר אמיתי מחפשים מסלול אוילר, מסלול שעובר בכל קשת פעם אחת. זו בעיה שפותרים ביעילות, וזו הסיבה שגרפי de Bruijn החליפו את שיטות ההרכבה הישנות שהשוו כל זוג קריאות.

דוגמה 2 ספקטרום k-mer: כמה פעמים מופיע כל k-mer?

נייצר גנום אקראי, נדגום ממנו קריאות בעומק של כ-30x, ונספור k-mers. ההיסטוגרמה של הספירות, ה"ספקטרום", חושפת

את עומק הריצוף, ומכאן אפשר להעריך את גודל הגנום בלי להרכיב אותו.

```python
import random
from collections import Counter

random.seed(7)
genome = "".join(random.choice("ACGT") for _ in range(400))
reads = []
for _ in range(120):                        # 120 reads x 100 bp over 400 bp = 30x
    start = random.randrange(len(genome) - 99)
    reads.append(genome[start:start + 100])

k = 15
kmer_counts = Counter(r[i:i + k] for r in reads for i in range(len(r) - k + 1))
spectrum = Counter(10 * (c // 10) for c in kmer_counts.values())

for low in sorted(spectrum):
    print(f"{low:>3}-{low + 9:<3}x  {spectrum[low]:>3} k-mers  {'#' * (spectrum[low] //
5)}")

total = sum(c for c in kmer_counts.values())
peak = max(kmer_counts.values(), key=list(kmer_counts.values()).count)
print("estimated genome size:", round(total / peak), "bp (true: 400)")
```

```
$ python example.py
  0-9  x   47 k-mers  #########
 10-19 x   32 k-mers  ######
 20-29 x  106 k-mers  ####################
 30-39 x  199 k-mers  ######################################
 40-49 x    2 k-mers
estimated genome size: 356 bp (true: 400)
```

איך זה עובד, שלב אחר שלב

1. `random.seed(7)` מבטיח שהגנום והקריאות יהיו זהים בכל הרצה. `random.randrange(len(genome) - 99)` בוחר עמדת התחלה כך שכל קריאה באורך 100 נכנסת במלואה לגנום.
2. ה-`Counter` מקבל גנרטור עם שתי לולאות: על כל קריאה, ועל כל חלון בתוכה. כך נספרים כל ה-15-mers מכל הקריאות, בלי לבנות רשימת ביניים ענקית.
3. `kmer_counts.values()` הן הספירות. `10 * (c // 10)` מקבץ אותן לתאים ברוחב 10 (0–9, 10–19, וכן הלאה), ו-`Counter` שני סופר כמה k-mers נופלים בכל תא. זה הספקטרום.
4. רוב ה-k-mers מתקבצים בתא 30–39. עומק הריצוף **הממוצע** הוא 30x, אבל במרכז הגנום הוא גבוה יותר, כי קריאות לא יכולות להתחיל קרוב לקצה הימני. מצד שני, k-mer באורך 15 נספר רק בקריאות שמכסות אותו במלואו, ולכן עומק ה-k-mer נמוך מעט מעומק הבסיסים. k-mers בקצוות הגנום מכוסים פחות, ולכן נופלים לתאים הנמוכים.
5. הערכת הגודל: סך כל מופעי ה-k-mers מחולק בספירה השכיחה ביותר (פסגת הספקטרום) נותן בקירוב את מספר ה-k-mers השונים בגנום, כלומר את אורכו. כאן מתקבל 356 במקום 400: הגנום קצר מאוד, והקצוות המכוסים פחות והרעש האקראי מטים את ההערכה. בגנומים של מיליארדי בסיסים ההשפעה הזו זניחה, ו-GenomeScope גם מתקן אותה במודל סטטיסטי.

דוגמה 3 השוואת רצפים לפי הרכב k-mer (מדד Jaccard)

אפשר להשוות שני רצפים בלי ליישר אותם: מספיק להשוות את קבוצות ה-k-mers שלהם. זה הרעיון שמאחורי Mash ו-sourmash, שמשווים אלפי גנומים שלמים בשניות.

PYTHON

```python
def kmer_set(seq, k):
    return {seq[i:i + k] for i in range(len(seq) - k + 1)}

def jaccard(a, b, k=4):
    A, B = kmer_set(a, k), kmer_set(b, k)
    return len(A & B) / len(A | B)

seqs = {
    "seqA": "ATGCGTACGTTAGCCGATCGATCGGATCCA",
    "seqB": "ATGCGTACGTTAGCCGATCGTTCGGATCCA",   # one substitution vs seqA
    "seqC": "TTGACCATGGCATTAGCAACCGTTAGCGAT",
}
names = list(seqs)
for i, a in enumerate(names):
    for b in names[i + 1:]:
        print(f"{a} vs {b}: Jaccard = {jaccard(seqs[a], seqs[b]):.2f}")
```

```
$ python example.py
seqA vs seqB: Jaccard = 0.85
seqA vs seqC: Jaccard = 0.12
seqB vs seqC: Jaccard = 0.11
```

איך זה עובד, שלב אחר שלב

1. `kmer_set` משתמשת ב*הבנת קבוצה* (set comprehension), עם סוגריים מסולסלים: כל k-mer נשמר פעם אחת בלבד, בלי ספירה.
2. `A & B` הוא החיתוך (k-mers משותפים), ו-`A | B` הוא האיחוד. מדד Jaccard הוא היחס ביניהם: 1 לקבוצות זהות, 0 לקבוצות זרות.
3. הלולאה הכפולה עם `names[i + 1:]` עוברת על כל זוג פעם אחת בלבד, בלי להשוות רצף לעצמו ובלי זוגות כפולים.
4. seqA ו-seqB נבדלים בבסיס אחד בלבד, ובכל זאת המדד נמוך מ-1 בצורה בולטת: מוטציה אחת משנה עד k k-mers. ל-seqC כמעט אין k-mers משותפים עם השניים האחרים.
5. בגנומים שלמים, Mash לא מחזיק את כל הקבוצה אלא "סקיצה" קטנה (MinHash) של כמה אלפי k-mers, ומעריך ממנה את מדד Jaccard ואת המרחק האבולוציוני.

7 המשימה שלכם

ממשו את `count_kmers(dna, k)`, שמחזירה מילון עם כל ה-k-mers והספירות שלהם. הבדיקות הנסתרות כוללות `k` שווה ל-0 ושלילי, `k` גדול מהרצף, רצף ריק ורצף ארוך מאוד.

מודול 9 · אתגר קוד בסגנון LEETCODE

התרגיל: `count_kmers(dna, k)`

קשה | הרכבת גנומים | חלון נע | ספירה במילון | כ-45 דקות

בהינתן מחרוזת DNA `dna` ומספר שלם `k`, ספרו כמה פעמים מופיע כל k-mer: כל תת-מחרוזת רציפה באורך `k`.

ממשו את `count_kmers(dna, k)` כך שתחזיר **מילון** שבו כל מפתח הוא k-mer שמופיע ברצף, והערך הוא מספר המופעים שלו.

- החלונות חופפים: ב-`"AAAA"` יש שלושה מופעים של `"AA"`.
- k-mers שאינם מופיעים לא נכללים במילון.
- אם `k` גדול מאורך הרצף, או ש-`k ≤ 0`, החזירו מילון ריק `{}`.
- מותר להחזיר `dict` רגיל או `Counter`. סדר המפתחות לא משנה.

דוגמאות

דוגמה 1

קלט: `dna = "ACGTACGT", k = 3`

פלט: `{'ACG': 2, 'CGT': 2, 'GTA': 1, 'TAC': 1}`

הסבר: שישה חלונות: ACG, CGT, GTA, TAC, ACG, CGT.

דוגמה 2

קלט: `dna = "AAAA", k = 2`

פלט: `{'AA': 3}`

דוגמה 3

קלט: `dna = "ACG", k = 5`

פלט: `{}`

הסבר: אין אף חלון באורך 5.

אילוצים

- $0 \le \text{len(dna)} \le 10^5$
- `k` הוא מספר שלם, ויכול להיות גם 0 או שלילי
- יש להחזיר `dict` (או `Counter`) עם מפתחות `str` וערכים `int`

רמזים

💡 יש `len(dna) - k + 1` חלונות.

💡 `counts[kmer] = counts.get(kmer, 0) + 1` מעדכן ספירה גם כשהמפתח עדיין לא קיים.

💡 בדקו `k <= 0` לפני הלולאה, אחרת תקבלו מפתח `""`.

בדיקות גלויות (הרצה)

אלה הבדיקות שכפתור **הרצה** באתר מריץ. הפתרון חייב לעבור את כולן.

קריאה	תוצאה צפויה
`count_kmers("ACGTACGT", 3)`	`{'ACG': 2, 'CGT': 2, 'GTA': 1, 'TAC': 1}`
`count_kmers("AAAA", 2)`	`{'AA': 3}`
`count_kmers("ACG", 5)`	`{}`

בדיקות נסתרות: מקרי קצה (הגשה)

כפתור **הגשה** מריץ גם את הבדיקות האלה. הן נועדו לתפוס קלטים ריקים, מקרי גבול, יעילות על קלט גדול וטיפול בשגיאות. כתבו את הפתרון כך שיעמוד בהן **לפני** שאתם מציצים בטבלה.

קריאה	תוצאה צפויה	מה נבדק
`count_kmers("", 3)`	`{}`	קלט ריק; k גדול מאורך הרצף
`count_kmers("ACGT", 0)`	`{}`	k לא חיובי
`count_kmers("ACGT", -2)`	`{}`	k לא חיובי
`count_kmers("ACGT", 4)`	`{'ACGT': 1}`	חלון יחיד
`count_kmers("GATTACA", 1)`	`{'G': 1, 'A': 3, 'T': 2, 'C': 1}`	
`count_kmers("ATATAT", 2)`	`{'AT': 3, 'TA': 2}`	
`count_kmers("ACGTTGCATGTCGCATGATGCATGAGAGCT", 4)`	מילון עם 21 מפתחות, למשל `{'ACGT': 1, 'CGTT': 1, 'GTTG': 1, …}`	
`count_kmers("ACGT" * 2500, 4)`	`{'ACGT': 2500, 'CGTA': 2499, 'GTAC': 2499, 'TACG': 2499}`	קלט גדול (10,000 תווים): יעילות

קוד התחלתי

כך נראה העורך כשפותחים את המודול. ממשו את גוף הפונקציה במקום `pass`.

PYTHON

```python
def count_kmers(dna, k):
    """
    Count every overlapping k-mer in dna.
    Return a dict {kmer: count}; return {} if k <= 0 or k > len(dna).

    >>> count_kmers("AAAA", 2)
    {'AA': 3}
    """
    # Write your code here
    pass
```

🔑 פתרון עיון מלא: `count_kmers`

הפתרון שעובר את כל הבדיקות, הגלויות והנסתרות. מספרי השורות בהסבר מתייחסים לקוד שלמעלה.

```python
def count_kmers(dna, k):
    counts = {}
    if k <= 0:
        return counts
    for i in range(len(dna) - k + 1):
        kmer = dna[i:i + k]
        counts[kmer] = counts.get(kmer, 0) + 1
    return counts
```

הסבר שורה אחר שורה

שורה 2 — המילון `counts` הוא הצובר: המפתח הוא k-mer, והערך הוא מספר המופעים. מתחילים ריק.

שורות 3–4 — **שמירה מפני** `k ≤ 0` . אין k-mers באורך 0 או שלילי. בלי הבדיקה, `dna[i:i + 0]` היה מחזיר `""` ו-`k` שלילי היה נותן חיתוכים מוזרים, והמילון היה מתמלא במפתחות חסרי משמעות. מחזירים את המילון הריק.

שורה 5 — **החלון הנע.** `range(len(dna) - k + 1)` מייצר את n − k + 1 עמדות ההתחלה. אם `k` גדול מאורך הרצף, הגבול הוא אפס או שלילי, ה-`range` ריק, ומוחזר `{}` בלי בדיקה מיוחדת.

שורה 6 — חיתוך של חלון אחד: תת-מחרוזת באורך `k` שמתחילה באינדקס `i` . חלונות סמוכים חופפים ב-k − 1 תווים.

שורה 7 — **האידיום לספירה במילון.** `counts.get(kmer, 0)` מחזיר את הספירה הנוכחית, או 0 אם המפתח עדיין לא קיים, ומוסיפים 1. כל עדכון הוא חיפוש בטבלת גיבוב, O(1) בממוצע.

שורה 8 — מחזירים את המילון. `Counter(dna[i:i + k] for i in range(len(dna) - k + 1))` היה נותן את אותה תוצאה, ו-`Counter` הוא תת-מחלקה של `dict` , ולכן גם הוא מתקבל.

⌘ למה זה הפתרון האופטימלי

O(n·k) בזמן: n − k + 1 חלונות, ובכל אחד חיתוך וגיבוב של מחרוזת באורך k. בזיכרון נשמרים רק k-mers שבאמת הופיעו, לכל היותר n − k + 1 מפתחות, ולא כל 4^k האפשרויות.

💡 איך הפתרון עומד במקרי הקצה הנסתרים

`""` , 3 → `{}` ; `"ACGT"` , 0 או −2 → `{}` (השמירה); `"ACG"` , 5 → `{}` (אין חלון); `"ACGT"` , 4 → `{"ACGT": 1}` (חלון יחיד); `"GATTACA"` , 1 → הרכב הבסיסים; `"ATATAT"` , 2 → `{"AT": 3, "TA": 2}` (חפיפות); רצף של 10,000 בסיסים → 4 מפתחות בלבד.

מודול 10 · ביולוגיה × מדעי המחשב

מטריצות פרופיל ורצפי קונצנזוס

קשה | כ-50 דקות | יישור מרובה | רשימות של רשימות | zip על עמודות | `consensus_sequence()`

מעבדה חזותית: מטריצת פרופיל עמודה-אחר-עמודה עם סימון תיקו ושורת קונצנזוס

כשמסדרים את אותו גן מעשרות מינים זה מעל זה, כמו שורות בגיליון אלקטרוני, רואים מיד שיש עמודות שכמעט לא משתנות ועמודות שמשתנות הרבה. **מטריצת פרופיל** סופרת "קולות" לכל אות בכל עמודה, **ורצף הקונצנזוס** הוא המנצח בכל עמודה. אלה הכלים הבסיסיים לתיאור מוטיבים ואזורים שמורים.

מה תלמדו

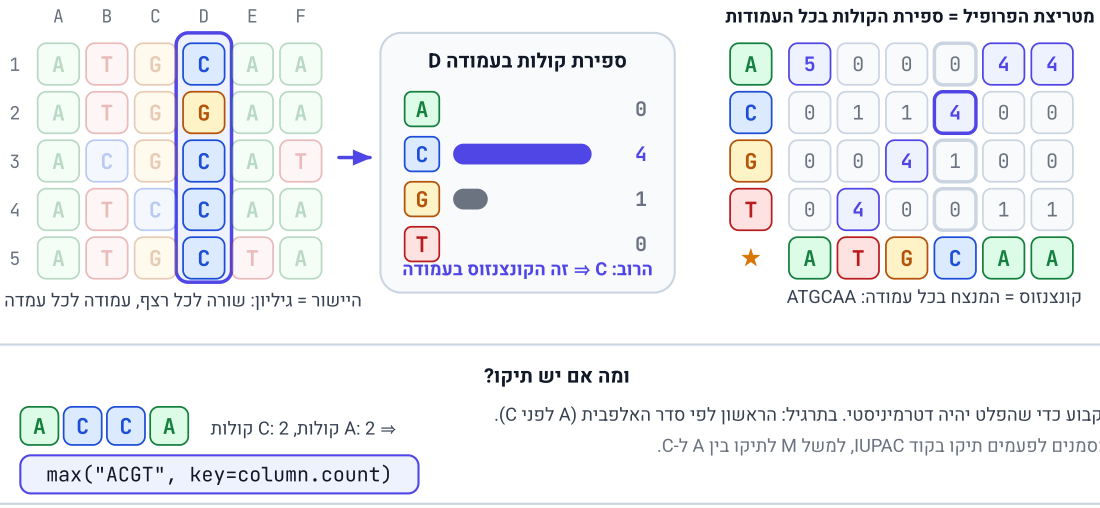
- הביולוגיה בשפה של מתכנתים: יישור מרובה כגיליון, קונצנזוס כהצבעת רוב
- יישור מרובה (MSA), ומה מלמדות עמודות שמורות
- מטריצת פרופיל, רצף קונצנזוס, PWM ולוגו רצף
- מערכים דו-ממדיים ב-Python: רשימות של רשימות
- מעבר על עמודות בעזרת `zip(*rows)`

1 הביולוגיה בשפה של מתכנתים

כשמסדרים כמה גרסאות של אותו גן זו מעל זו, מקבלים **גיליון אלקטרוני**: שורה לכל רצף, עמודה לכל עמדה. עכשיו מסתכלים על עמודה אחת ושואלים "מה כתוב בה ברוב השורות?". זו הצבעת רוב, בדיוק כמו ב-ensemble של מודלים או בבחירות: סופרים קולות לכל אות, והאות עם הכי הרבה קולות מנצחת. הטבלה שמחזיקה את ספירת הקולות בכל עמודה היא **מטריצת הפרופיל** (4 שורות, אחת לכל אות, ועמודה לכל עמדה), והמחרוזת של המנצחים היא **רצף הקונצנזוס**.

למה זה מעניין? כי האבולוציה מוחקת שינויים במקומות חשובים: אם בעמודה מסוימת כל 50 המינים שומרים על אותה אות, סביר שהאות הזאת חיונית, בערך כמו שורת קוד שאף אחד לא מעז לשנות כי הכול תלוי בה. עמודה שבה כל מין "הצביע" אחרת היא כנראה עמדה שלא משנה הרבה. כשיש תיקו בהצבעה צריך כלל קבוע לשבירתו, ובתרגיל הכלל הוא סדר האלפבית, כדי שהפלט יהיה דטרמיניסטי ואפשר יהיה לבדוק אותו.

איור יישור מרובה כגיליון אלקטרוני: הצבעת רוב בכל עמודה



ומה אם יש תיקו?

צריך כלל קבוע כדי שהפלט יהיה דטרמיניסטי. בתרגיל: הראשון לפי סדר האלפבית (A לפני C).
ביולוגים מסמנים לפעמים תיקו בקוד IUPAC, למשל M לתיקו בין A ל-C.

A C C A ⇒ A: 2 קולות, C: 2 קולות

```
max("ACGT", key=column.count)
```

משמאל: חמישה רצפים מיושרים, שורה לכל רצף. לוקחים עמודה אחת (D) וסופרים כמה פעמים כל אות מופיעה בה; האות עם הכי הרבה קולות היא הקונצנזוס של העמודה. עושים זאת לכל העמודות ומקבלים את מטריצת הפרופיל (4 × n) ואת רצף הקונצנזוס. למטה: הכלל לשבירת תיקו.

מילון מונחים: ביולוגיה ↔ מדעי המחשב

בביולוגיה	במדעי המחשב	למה הדימוי עובד
יישור מרובה (MSA)	**גיליון אלקטרוני / מערך דו-ממדי**	שורה לכל רצף, עמודה לכל עמדה; `rows[i][j]`.
עמודה ביישור	`zip(*rows)`	אותה עמדה בכל הרצפים; זה מה שסופרים.
מטריצת פרופיל	**טבלת ספירות 4 × n**	כמה קולות קיבלה כל אות בכל עמודה.
רצף הקונצנזוס	**הצבעת רוב בכל עמודה**	האות עם הכי הרבה קולות, עמודה אחר עמודה.
עמודה שמורה	**שורת קוד שאיש לא משנה**	כל הרצפים מסכימים, ולכן כנראה חיונית לתפקוד.
שבירת תיקו	**כלל דטרמיניסטי (סדר אלפביתי)**	בלי כלל קבוע, שתי הרצות יתנו תשובות שונות ואי אפשר לבדוק.

⚡ בשורה אחת

יישור מרובה הוא גיליון, מטריצת הפרופיל סופרת קולות בכל עמודה, והקונצנזוס הוא האות שזכתה ברוב, עם כלל אלפביתי קבוע לשבירת תיקו.

2 יישור מרובה ואזורים שמורים

מאיפה מגיעות השורות בגיליון: יישור מרובה, ומה מלמדות העמודות שבהן כולם מסכימים.

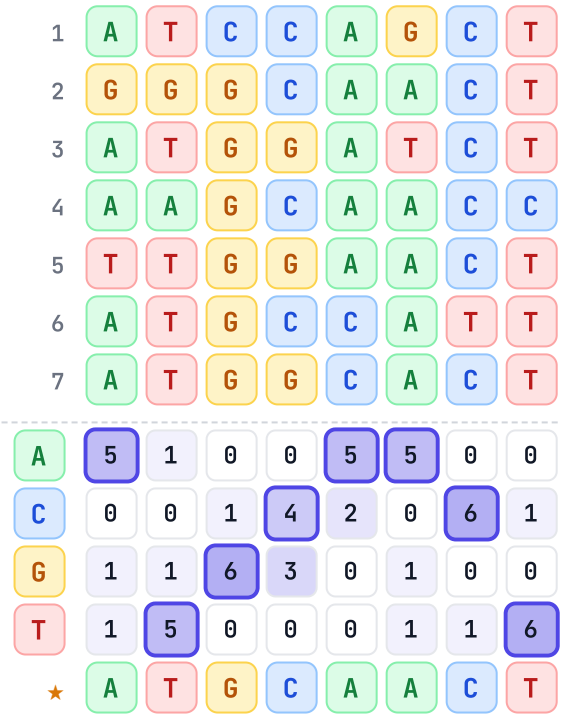
יישור רצפים מרובה (MSA ,Multiple Sequence Alignment) מסדר כמה רצפים הומולוגיים זה מעל זה, כך שכל עמודה מכילה עמדות בעלות מוצא משותף. כלים כמו Clustal Omega ,MAFFT ו-MUSCLE בונים יישורים כאלה מעשרות ועד מיליוני רצפים.

הברירה הטבעית פועלת כמסננת: מוטציה בעמדה חיונית לתפקוד, כמו האתר הפעיל של אנזים או בסיס שגורם שעתוק חייב לזהות, פוגעת ביצור ולכן נעלמת מהאוכלוסייה. כתוצאה מכך, **עמודות שמורות** ביישור מצביעות על עמדות חשובות, ועמודות משתנות מצביעות על עמדות שבהן לשינוי אין מחיר גבוה.

---

3 מטריצת פרופיל ורצף קונצנזוס

מטריצת פרופיל היא טבלה בגודל n × 4: שורה לכל בסיס ועמודה לכל עמדה ביישור. כל תא סופר כמה רצפים מכילים את הבסיס הזה בעמדה הזו. **רצף הקונצנזוס** נוצר כשבוחרים בכל עמודה את הבסיס עם הספירה הגבוהה ביותר.



למעלה: שבעה רצפים מיושרים. באמצע: מטריצת הפרופיל, כשהמקסימום בכל עמודה מודגש. למטה (★): רצף הקונצנזוס `ATGCAACT`.

> **⌘ כשיש תיקו**
>
> ייתכן ששני בסיסים יקבלו את אותה ספירה מקסימלית בעמודה. אין תשובה "נכונה" אחת, אבל צריך כלל עקבי. בתרגיל בוחרים את הבסיס **הראשון לפי סדר האלפבית** (A, אחריו C, G ו-T). שימו לב: `Counter.most_common` שובר תיקו לפי סדר ההופעה ולא לפי האלפבית, ולכן הוא עלול לתת תשובה אחרת. ביולוגים מסמנים לפעמים תיקו בקוד IUPAC, למשל `R` לתיקו בין A ל-G.

מקונצנזוס ל-PWM וללוגו רצף

רצף קונצנזוס מאבד מידע: עמודה עם 90% A ועמודה עם 40% A יקבלו שתיהן A. לכן מחלקים כל ספירה במספר הרצפים ומקבלים **מטריצת הסתברויות**, ומשם **מטריצת משקל-עמדה** (PWM): הלוגריתם של היחס בין כל הסתברות להסתברות הרקע. בעזרת PWM סורקים גנומים כדי למצוא אתרי קישור חדשים. הייצוג החזותי המוכר הוא **לוגו רצף** (sequence

logo): בכל עמודה נערמות האותיות, וגובהן פרופורציונלי לשימור, שנמדד בביטים. מאגרים כמו JASPAR מפרסמים אלפי מוטיבים כאלה.

תוכן מידע בעמודה — 0 עד 2 ביטים ב-DNA

$$IC = 2 + \Sigma_b \; p_b \cdot \log_2 p_b$$

---

4 מעבדה חזותית: בונים פרופיל

ערכו את היישור (שורה לכל רצף) ולחצו **סריקה**. בכל צעד נספרת עמודה אחת, ספירות הבסיסים נכנסות למטריצת הפרופיל, והבסיס הנבחר מתווסף לקונצנזוס. עמודות עם תיקו מסומנות, והבחירה בהן היא לפי סדר האלפבית.

מעבדה חזותית · מטריצת פרופיל וקונצנזוס

סופרים כל עמודה ביישור, ובוחרים את הבסיס השכיח ביותר. בתיקו בוחרים לפי הסדר A, C, G, T.

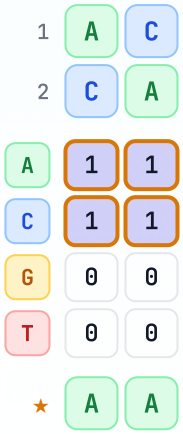
סריקה באמצע העבודה, בעמודה j = 3 (מוקפת בתכלת). למעלה: שבעת הרצפים. באמצע: מטריצת הפרופיל, שמתמלאת עמודה אחר עמודה; עוצמת הצבע היא שיעור הרצפים, והתא המוקף הוא המקסימום. למטה (★): הקונצנזוס שנבנה עד כה.



עמודה 3: `A=0, C=4, G=3, T=0` → `C`. בסוף הסריקה: **הקונצנזוס** `ATGCAACT`.

תיקו: ביישור ["AC", "CA"] בכל עמודה יש A אחד ו-C אחד. שני התאים מסומנים בכתום, והבחירה נופלת על A, הראשון בסדר A, C, G, T.

עמודה 0: `A=1, C=1, G=0, T=0` → `A` **תיקו בין A, C: נבחר A, הראשון באלפבית.** עמודה 1: אותו דבר. **הקונצנזוס:** `AA`



בגרסה המקוונת: מקלידים יישור (רצף בכל שורה, עד 12 רצפים ו-40 עמודות) ולוחצים **סריקה** או **צעד**. בכל צעד נספרת עמודה אחת, התאים במטריצה מתמלאים, ועמודות עם תיקו מסומנות בכתום.

5 מערכים דו-ממדיים ב-Python

ל-Python אין מערך דו-ממדי מובנה (בלי ספריות כמו NumPy), ולכן משתמשים ב**רשימה של רשימות**: הרשימה החיצונית מחזיקה שורות, וכל שורה היא רשימה של ערכים. ניגשים לתא בעזרת שני אינדקסים: `matrix[row][col]`. גם רשימה של מחרוזות באותו אורך מתנהגת כמו טבלה דו-ממדית.

```python
alignment = ["ATCC", "GGGC", "ATGG"]

# Rows are the strings themselves; one cell is alignment[row][col]
alignment[2][1]                 # 'T'  -> row 2, column 1

# Columns: zip(*rows) "transposes" the alignment
columns = list(zip(*alignment))
columns[0]                      # ('A', 'G', 'A')
len(columns)                    # 4    -> number of positions

# A 4 x n profile matrix as a list of lists, one row per base
BASES = "ACGT"
n = len(alignment[0])
profile = [[0] * n for _ in BASES]
for j, column in enumerate(zip(*alignment)):
    for base in column:
        profile[BASES.index(base)][j] += 1
print(profile)    # [[2, 0, 0, 0], [0, 0, 1, 2], [1, 1, 2, 1], [0, 2, 0, 0]]
```

⚠ המלכודת של `[[0] * n] * 4`

הביטוי הזה נראה כמו ארבע שורות, אבל בפועל הוא ארבע הפניות לאותה רשימה בדיוק. שינוי של `profile[0][0]` ישנה את כל השורות! תמיד בנו כל שורה מחדש בעזרת רשימת הבנה: `[[0] * n for _ in range(4)]`. תפגשו את המלכודת הזו שוב במודול 11.

```python
bad = [[0] * 3] * 2
bad[0][0] = 9
print(bad)        # [[9, 0, 0], [9, 0, 0]]  <- both rows changed!

good = [[0] * 3 for _ in range(2)]
good[0][0] = 9
print(good)       # [[9, 0, 0], [0, 0, 0]]
```

בחירת מקסימום עם כלל לשבירת תיקו

`max` מחזיר את האיבר הראשון עם הערך הגבוה ביותר. אם עוברים על הבסיסים בסדר `"ACGT"`, תיקו נשבר אוטומטית לטובת הבסיס שקודם באלפבית:

```python
column = ("A", "C", "C", "A")
max("ACGT", key=column.count)       # 'A'  -> A and C tie at 2; A comes first
```

זמן	זיכרון
$O(m \cdot n)$	$O(n)$
m רצפים באורך n: כל תו נספר פעם אחת	מטריצת פרופיל n × 4 ורצף קונצנזוס באורך n

---

6 דוגמאות קוד

לפני שתכתבו פתרון משלכם, הנה דוגמאות מוכנות שמשתמשות באותם כלים ורעיונות. מתחת לכל דוגמה מופיע **הפלט המלא** של הרצת הקוד (כפי שהתקבל מ-Python 3), ואחריו ההסבר המפורט. מומלץ להקליד את הקוד בעצמכם, להריץ אותו, ואז לשנות אותו ולבדוק איך הפלט משתנה. בגרסת האתר כל דוגמה ניתנת להרצה בלחיצת כפתור.

דוגמה 1 מטריצת פרופיל בפורמט של Rosalind

הדוגמה בונה את מטריצת הפרופיל של שבעת הרצפים מהאיור, כמילון שבו לכל בסיס יש רשימת ספירות, ומדפיסה אותה

בפורמט של בעיית CONS ב-Rosalind.

```python
dna_list = [
    "ATCCAGCT", "GGGCAACT", "ATGGATCT", "AAGCAACC",
    "TTGGAACT", "ATGCCATT", "ATGGCACT",
]
BASES = "ACGT"
n = len(dna_list[0])

profile = {base: [0] * n for base in BASES}
for seq in dna_list:
    for j, base in enumerate(seq):
        profile[base][j] += 1

for base in BASES:
    print(base + ":", " ".join(str(count) for count in profile[base]))

column_totals = [sum(profile[b][j] for b in BASES) for j in range(n)]
print("column totals:", column_totals)
```

```
$ python example.py
A: 5 1 0 0 5 5 0 0
C: 0 0 1 4 2 0 6 1
G: 1 1 6 3 0 1 0 0
T: 1 5 0 0 0 1 1 6
column totals: [7, 7, 7, 7, 7, 7, 7, 7]
```

איך זה עובד, שלב אחר שלב

1. `{base: [0] * n for base in BASES}` היא הבנת מילון: היא יוצרת ארבע רשימות נפרדות של אפסים, אחת לכל בסיס. כאן אין מלכודת הפניות, כי `[0] * n` מחושב מחדש עבור כל מפתח.
2. הלולאה החיצונית עוברת על הרצפים (השורות), ו-`enumerate(seq)` נותן לכל תו את מספר העמודה שלו. `profile[base][j] += 1` מעלה את הספירה בתא הנכון.
3. ההדפסה ממירה כל ספירה למחרוזת ומחברת ברווחים. `" ".join` דורש מחרוזות, ולכן צריך את `str(count)`.
4. `column_totals` היא בדיקת שפיות: בכל עמודה סכום הספירות חייב להיות שווה למספר הרצפים, 7. בדיקות כאלה תופסות באגים של אינדקסים מוקדם.
5. כדי לקבל רצף קונצנזוס מהטבלה, בוחרים בכל עמודה את הבסיס עם הספירה הגבוהה ביותר. את השלב הזה, כולל כלל שבירת התיקו, תממשו בתרגיל.

דוגמה 2 תוכן מידע לכל עמודה: הבסיס ללוגו רצף

כמה "מידע" יש בכל עמודה של מוטיב? עמודה שמורה לגמרי שווה 2 ביטים, ועמודה אקראית לגמרי שווה 0. הדוגמה

מחשבת זאת עבור אתרי TATA box, ומשרטטת פס טקסטואלי לכל עמדה.

PYTHON

```python
import math

sites = ["TATAAA", "TATATA", "TATAAT", "TATAAA", "CATAAA", "TATAAA"]

for j, column in enumerate(zip(*sites)):
    n = len(column)
    entropy = 0.0
    for base in "ACGT":
        p = column.count(base) / n
        if p > 0:
            entropy -= p * math.log2(p)
    info = 2 - entropy
    print(f"pos {j}: {''.join(column)}  IC = {info:.2f} bits  {'#' * round(info * 10)}")
```

```
$ python example.py
pos 0: TTTTCT  IC = 1.35 bits  #############
pos 1: AAAAAA  IC = 2.00 bits  ####################
pos 2: TTTTTT  IC = 2.00 bits  ####################
pos 3: AAAAAA  IC = 2.00 bits  ####################
pos 4: ATAAAA  IC = 1.35 bits  #############
pos 5: AATAAA  IC = 1.35 bits  #############
```

איך זה עובד, שלב אחר שלב

1. `zip(*sites)` מחזיר את העמודות, ו-`enumerate` מוסיף לכל אחת את מספר העמדה.
2. `column.count(base) / n` הוא שיעור הבסיס בעמודה, כלומר הסתברות. עבור p שווה 0 מדלגים, כי $\log_2 0$ אינו מוגדר (ובגבול, התרומה שלו היא 0).
3. האנטרופיה של שאנון, $H = -\Sigma\, p \cdot \log_2 p$, מודדת אי-ודאות: 0 לעמודה שמורה לגמרי, ו-2 ביטים לעמודה שבה ארבעת הבסיסים שווים בשכיחותם.
4. תוכן המידע הוא $2 - H$. עמודות 1, 2 ו-3, שבהן אותו בסיס בכל האתרים (`AAAAAA`, `TTTTTT`, `AAAAAA`), מקבלות את המקסימום: 2 ביטים. עמודות 0, 4 ו-5, שבהן יש שונות, מקבלות פחות.
5. `'#' * round(info * 10)` משרטט פס באורך פרופורציונלי למידע. בלוגו רצף אמיתי, זה גובה ערימת האותיות בכל עמודה.

דוגמה 3 סריקת רצף בעזרת מטריצת משקל-עמדה (PWM)

מהאתרים הידועים בונים PWM, ואז "מעבירים" אותה לאורך רצף חדש ונותנים ציון לכל חלון. ציון גבוה פירושו חלון שדומה

למוטיב, גם אם הוא לא זהה לאף אחד מהאתרים המקוריים.

```python
import math

sites = ["TATAAA", "TATATA", "TATAAT", "TATAAA", "CATAAA", "TATAAA"]
BASES = "ACGT"
k = len(sites[0])

# Log-odds weights with a pseudocount of 1 and a uniform 25% background
pwm = []
for column in zip(*sites):
    total = len(column) + 4
    pwm.append({b: math.log2(((column.count(b) + 1) / total) / 0.25) for b in BASES})

def score(window):
    return sum(pwm[j][base] for j, base in enumerate(window))

promoter = "GCGCTATAAAAGGCCTATTTAGGC"
hits = [(score(promoter[i:i + k]), i) for i in range(len(promoter) - k + 1)]
for s, i in sorted(hits, reverse=True)[:3]:
    print(f"index {i:>2}: {promoter[i:i + k]}  score = {s:+.2f}")
```

```
$ python example.py
index  4: TATAAA  score = +8.25
index 15: TATTTA  score = +3.85
index  6: TAAAAG  score = +2.85
```

איך זה עובד, שלב אחר שלב

1. לכל עמודה מחושב מילון משקלים, אחד לכל בסיס. המשקל הוא $\log_2$ של היחס בין שכיחות הבסיס בעמודה לבין שכיחות הרקע (25%). ציון חיובי פירושו "שכיח יותר מבמקרה".
2. Pseudocount של 1 מתווסף לכל ספירה (ולכן המכנה הוא 4 + len(column)). בלעדיו, בסיס שלא הופיע באף אתר היה מקבל הסתברות 0, והלוגריתם שלו היה מינוס אינסוף.
3. `score` מסכמת את משקלי הבסיסים בחלון, עמדה אחר עמדה. זהו בדיוק הניקוד של מודול 7, רק שהפעם לכל עמדה יש "מטריצת ניקוד" משלה.
4. רשימת ההבנה מנקדת כל חלון ברצף, ו-`sorted(hits, reverse=True)[:3]` מחזיר את שלושת הגבוהים ביותר. tuples ממוינים קודם לפי האיבר הראשון, הציון.
5. `TATAAA` באינדקס 4 מקבל את הציון הגבוה ביותר. חלונות שדומים לו רק חלקית מקבלים ציונים נמוכים יותר, כך שהשיטה מדרגת אתרים לפי חוזקם ולא רק לפי "כן או לא". כך עובדים כלים כמו FIMO ו-HOMER.

7 המשימה שלכם

ממשו את `consensus_sequence(dna_list)`. הבדיקות הנסתרות כוללות רשימה ריקה, רצף בודד, הרבה עמודות עם תיקו, ויישור גדול של 60 רצפים באורך 80.

מודול 10 · אתגר קוד בסגנון LEETCODE

✎ התרגיל: `consensus_sequence(dna_list)`

קשה | יישור מרובה | רשימות של רשימות | zip על עמודות | כ-50 דקות

נתונה רשימה `dna_list` של מחרוזות DNA באורך שווה, שמייצגת יישור: כל מחרוזת היא שורה, וכל אינדקס הוא עמודה.

ממשו את `consensus_sequence(dna_list)` כך שתחזיר את **רצף הקונצנזוס**: מחרוזת שבה בכל עמדה מופיע הבסיס השכיח ביותר בעמודה המתאימה.

- אם יש תיקו בין כמה בסיסים, בחרו את הראשון לפי הסדר `A` , `C` , `G` , `T` .
- אם הרשימה ריקה, החזירו מחרוזת ריקה `""` .

דוגמאות

דוגמה 1

קלט: `dna_list = ["ATCCAGCT", "GGGCAACT", "ATGGATCT", "AAGCAACC", "TTGGAACT", "ATGCCATT", "ATGGCACT"]`

פלט: `"ATGCAACT"`

הסבר: זה הקונצנזוס מהאיור בשיעור.

דוגמה 2

קלט: `dna_list = ["AC", "CA"]`

פלט: `"AA"`

הסבר: בשתי העמודות יש תיקו בין A ל-C, ו-A קודם באלפבית.

דוגמה 3

קלט: `dna_list = ["GT", "GT", "AC"]`

פלט: `"GT"`

אילוצים

- `0 ≤ len(dna_list) ≤ 500` , ואורך כל מחרוזת עד `1000`
- כל המחרוזות באותו אורך, ומכילות רק `A` , `C` , `G` , `T`
- יש להחזיר `str`

רמזים

💡 `zip(*dna_list)` מחזיר את העמודות אחת אחרי השנייה.

💡 עבור כל עמודה, ספרו כמה פעמים מופיע כל בסיס. `column.count(base)` מספיק.

💡 `max("ACGT", key=column.count)` בוחר את הבסיס עם הספירה הגבוהה ביותר, ושובר תיקו לטובת הקודם באלפבית.

💡 רשימה ריקה: `zip()` בלי ארגומנטים מחזיר אפס עמודות, וזה בדיוק מה שצריך.

בדיקות גלויות (הרצה)

אלה הבדיקות שכפתור **הרצה** באתר מריץ. הפתרון חייב לעבור את כולן.

קריאה	תוצאה צפויה
`consensus_sequence(["ATCCAGCT", "GGGCAACT", "ATGGATCT", "AAGCAACC", "TTGGAACT", "ATGCCATT", "ATGGCACT"])`	`"ATGCAACT"`
`consensus_sequence(["AC", "CA"])`	`"AA"`
`consensus_sequence(["GT", "GT", "AC"])`	`"GT"`

בדיקות נסתרות: מקרי קצה (הגשה)

כפתור **הגשה** מריץ גם את הבדיקות האלה. הן נועדו לתפוס קלטים ריקים, מקרי גבול, יעילות על קלט גדול וטיפול בשגיאות. כתבו את הפתרון כך שיעמוד בהן **לפני** שאתם מציצים בטבלה.

קריאה	תוצאה צפויה	מה נבדק
`consensus_sequence([])`	`""`	קלט ריק
`consensus_sequence(["ACGT"])`	`"ACGT"`	
`consensus_sequence(["T", "G"])`	`"G"`	
`consensus_sequence(["", ""])`	`""`	קלט ריק
`consensus_sequence(["AAAA", "CCCC", "GGGG", "TTTT"])`	`"AAAA"`	
`consensus_sequence(["TTTA", "TTTC", "GGGC"])`	`"TTTC"`	
`consensus_sequence(["CGT", "GTC", "TCG", "TGC"])`	`"TGC"`	
`consensus_sequence(רשימה של 60 רצפים באורך 80)`	`"AAAGGCGTATAAGCGTTACCGTCGTCAGAT…"` (מחרוזת באורך 80)	יישור גדול (60 × 80)

קוד התחלתי

כך נראה העורך כשפותחים את המודול. ממשו את גוף הפונקציה במקום `pass`.

PYTHON

```python
def consensus_sequence(dna_list):
    """
    Return the consensus of equal-length DNA strings: the most frequent
    base in every column, breaking ties in the order A, C, G, T.

    >>> consensus_sequence(["AC", "CA"])
    'AA'
    """
    # Write your code here
    pass
```

🔑 פתרון עיון מלא: `consensus_sequence`

הפתרון שעובר את כל הבדיקות, הגלויות והנסתרות. מספרי השורות בהסבר מתייחסים לקוד שלמעלה.

```python
def consensus_sequence(dna_list):
    consensus = []
    for column in zip(*dna_list):
        # max() keeps the first maximum, so iterating "ACGT" breaks ties alphabetically
        consensus.append(max("ACGT", key=column.count))
    return "".join(consensus)
```

הסבר שורה אחר שורה

שורה 2 — רשימת צובר לבסיסי הקונצנזוס, אות אחת לכל עמודה.

שורה 3 — **מעבר על עמודות.** `zip(*dna_list)` פורק את הרשימה לארגומנטים נפרדים ומחזיר בכל צעד tuple עם התו של כל רצף באותה עמדה: "שיקוף" של היישור. אם הרשימה ריקה, `zip()` בלי ארגומנטים מחזיר אפס עמודות, והלולאה לא רצה כלל.

שורה 4 — ההערה מסבירה את הטריק המרכזי: `max` מחזיר את האיבר הראשון בעל הערך המקסימלי.

שורה 5 — **בחירת הבסיס השכיח עם שבירת תיקו.** `max("ACGT", key=column.count)` עובר על ארבעת הבסיסים בסדר A, C, G, T, ומודד כל אחד לפי מספר המופעים שלו בעמודה (`tuple.count`). בתיקו נבחר הבסיס שהופיע קודם במחרוזת `"ACGT"`, כלומר הראשון באלפבית, בדיוק כפי שהתרגיל דורש.
(`Counter.most_common` היה שובר תיקו לפי סדר ההופעה בעמודה, וזה עלול לתת תשובה אחרת.)

שורה 6 — `"".join(consensus)` מחבר את האותיות למחרוזת. לרשימה ריקה, או לרשימה של מחרוזות ריקות, מתקבלת `""`.

⌘ למה זה הפתרון האופטימלי

m רצפים באורך n: כל תו נספר מספר קבוע של פעמים (ארבע קריאות `count` לכל עמודה), ולכן O(m·n) בזמן ו-O(n) בזיכרון נוסף (העמודה הנוכחית והקונצנזוס). זו הגרסה הקצרה ביותר שגם ממשת נכון את כלל התיקו; מטריצת פרופיל מלאה (כמו בדוגמת הקוד) נחוצה רק אם רוצים גם את הספירות עצמן.

💡 איך הפתרון עומד במקרי הקצה הנסתרים

`[]` → `""`; `["ACGT"]` → `"ACGT"` (רצף בודד הוא הקונצנזוס של עצמו); `["T", "G"]` → `"G"` (תיקו: G לפני T); `["", ""]` → `""`; `["AAAA", "CCCC", "GGGG", "TTTT"]` → `"AAAA"` (תיקו משולש בכל עמודה); יישור של 60 רצפים באורך 80 נפתר באותו קוד בדיוק.

מודול 11 · ביולוגיה × מדעי המחשב

תכנון דינמי: מטריצת היישור

מומחה | כ-60 דקות | Needleman–Wunsch | עונשי פער | מטריצות דו-ממדיות | `init_dp_matrix()`

מעבדה חזותית: רשת Needleman–Wunsch: אתחול, מילוי תא-אחר-תא ו-traceback

במודול 7 ניקדתם יישור **נתון**. אבל איך מוצאים את היישור הטוב ביותר מבין מספר עצום של אפשרויות? התשובה היא **תכנון דינמי**: גיליון אלקטרוני שבו כל תא שומר את התשובה לתת-בעיה קטנה, וממלאים אותו צעד אחר צעד. זה הרעיון שמאחורי אלגוריתם Needleman–Wunsch, ובמודול הזה תבנו את השלב הראשון שלו.

מה תלמדו

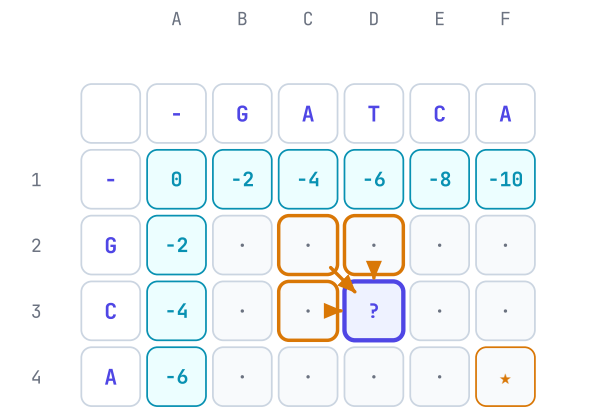
- הביולוגיה בשפה של מתכנתים: תכנון דינמי כגיליון של תשובות שמורות (memoization)
- למה אי אפשר פשוט לבדוק את כל היישורים האפשריים
- הרעיון של תכנון דינמי: תת-בעיות חופפות ומבנה אופטימלי
- מטריצת Needleman–Wunsch: אתחול, מילוי ו-traceback
- יצירה ומילוי של מטריצות דו-ממדיות ב-Python, בלי מלכודת ההפניות המשותפות

1 הביולוגיה בשפה של מתכנתים

כדי למצוא את היישור הטוב ביותר אי אפשר לנסות את כל האפשרויות: יש יותר מהן מאטומים ביקום. **תכנון דינמי** פותר את זה בטריק שמתכנתים מכירים בשם memoization: במקום לחשב את אותה תת-בעיה שוב ושוב, שומרים את התשובה בטבלה ומשתמשים בה שוב. כאן הטבלה היא **גיליון אלקטרוני**: התא `F[i][j]` מחזיק את הציון הטוב ביותר ליישור של i האותיות הראשונות של רצף 1 מול j האותיות הראשונות של רצף 2, והנוסחה בכל תא מסתכלת על שלושת השכנים שלו: באלכסון, מלמעלה ומשמאל.

כמו בכל רקורסיה, צריך **מקרי בסיס**: השורה והעמודה הראשונות מתארות יישור מול מחרוזת ריקה, שעולה פער אחד לכל אות, ולכן הן מתמלאות בקנסות מצטברים (0, −2, −4, …). זה בדיוק החלק שתבנו בתרגיל. אחרי שהטבלה מלאה, הציון הסופי יושב בפינה, וכדי לשחזר את היישור עצמו הולכים אחורה לאורך "ההפניות" של הנוסחאות, כמו "trace precedents" בגיליון אלקטרוני. זהו אלגוריתם Needleman–Wunsch, והוא הבסיס לכל כלי היישור שתפגשו.

איור תכנון דינמי כגיליון אלקטרוני: כל תא הוא תשובה שמורה



השורה והעמודה הראשונות = מקרי הבסיס: יישור מול מחרוזת ריקה, פער לכל אות
★ הפינה = התשובה הסופית, אחרי שכל התאים מולאו

הנוסחה בכל תא (כמו תא בגיליון אלקטרוני)

```
F[2][3] = max(
  F[1][2] + s(C, T),      ↖ שני התווים מיושרים
  F[1][3] + gap,          ↑ פער ברצף 2
  F[2][2] + gap )         ← פער ברצף 1
-----------------------------------
```

כל תא = תשובה שמורה (memo) לתת-בעיה אחת:
הציון הטוב ביותר ליישור i האותיות הראשונות
של רצף 1 מול j הראשונות של רצף 2.

למה זה עובד?

כוח גס: יותר מ-10^{75} יישורים לשני רצפים באורך 100.
תכנון דינמי: כל תת-בעיה נפתרת פעם אחת ונשמרת,
ולכן רק $(n+1)\cdot(m+1)$ תאים, $O(1)$ עבודה לכל תא.

המטריצה של GCA (שורות) מול GATCA (עמודות). השורה והעמודה הראשונות (תכלת) הן מקרי הבסיס שתמלאו בתרגיל: יישור מול מחרוזת ריקה עולה פער לכל אות. כל תא אחר מחושב משלושת שכניו (החצים): באלכסון, מלמעלה ומשמאל. הפינה (★) תחזיק בסוף את ציון היישור הטוב ביותר.

מילון מונחים: ביולוגיה ↔ מדעי המחשב

בביולוגיה	במדעי המחשב	למה הדימוי עובד
היישור הטוב ביותר	**אופטימיזציה על מרחב עצום**	אי אפשר לנסות הכול; צריך לפרק לתת-בעיות.
תכנון דינמי	**memoization בטבלה**	כל תת-בעיה נפתרת פעם אחת, התשובה נשמרת ומשמשת שוב.
המטריצה F	**גיליון אלקטרוני**	תא לכל זוג תחיליות; הנוסחה בכל תא מפנה לשלושה שכנים.
אתחול	**מקרי הבסיס של הרקורסיה**	יישור מול מחרוזת ריקה = פער לכל אות: 0, −2, −4, …
מילוי	**חישוב הגיליון שורה אחר שורה**	כשמגיעים לתא, שלושת שכניו כבר מחושבים.
traceback	**מעקב / "trace precedents" אחורה**	הולכים מהפינה חזרה לפי ההפניות ומשחזרים את היישור עצמו.
המלכודת `[[0]*m]*n`	**n הפניות לאותה שורה**	כל השורות הן אותו אובייקט; שינוי באחת משנה את כולן.

⚡ בשורה אחת

תכנון דינמי הוא גיליון אלקטרוני של תשובות שמורות: השורה והעמודה הראשונות הן מקרי הבסיס (פער לכל אות), וכל תא אחר הוא המקסימום של שלושת שכניו ועוד הציון המתאים.

2 כמה יישורים יש?

> קודם נבין למה כוח גס לא עובד כאן, ואז נגיע לתובנה שמאפשרת לפתור את הבעיה בזמן סביר.

בין שני רצפים באורך 100 יש יותר מ-10^{75} יישורים גלובליים אפשריים: בכל עמדה אפשר להתאים שני תווים, או להכניס פער באחד מהם. גם מחשב שבודק מיליארד יישורים בשנייה לא יסיים לפני סוף היקום. צריך דרך חכמה יותר.

התובנה של Needleman ו-Wunsch (1970)

ליישור הטוב ביותר יש מבנה רקורסיבי. הסתכלו על העמודה האחרונה שלו: יש בה רק שלוש אפשרויות.

- שני התווים האחרונים מיושרים זה מול זה (התאמה או אי-התאמה);
- התו האחרון של רצף 1 מול פער;
- התו האחרון של רצף 2 מול פער.

בכל אחד מהמקרים, מה שנשאר לפני העמודה האחרונה הוא יישור טוב ביותר של תחיליות קצרות יותר. אם כבר יודעים את הציונים הטובים ביותר של התחיליות, הציון הטוב ביותר עכשיו הוא פשוט המקסימום מבין שלוש האפשרויות. זהו **תכנון דינמי**: פותרים כל תת-בעיה פעם אחת, שומרים את התוצאה בטבלה, ומשתמשים בה שוב ושוב.

נוסחת הנסיגה

s הוא ציון ההתאמה או אי-ההתאמה

```
F[i][j] = max( F[i-1][j-1] + s(x_i, y_j),  F[i-1][j] + gap,  F[i][j-1] + gap )
```

3 המטריצה ואתחולה

המטריצה `F` היא בגודל (len(seq2) + 1) × (len(seq1) + 1). התא `F[i][j]` מחזיק את הציון הטוב ביותר של יישור `seq1[:i]` מול `seq2[:j]`. השורה והעמודה הנוספות מייצגות את **התחילית הריקה**.

השורה והעמודה הראשונות הן **מקרי הבסיס** של הרקורסיה. יישור של תחילית באורך i מול מחרוזת ריקה מחייב i פערים, ולכן F[i][0] = i × gap, ובאופן דומה F[0][j] = j × gap. התא `F[0][0] = 0` , כי יישור של שתי מחרוזות ריקות לא עולה כלום. זה בדיוק מה שתבנו בתרגיל.

	-	G	A	T	C	A
-	0	-2	-4	-6	-8	-10
G	-2	·	·	·	·	·
C	-4	·	·	·	·	·
A	-6	·	·	·	·	·

המטריצה של `seq1 = "GCA"` (שורות) ו-`seq2 = "GATCA"` (עמודות) עם עונש פער של −2, אחרי האתחול. התאים המסומנים ב-· עוד לא חושבו.

⌘ למה עונש הפער מצטבר?

כל פער נוסף מוסיף את העונש פעם נוספת, ולכן עונש של 2− לפער נותן 0, −2, −4, −6, … לאורך השורה הראשונה. ביישור מקומי (Smith–Waterman) מאתחלים דווקא את השורה והעמודה הראשונות ל-0, כי מותר להתחיל את היישור בכל מקום בלי עונש. ההבדל הקטן הזה באתחול הוא כל ההבדל בין יישור גלובלי ליישור מקומי.

מילוי ו-traceback

אחרי האתחול ממלאים את שאר התאים שורה אחר שורה, ובכל שורה עמודה אחר עמודה, כך ששלושת השכנים של כל תא (באלכסון, מעליו ומשמאלו) כבר חושבו. הציון של היישור הטוב ביותר נמצא בפינה הימנית התחתונה. כדי לשחזר את היישור עצמו, הולכים אחורה מהפינה (traceback) ובכל תא בודקים מאיזה שכן הגענו: אלכסון פירושו שני תווים מיושרים, למעלה פירושו פער ברצף 2, ושמאלה פירושו פער ברצף 1.

---

4 מעבדה חזותית: ממלאים את המטריצה

לחצו **אתחול** כדי לראות את השורה והעמודה הראשונות מתמלאות בעונשי פער מצטברים: זה החלק של התרגיל. אחר כך **מילוי** מחשב כל תא מתוך שלושת שכניו, ו-**traceback** מסמן את המסלול חזרה מהפינה ומציג את היישור האופטימלי. נסו לשנות את עונש הפער ולראות איך היישור משתנה.

מעבדה חזותית · מטריצת Needleman–Wunsch

אתחול → מילוי → traceback. כל תא הוא המקסימום מבין שלושה שכנים.

שלב א · אתחול (זה החלק של התרגיל): מטריצה בגודל 8 × 8. השורה הראשונה והעמודה הראשונה מתמלאות בעונשי פער מצטברים, 0, −2, −4, …; שאר התאים (·) עוד לא חושבו.

	-	G	C	A	T	G	C	A
-	0	-2	-4	-6	-8	-10	-12	-14
G	-2	·	·	·	·	·	·	·
A	-4	·	·	·	·	·	·	·
T	-6	·	·	·	·	·	·	·
T	-8	·	·	·	·	·	·	·
A	-10	·	·	·	·	·	·	·
C	-12	·	·	·	·	·	·	·
A	-14	·	·	·	·	·	·	·

אתחול: `F[0][0] = 0` . יישור של שתי מחרוזות ריקות לא עולה כלום. `F[0][3] = 3 × (-2) = -6` : 3 תווים מ-seq2 מול פערים. `F[2][0] = 2 × (-2) = -4` : 2 תווים מ-seq1 מול פערים.

שלב ב · מילוי, שורה אחר שורה: התא הנוכחי הוא F[3][4] (seq1[2] = T מול seq2[3] = T). שלושת השכנים שלו, באלכסון, מעליו ומשמאלו, כבר חושבו.

	-	G	C	A	T	G	C	A
-	0	-2	-4	-6	-8	-10	-12	-14
G	-2	1	-1	-3	-5	-7	-9	-11
A	-4	-1	0	0	-2	-4	-6	-8
T	-6	-3	-2	-1	1	·	·	·
T	-8	·	·	·	·	·	·	·
A	-10	·	·	·	·	·	·	·
C	-12	·	·	·	·	·	·	·
A	-14	·	·	·	·	·	·	·

`F[3][4]`: `T vs T` (התאמה). **1 ↖ אלכסון** · -4 ↑ למעלה · -3 ← שמאלה → **1**

שלב ג · המטריצה המלאה ומסלול ה-traceback (ירוק) מהפינה הימנית התחתונה חזרה ל-F[0][0]. צעד באלכסון = שני תווים מיושרים; למעלה = פער ב-seq2; שמאלה = פער ב-seq1.

	-	G	C	A	T	G	C	A
-	0	-2	-4	-6	-8	-10	-12	-14
G	-2	1	-1	-3	-5	-7	-9	-11
A	-4	-1	0	0	-2	-4	-6	-8
T	-6	-3	-2	-1	1	-1	-3	-5
T	-8	-5	-4	-3	0	0	-2	-4
A	-10	-7	-6	-3	-2	-1	-1	-1
C	-12	-9	-6	-5	-4	-3	0	-2
A	-14	-11	-8	-5	-6	-5	-2	1

אתחול (התרגיל) · התא הנוכחי · שכנים · מסלול ה-traceback

המטריצה מלאה. ציון היישור הגלובלי הטוב ביותר: `F[7][7] = 1`. ה-traceback שחזר יישור אופטימלי:

```
GATTACA
|..|.||
GCATGCA
```

בגרסה המקוונת: מקלידים שני רצפים (עד 10 תווים), קובעים התאמה/אי-התאמה/פער, ולוחצים **אתחול ומילוי** או **צעד**: כל תא מחושב בנפרד, עם שלושת שכניו מודגשים וההסבר מאיזה כיוון הגיע המקסימום. **traceback** מסמן את המסלול ומציג את היישור.

5 מטריצות ב-Python: יצירה נכונה

```python
n, m = 3, 4

# Correct: every row is a brand-new list
matrix = [[0] * (m + 1) for _ in range(n + 1)]

# The same thing with explicit loops
matrix = []
for i in range(n + 1):
    matrix.append([0] * (m + 1))

len(matrix), len(matrix[0])     # (4, 5)  -> rows, columns
matrix[2][3] = 7                # row 2, column 3
```

⚠️ המלכודת: `[[0] * (m + 1)] * (n + 1)`

האופרטור `*` על רשימה משכפל הפניות, לא העתקים. כל השורות יהיו אותו אובייקט, וכשתכתבו `matrix[i][0] = i * gap` בלולאה, כל השורות ישתנו יחד. בסוף, העמודה הראשונה של כל השורות תכיל רק את הערך האחרון שנכתב, וגם `matrix[0][0]` כבר לא יהיה 0. הבדיקות הנסתרות בתרגיל תופסות בדיוק את הבאג הזה.

```python
gap = -2
bad = [[0] * 3] * 3
for i in range(3):
    bad[i][0] = i * gap
print(bad)                # [[-4, 0, 0], [-4, 0, 0], [-4, 0, 0]]  <- one row, three names
print(bad[0] is bad[1])   # True
```

אתחול השורה והעמודה הראשונות

```python
gap = -2
first_row = [j * gap for j in range(m + 1)]    # [0, -2, -4, -6, -8]

for i in range(1, n + 1):                      # column 0, below the corner
    matrix[i][0] = i * gap
```

זמן $O(n \cdot m)$

יצירת המטריצה; גם המילוי המלא הוא $O(n \cdot m)$, $O(1)$ לכל תא

זיכרון $O(n \cdot m)$

התא לכל זוג תחיליות (אפשר לרדת ל-$O(m)$ אם צריך רק את הציון)

6 דוגמאות קוד

לפני שתכתבו פתרון משלכם, הנה דוגמאות מוכנות שמשתמשות באותם כלים ורעיונות. מתחת לכל דוגמה מופיע **הפלט** של הרצת הקוד (כפי שהתקבל מ-Python 3), ואחריו ההסבר המפורט. מומלץ להקליד את הקוד בעצמכם, להריץ אותו, ואז לשנות אותו ולבדוק איך הפלט משתנה. בגרסת האתר כל דוגמה ניתנת להרצה בלחיצת כפתור.

דוגמה 1 מילוי מלא של מטריצת Needleman–Wunsch

אחרי האתחול, שתי לולאות מקוננות ממלאות כל תא לפי נוסחת הנסיגה. הדוגמה מדפיסה את המטריצה המלאה עבור הזוג הקלאסי GATTACA ו-GCATGCA, ואת הציון הסופי.

PYTHON

```python
def needleman_wunsch_matrix(seq1, seq2, match=1, mismatch=-1, gap=-2):
    n, m = len(seq1), len(seq2)
    F = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        F[i][0] = i * gap
    for j in range(1, m + 1):
        F[0][j] = j * gap
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            s = match if seq1[i - 1] == seq2[j - 1] else mismatch
            F[i][j] = max(F[i - 1][j - 1] + s,   # diagonal: align two characters
                          F[i - 1][j] + gap,     # up: gap in seq2
                          F[i][j - 1] + gap)     # left: gap in seq1
    return F

seq1, seq2 = "GATTACA", "GCATGCA"
F = needleman_wunsch_matrix(seq1, seq2)

print("     " + "".join(f"{c:>4}" for c in "-" + seq2))
for i, row in enumerate(F):
    label = "-" if i == 0 else seq1[i - 1]
    print(f"{label:>4} " + "".join(f"{v:>4}" for v in row))
print("best global alignment score:", F[-1][-1])
```

```
$ python example.py
        -   G   C   A   T   G   C   A
   -    0  -2  -4  -6  -8 -10 -12 -14
   G   -2   1  -1  -3  -5  -7  -9 -11
   A   -4  -1   0   0  -2  -4  -6  -8
   T   -6  -3  -2  -1   1  -1  -3  -5
   T   -8  -5  -4  -3   0   0  -2  -4
   A  -10  -7  -6  -3  -2  -1  -1  -1
   C  -12  -9  -6  -5  -4  -3   0  -2
   A  -14 -11  -8  -5  -6  -5  -2   1
best global alignment score: 1
```

איך זה עובד, שלב אחר שלב

1. שלושת החלקים הראשונים של הפונקציה הם האתחול שתממשו בתרגיל: יצירת מטריצה עם שורות נפרדות, ואז העמודה הראשונה והשורה הראשונה.

2. הלולאות המקוננות עוברות שורה אחר שורה ועמודה אחר עמודה, החל מ-1. כשמחשבים את `F[i][j]`, שלושת השכנים שלו כבר חושבו.

3. `seq1[i - 1]` ולא `seq1[i]`: שורה i במטריצה מייצגת את התחילית באורך i, שהתו האחרון שלה נמצא באינדקס i − 1. זו ההזזה באחד שנובעת מהשורה הנוספת של התחילית הריקה.

4. `max` של שלוש האפשרויות הוא נוסחת הנסיגה עצמה. ההערות בקוד מסבירות מה כל כיוון אומר על היישור.

5. ההדפסה מעצבת כל מספר ברוחב 4 (`{v:>4}`), כך שהעמודות מתיישרות. `F[-1][-1]` הוא התא בפינה הימנית התחתונה: הציון של היישור הגלובלי הטוב ביותר.

דוגמה 2 Traceback: שחזור היישור עצמו

הציון לבדו לא מספיק: רוצים לראות את היישור. אחרי המילוי הולכים אחורה מהפינה, ובכל תא בודקים איזו מבין שלוש האפשרויות יצרה את הערך שלו.

PYTHON

```python
def align(seq1, seq2, match=1, mismatch=-1, gap=-2):
    n, m = len(seq1), len(seq2)
    F = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        F[i][0] = i * gap
    for j in range(m + 1):
        F[0][j] = j * gap
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            s = match if seq1[i - 1] == seq2[j - 1] else mismatch
            F[i][j] = max(F[i - 1][j - 1] + s, F[i - 1][j] + gap, F[i][j - 1] + gap)

    top, bottom = [], []
    i, j = n, m
    while i > 0 or j > 0:
        diagonal_ok = i > 0 and j > 0
        s = match if diagonal_ok and seq1[i - 1] == seq2[j - 1] else mismatch
        if diagonal_ok and F[i][j] == F[i - 1][j - 1] + s:
            top.append(seq1[i - 1])
            bottom.append(seq2[j - 1])
            i, j = i - 1, j - 1
        elif i > 0 and F[i][j] == F[i - 1][j] + gap:
            top.append(seq1[i - 1])
            bottom.append("-")
            i -= 1
        else:
            top.append("-")
            bottom.append(seq2[j - 1])
            j -= 1
    return "".join(reversed(top)), "".join(reversed(bottom)), F[n][m]

a, b, score = align("GATTACA", "GCATGCA")
print(a)
print("".join("|" if x == y else " " for x, y in zip(a, b)))
```

```
print(b)
print("score:", score)
```

```
$ python example.py
GATTACA
|  | ||
GCATGCA
score: 1
```

איך זה עובד, שלב אחר שלב

1. החלק הראשון זהה לדוגמה הקודמת: אתחול ומילוי. שימו לב שכאן האתחול מתחיל מ-0, וזה תקין, כי 0 = gap × 0.
2. ה-traceback מתחיל בפינה `(n, m)`, ורץ כל עוד לא הגענו ל-`(0, 0)`. התנאי `i > 0 or j > 0` מטפל גם במצב שבו נשארו תווים רק באחד הרצפים.
3. בכל צעד בודקים קודם את האלכסון: אם הערך בתא שווה לשכן באלכסון ועוד ציון ההתאמה, שני התווים מיושרים זה מול זה. אחרת בודקים את השכן מלמעלה (פער ב-seq2), ואם גם הוא לא מתאים, השכן משמאל (פער ב-seq1).
4. מכיוון שהולכים מהסוף להתחלה, התווים נאספים בסדר הפוך. `"".join(reversed(top))` מחזיר אותם לסדר הנכון.
5. כשיש כמה יישורים עם אותו ציון מקסימלי, סדר הבדיקות (אלכסון, למעלה, שמאלה) קובע איזה מהם נקבל. כלים אמיתיים מאפשרים לבחור את המדיניות הזו, או מחזירים את כל היישורים האופטימליים.

דוגמה 3 — Smith–Waterman: יישור מקומי בשינוי של שלוש שורות

אותו אלגוריתם, עם שני שינויים קטנים, מוצא את הקטע הדומה ביותר בתוך רצפים ארוכים: השורה והעמודה הראשונות

נשארות 0, וציון של תא לעולם לא יורד מתחת ל-0. התשובה היא התא עם הציון הגבוה ביותר בכל המטריצה.

```python
def local_alignment_score(seq1, seq2, match=2, mismatch=-1, gap=-2):
    n, m = len(seq1), len(seq2)
    H = [[0] * (m + 1) for _ in range(n + 1)]      # first row and column stay 0
    best, best_cell = 0, (0, 0)
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            s = match if seq1[i - 1] == seq2[j - 1] else mismatch
            H[i][j] = max(0,
                          H[i - 1][j - 1] + s,
                          H[i - 1][j] + gap,
                          H[i][j - 1] + gap)
            if H[i][j] > best:
                best, best_cell = H[i][j], (i, j)
    return best, best_cell

seq1 = "TTTTGATTACATTTT"
seq2 = "CCGATTACACC"
score, (i, j) = local_alignment_score(seq1, seq2)
print("best local score:", score)
print("ends after seq1[:%d] = ...%s and seq2[:%d] = ...%s" % (i, seq1[i - 7:i], j,
seq2[j - 7:j]))
```

```
$ python example.py
best local score: 14
ends after seq1[:11] = ...GATTACA and seq2[:9] = ...GATTACA
```

איך זה עובד, שלב אחר שלב

1. ההבדל הראשון: אין לולאות אתחול. מטריצה של אפסים כבר מתאימה, כי ביישור מקומי מותר להתחיל בכל מקום בלי לשלם על פערים.
2. ההבדל השני: `max(0, ...)` . כשהציון היה הופך לשלילי, מתחילים יישור חדש מאפס במקום "לגרור" עונש מקטע לא דומה.
3. ההבדל השלישי: התשובה אינה בפינה, אלא בתא עם הציון הגבוה ביותר בכל המטריצה. לכן שומרים את `best` ואת מיקומו תוך כדי המילוי.
4. הקטע המשותף הוא `GATTACA` : שבע התאמות, כל אחת שווה 2+, ולכן הציון הוא 14. הזנבות השונים (`TTTT` ו- `CC`) לא מורידים את הציון, כי היישור המקומי פשוט מתעלם מהם.
5. ההדפסה מציגה את 7 התווים שלפני תא הסיום בכל רצף (אנחנו יודעים שהיישור כאן באורך 7). במימוש מלא מוצאים את תחילת הקטע ב-traceback, שעוצר בתא הראשון שערכו 0. זהו הבסיס של BLAST ושל חיפוש דומיינים בחלבונים.

7 המשימה שלכם

ממשו את `init_dp_matrix(seq1, seq2, gap_penalty)` , שיוצרת את מטריצת ה-DP ומאתחלת את השורה והעמודה הראשונות. הבדיקות הנסתרות כוללות רצפים ריקים, עונש פער חיובי ועשרוני, מטריצות גדולות, ואת באג

השורות המשותפות.

מודול 11 · אתגר קוד בסגנון LEETCODE

התרגיל: `init_dp_matrix(seq1, seq2, gap_penalty)`

מומחה | Needleman–Wunsch | עונשי פער | מטריצות דו-ממדיות | כ-60 דקות

אלגוריתם Needleman–Wunsch מתחיל ממטריצה `F` עם len(seq1) + 1 שורות ו-len(seq2) + 1 עמודות. השורה הראשונה והעמודה הראשונה מאותחלות בעונשי פער מצטברים, וכל שאר התאים מתחילים ב-0.

ממשו את `init_dp_matrix(seq1, seq2, gap_penalty)` כך שתחזיר **רשימה של רשימות** שבה:

- יש `len(seq1) + 1` שורות, ובכל שורה `len(seq2) + 1` ערכים.
- `F[i][0] = i * gap_penalty` לכל שורה `i`.
- `F[0][j] = j * gap_penalty` לכל עמודה `j`.
- כל שאר התאים שווים ל-`0`, ובפרט `F[0][0] = 0`.
- כל שורה חייבת להיות רשימה **נפרדת** בזיכרון.

דוגמאות

דוגמה 1

קלט: `seq1 = "AC", seq2 = "AGC", gap_penalty = -2`

פלט: `[[0, -2, -4, -6], [-2, 0, 0, 0], [-4, 0, 0, 0]]`

הסבר: 3 שורות (2 + 1) ו-4 עמודות (3 + 1). השורה והעמודה הראשונות הן כפולות של −2.

דוגמה 2

קלט: `seq1 = "", seq2 = "GA", gap_penalty = -1`

פלט: `[[0, -1, -2]]`

הסבר: seq1 ריק, ולכן יש שורה אחת בלבד: השורה של התחילית הריקה.

דוגמה 3

קלט: `seq1 = "A", seq2 = "", gap_penalty = -3`

פלט: `[[0], [-3]]`

אילוצים

- `0 ≤ len(seq1), len(seq2) ≤ 500`
- `gap_penalty` הוא `int` או `float` (בדרך כלל שלילי)
- יש להחזיר `list` של `list`, לא tuples ולא מערך NumPy

רמזים

- `[[0] * (m + 1) for _ in range(n + 1)]` יוצר שורות נפרדות.
- שתי לולאות קצרות מספיקות: אחת לעמודה הראשונה ואחת לשורה הראשונה.
- אל תשתמשו ב-`[[0] * (m + 1)] * (n + 1)`: כל השורות יהיו אותה רשימה.
- חשבו על המקרה שבו אחד הרצפים ריק: המטריצה עדיין צריכה שורה אחת או עמודה אחת לפחות.

בדיקות גלויות (הרצה)

אלה הבדיקות שכפתור **הרצה** באתר מריץ. הפתרון חייב לעבור את כולן.

קריאה	תוצאה צפויה
`init_dp_matrix("AC", "AGC", -2)`	`[[0, -2, -4, -6], [-2, 0, 0, 0], [-4, 0, 0, 0]]`
`init_dp_matrix("", "GA", -1)`	`[[0, -1, -2]]`
`init_dp_matrix("A", "", -3)`	`[[0], [-3]]`

בדיקות נסתרות: מקרי קצה (הגשה)

כפתור **הגשה** מריץ גם את הבדיקות האלה. הן נועדו לתפוס קלטים ריקים, מקרי גבול, יעילות על קלט גדול וטיפול בשגיאות. כתבו את הפתרון כך שיעמוד בהן **לפני** שאתם מציצים בטבלה.

קריאה	תוצאה צפויה	מה נבדק
`init_dp_matrix("", "", -1)`	`[[0]]`	קלט ריק
`init_dp_matrix("A", "A", -1)`	`[[0, -1], [-1, 0]]`	תו בודד
`init_dp_matrix("GATTACA", "GCATGCA", -2)`	`[[0, -2, -4, -6, -8, -10, -12, -14], [-2, 0, 0, 0, 0, 0, 0, 0], [-4, 0, 0, 0, 0, 0, 0, 0], [-6, 0, 0, 0, 0, 0, 0, 0], [-8, 0, 0, 0, 0, 0, 0, 0], [-10, 0, 0, 0, 0, 0, 0, 0], [-12, 0, 0, 0, 0, 0, 0, 0], [-14, 0, 0, 0, 0, 0, 0, 0]]`	
`init_dp_matrix("AC", "G", 2)`	`[[0, 2], [2, 0], [4, 0]]`	תו בודד; עונש פער חיובי
`init_dp_matrix("AG", "CT", -0.5)`	`[[0, -0.5, -1], [-0.5, 0, 0], [-1, 0, 0]]`	עונש פער עשרוני
`init_dp_matrix("A" * 60, "CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC", -1)`	מטריצה 61 × 46: שורה 0 היא [0, -1, -2, -3, -4, -5, …], עמודה 0 היא [0, -1, -2, -3, …], שאר התאים 0	מטריצה גדולה: שורות נפרדות

קוד התחלתי

כך נראה העורך כשפותחים את המודול. ממשו את גוף הפונקציה במקום `pass`.

```python
def init_dp_matrix(seq1, seq2, gap_penalty):
    """
    Create the (len(seq1) + 1) x (len(seq2) + 1) Needleman-Wunsch matrix.
    F[i][0] = i * gap_penalty, F[0][j] = j * gap_penalty, everything else 0.

    >>> init_dp_matrix("AC", "AGC", -2)
    [[0, -2, -4, -6], [-2, 0, 0, 0], [-4, 0, 0, 0]]
    """
    # Write your code here
    pass
```

🔑 פתרון עיון מלא: `init_dp_matrix`

הפתרון שעובר את כל הבדיקות, הגלויות והנסתרות. מספרי השורות בהסבר מתייחסים לקוד שלמעלה.

```python
def init_dp_matrix(seq1, seq2, gap_penalty):
    n, m = len(seq1), len(seq2)
    # A comprehension builds n + 1 independent rows (no shared references)
    matrix = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        matrix[i][0] = i * gap_penalty
    for j in range(m + 1):
        matrix[0][j] = j * gap_penalty
    return matrix
```

הסבר שורה אחר שורה

שורה 2 — `n` ו-`m` הם אורכי שני הרצפים. למטריצה יש n + 1 שורות ו-m + 1 עמודות: השורה והעמודה הנוספות מייצגות את התחילית הריקה.

שורות 4–3 — **יצירת המטריצה נכון.** רשימת ההבנה מחשבת את `[0] * (m + 1)` מחדש בכל איטרציה, ולכן כל שורה היא אובייקט נפרד בזיכרון. הכתיב המפתה `[[0] * (m + 1)] * (n + 1)` היה יוצר n + 1 הפניות לאותה רשימה, וכל כתיבה לתא הייתה משנה את כל השורות; הבדיקות הנסתרות תופסות בדיוק את זה.

שורות 6–5 — **העמודה הראשונה.** לכל שורה `i` (כולל 0), `matrix[i][0] = i * gap_penalty`: יישור של תחילית באורך i של `seq1` מול מחרוזת ריקה דורש i פערים. עבור `i = 0` מתקבל 0, ולכן אין צורך להתחיל מ-1.

שורות 8–7 — **השורה הראשונה.** באופן סימטרי, `matrix[0][j] = j * gap_penalty`. הפינה `matrix[0][0]` נכתבת פעמיים, ובשני המקרים מקבלת 0. הכפל עובד גם עם `gap_penalty` עשרוני או חיובי.

שורה 9 — מחזירים רשימה של רשימות, כנדרש (לא tuples ולא מערך NumPy). כל שאר התאים נשארים 0, מוכנים לשלב המילוי.

⌘ למה זה הפתרון האופטימלי

יצירת המטריצה היא O(n·m) בזמן ובזיכרון, והאתחול עצמו מוסיף רק O(n + m). זה המחיר ההכרחי של Needleman–Wunsch, שגם המילוי המלא שלו הוא O(n·m), O(1) לכל תא. אם צריך רק את הציון, אפשר לשמור שתי שורות בלבד ולרדת ל-O(m) זיכרון.

💡 איך הפתרון עומד במקרי הקצה הנסתרים

`[[0]]` → `""`, `""`; `[[0], [-3]]` → `""`, `"A"`; `[[0, -1, -2]]` → שורה אחת → `"GA"`, `""`; עונש חיובי 2 → `[[0, 2], [2, 0], [4, 0]]`; עונש עשרוני −0.5 → ערכים עשרוניים; מטריצה 61 × 46 נבנית בלי שורות משותפות.

נספח: האתר האינטראקטיבי

הקורס הוא אתר סטטי (HTML, CSS ו-JavaScript) ללא שרת וללא שלב בנייה, מוכן ל-GitHub Pages. סביבת Python רצה ב-Web Worker בעזרת Pyodide (CPython שקומפל ל-WebAssembly); הטעינה הראשונה מורידה כ-10 MB מ-CDN והדפדפן שומר אותם במטמון. דפדפנים חוסמים Workers בפרוטוקול `file://`, ולכן מריצים את התיקייה מעל HTTP:

```bash
cd compbio-lab
python3 -m http.server 8000
# open http://localhost:8000
```

1 ארכיטקטורה

קובץ	תפקיד
`index.html`	שלד האפליקציה: סרגל צד, סרגל עליון, סביבת עבודה בשני חלונות (שיעור/תרגיל מול עורך/מסוף)
`css/styles.css`, `css/visuals.css`	עיצוב, פריסה, ערכת העורך, טיפוגרפיה של השיעורים ורכיבי המעבדות החזותיות
`js/curriculum*.js`	כל המודולים כנתונים: שיעור (HTML), בעיה, קוד התחלתי, פתרון עיון ובדיקות. מודולים 1–4, 5–7 ו-8–11 בשלושה קבצים, עם הקוד הגנטי המשותף
`js/visuals*.js`	המעבדות החזותיות (פונקציית יצירה אחת לכל מעבדה), מחוברות ל-`<div data-viz="…">` בשיעור
`js/pyworker.js`, `js/runner.js`	טעינת Pyodide ב-Worker, הרצת מערכת הבדיקות, מגבלת זמן של 8 שניות (הורג ומפעיל מחדש בלולאה אינסופית)
`js/editor.js`, `js/console.js`	עטיפת CodeMirror 5 (היסטוריית ביטול לכל מודול, עם textarea כגיבוי) והצגת תוצאות הבדיקות, מעקבי שגיאות ופלט
`js/app.js`, `js/storage.js`	ניווט (`#/m1`…), זרימת הרצה/הגשה, חלונות שניתן לשנות את גודלם, קיצורים; שמירת קוד והתקדמות ב-localStorage

2 מבנה מקרה בדיקה

הוספת מודול היא הוספת אובייקט עם `id`, `title`, `lesson`, `problem`, `starter`, `solution`, `fnName` ו-`tests`. מקרי הבדיקה נראים כך:

```javascript
{ args: ["ACGT"], expected: "ACGU" }                  // exact match (type-checked)
{ args: ["GGCC"], expected: 100.0, tol: 1e-3 }       // float comparison
{ args: ["AAAA", 2], expected: { AA: 3 } }           // dict (any mapping: dict, Counter, defaultdict)
{ args: ["AC", "ACG"], raises: "ValueError" }        // expects an exception
{ args: [...], expected: ..., hidden: true }         // only runs on Submit
```

3 קיצורי מקלדת

פעולה	MACOS	WINDOWS/LINUX
הרצת הבדיקות הגלויות	⌘ ↵	Ctrl ↵
הגשה (כולל הבדיקות הנסתרות)	⌘ ⇧ ↵	Ctrl ⇧ ↵
הפיכת שורה להערה	⌘ /	Ctrl /

סיום: מה הלאה?

השלמתם את כל 11 המודולים: משעתוק של מחרוזת ועד מטריצת תכנון דינמי. הכלים שבידיכם, מחרוזות, מילונים, חלונות נעים, פונקציות ניקוד ומטריצות, הם בדיוק אבני הבניין של הכלים המקצועיים שהוזכרו לאורך הדרך.

- **Rosalind, Bioinformatics Stronghold**: הבעיות שעליהן מבוססים התרגילים (DNA, RNA, REVC, GC, HAMM, SUBS, PROT, ORF, CONS, KMER ועוד) ממשיכות משם, עם עשרות אתגרים נוספים. זכרו שב-Rosalind סופרים עמדות מ-1.
- **Biopython**: `Bio.Seq.Seq` , `Bio.SeqIO` ו-`Bio.Align.PairwiseAligner` עושים מאחורי הקלעים בדיוק את מה שכתבתם, על קבצים אמיתיים.
- **יישור מלא**: השלימו את Needleman–Wunsch ו-Smith–Waterman מהדוגמאות במודול 11 לפונקציה אחת עם traceback ועונש פער אפיני.
- **נתונים אמיתיים**: הורידו גנום חיידקי מ-NCBI בפורמט FASTA, חשבו תכולת GC ו-GC skew (מודול 2), מצאו ORFs ארוכים בשש מסגרות (מודול 6) וספרו k-mers (מודול 9). זו עבודת עוזר מחקר אמיתית.

החוברת נבנתה אוטומטית מקוד המקור של הקורס; הפלטים של דוגמאות הקוד התקבלו מהרצה בפועל ב-Python 3.